Starling: the complete project guide

Read this and you can explain the whole project, screen by screen, even if you have never seen it run.

Every screenshot in this guide was taken from the running system (`docs/img/`). Under each one you will find **what you are looking at**, **what it proves**, and **the technical detail** behind it. Sections 5 to 9 go deeper into the ideas, the numbers, the real-world deployment and how it scales. Section 10 is a bank of interview questions with answers.

Contents

1. The project in 60 seconds
2. Words you need
3. The system at a glance
4. Screen-by-screen tour
5. 4.1 Live monitor
6. 4.2 The 3D warehouse
7. 4.3 Inside the system
8. 4.4 What if
9. 4.5 How it works
10. 4.6 Present and Autopilot
11. 4.7 Engineer view
12. The ideas in depth (C1 to C6)
13. Technical reference
14. Numbers you can quote, and the honest limits
15. From demo to a real warehouse with CCTV
16. Scalability and the cloud
17. Interview questions and answers
18. Repository map and how to run it

1. The project in 60 seconds

The problem. A warehouse has many cameras. A normal system streams every camera's video to one server, which finds the people and works out who is who. That has three weaknesses: (1) the server is a single point of failure, so if it or its network dies, every camera goes blind; (2) it needs enormous bandwidth and stores everyone's footage in one place, a privacy risk; (3) when a person walks where no camera can see, such as between tall racks, the system simply loses them.

The idea. *Starling* removes the server. Every camera gets its own small computer (a **node**). The node finds people in its own pictures, works out where they stand on the floor plan, and writes a tiny **signed note** (about 500 bytes, never video) such as: "*Camera 2 saw someone at (8.5 m, 8.4 m) at 07:02:13, wearing a purple top.*" It passes the note only to its two neighbouring cameras, who pass on what the others are missing. Soon every camera holds the same set of notes. Each one independently works out "who is where" from those notes using the same deterministic recipe, so they all reach the **same answer with nobody in charge**.

What makes it more than object detection. Detecting a person is only step 1 of 6. Starling also places them on the floor, recognises who they are, shares signed notes, makes every camera agree, and **reasons**: it finds people no camera can see (from what working cameras did *not* see), catches a camera that lies, and keeps working when the network is cut or a camera dies.

The one sentence to remember:

> *Starling is a camera network with no central server: each camera thinks for itself, they share only small signed facts, and together they still know where everyone is, even someone no camera can see.*

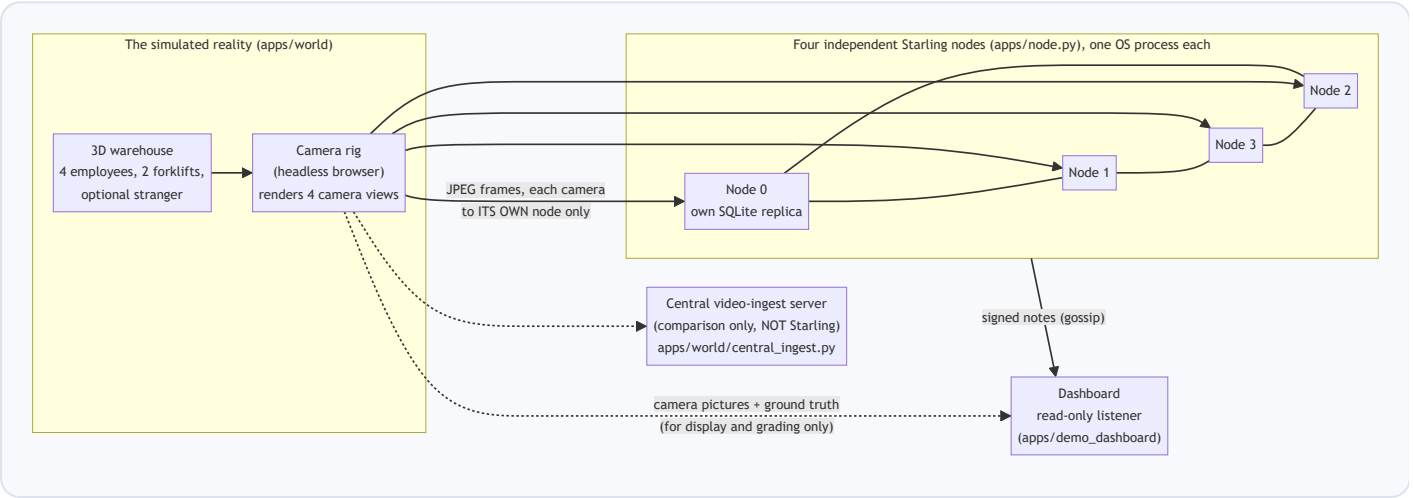
---

2. Words you need

Word	Plain meaning	Where it lives in the code
Node	One camera plus its own small computer. An **operating-system process** with its own database and its own network socket (never a thread).	`apps/node.py`
Claim ("note")	A small signed record: who saw what, where, when, with an appearance signature. The only thing that crosses the network about people.	`packages/starling_proto/starling.proto` (`IdentityClaim`)
Gossip	Each node tells only its configured neighbours; they pass it on. No broadcast, no full mesh.	`packages/starling_net/gossip.py`
Anti-entropy	"Tell me what you have; I will send what you miss." How nodes catch up after a gap.	`packages/starling_net/anti_entropy.py`
CRDT	A data type whose copies can be merged in any order and always agree. Starling's set of claims is a **grow-only set**.	`packages/starling_crdt/claims.py`
Resolver	The pure function that turns the set of claims into "who is where". Same claims in, same answer out.	`packages/starling_crdt/resolver.py`
Fork	Two claim chains tie one identity to places that cannot both be true. Left **open** for a human, never settled by "highest score".	`packages/starling_crdt/forks.py`
Attestation	A signed statement from a healthy camera: "my view is clear and I saw nobody."	`packages/starling_attest/attestation.py`

Word	Plain meaning	Where it lives in the code
Negative evidence	Using attestations to *rule out* where someone could be.	`packages/starling_attest/negative_evidence.py`
Reputation / trust	How much other nodes believe a node, lowered when its claims fail checks.	`packages/starling_consensus/reputation.py`
Navmesh / reachability	The walkable floor plan and "could a person get from A to B in that time?"	`packages/starling_geometry/`
HLC	Hybrid logical clock: a timestamp that is both real-time-ish and causally ordered, so every node orders claims identically.	`packages/starling_net/hlc.py`
Capability token	A signed pass stating the **purpose** of a question (safety, incident, audit). "Productivity" is refused.	`packages/starling_query/capability.py`
Replica	One node's own copy of all claims (its SQLite file).	`packages/starling_store/identity_store.py`

3. The system at a glance



Three rules the whole design obeys (they are in `CLAUDE.md` and enforced by tests):

1. A node is an **OS process** with its **own** database. No module reads another node's data.
2. **Raw video never crosses the wire**, only claims, attestations, reputation updates and queries.
3. Gossip goes to a **configured neighbour set**. A full mesh would be a coordinator in disguise.

What is real and what is simulated

Real (the actual Starling code running on pixels)	Simulated
Each node is a separate process with its own database and signed network sockets	The warehouse and its people are a 3D scene
YOLO person detection on rendered frames; ByteTrack tracking	The four "cameras" are virtual cameras rendered by a headless browser
Calibrated floor homography; appearance matching; gate verification	The "network cut" button drops messages in software rather than unplugging a cable
CRDT merge, resolver, negative evidence, reputation, queries	
The comparison "central server" is a real process receiving the real streams	

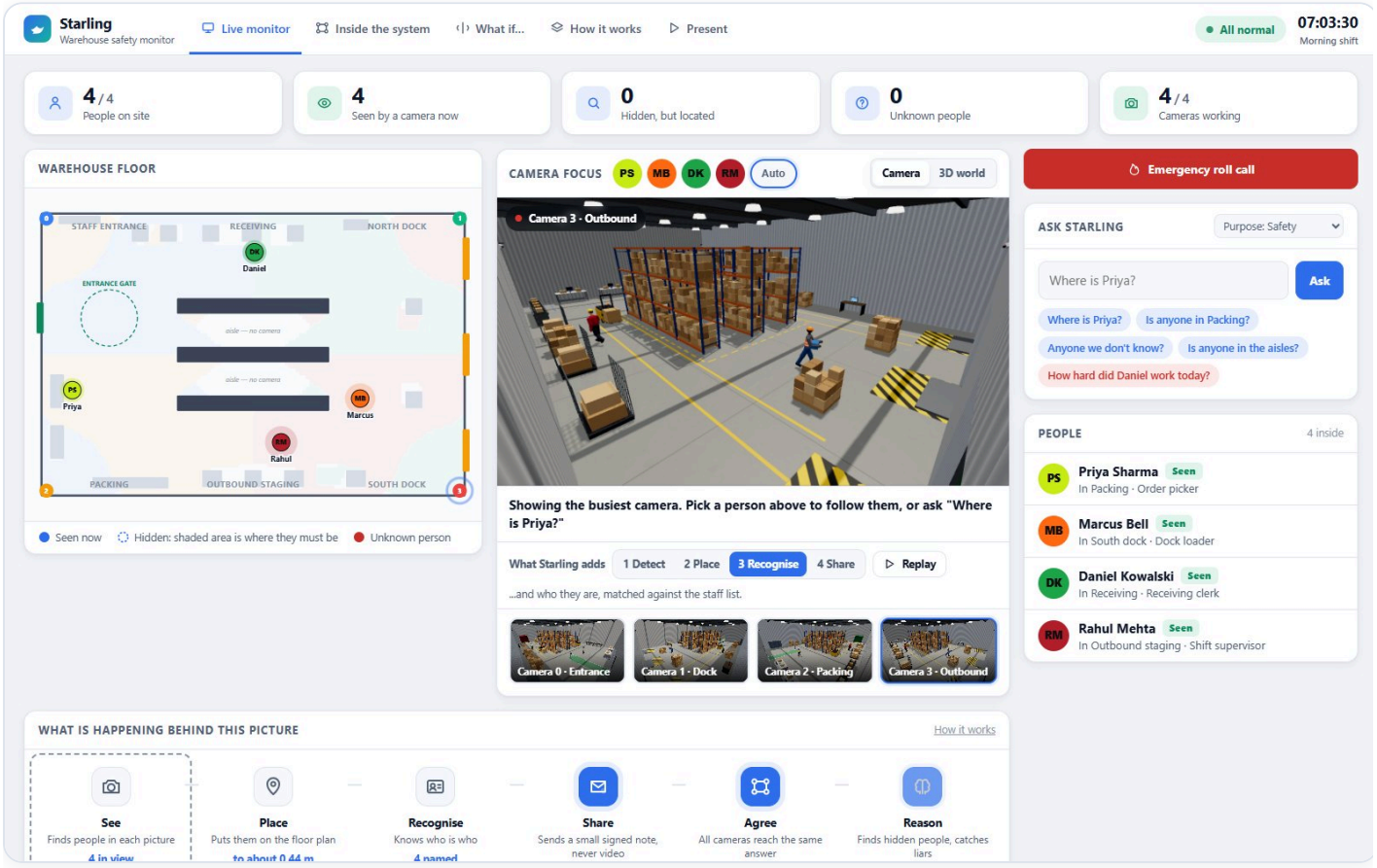
Ports (all on `127.0.0.1`): dashboard **8765** · 3D world **8770** · node gossip **5555 to 5558** · node control **6555 to 6558** (gossip port + 1000) · camera frame bus **5570** · central comparison server **7010**.

4. Screen-by-screen tour

All pages share one header with five tabs: **Live monitor · Inside the system · What if… · How it works · Present**, plus a status pill (green "All normal", amber warnings, red alerts) and the shift clock. The dashboard is a **read-only observer**: it listens to gossip like any other peer and never has privileged access (rule 8 of the design).

4.1 Live monitor

URL: `http://127.0.0.1:8765/`. This is what a warehouse manager would watch.

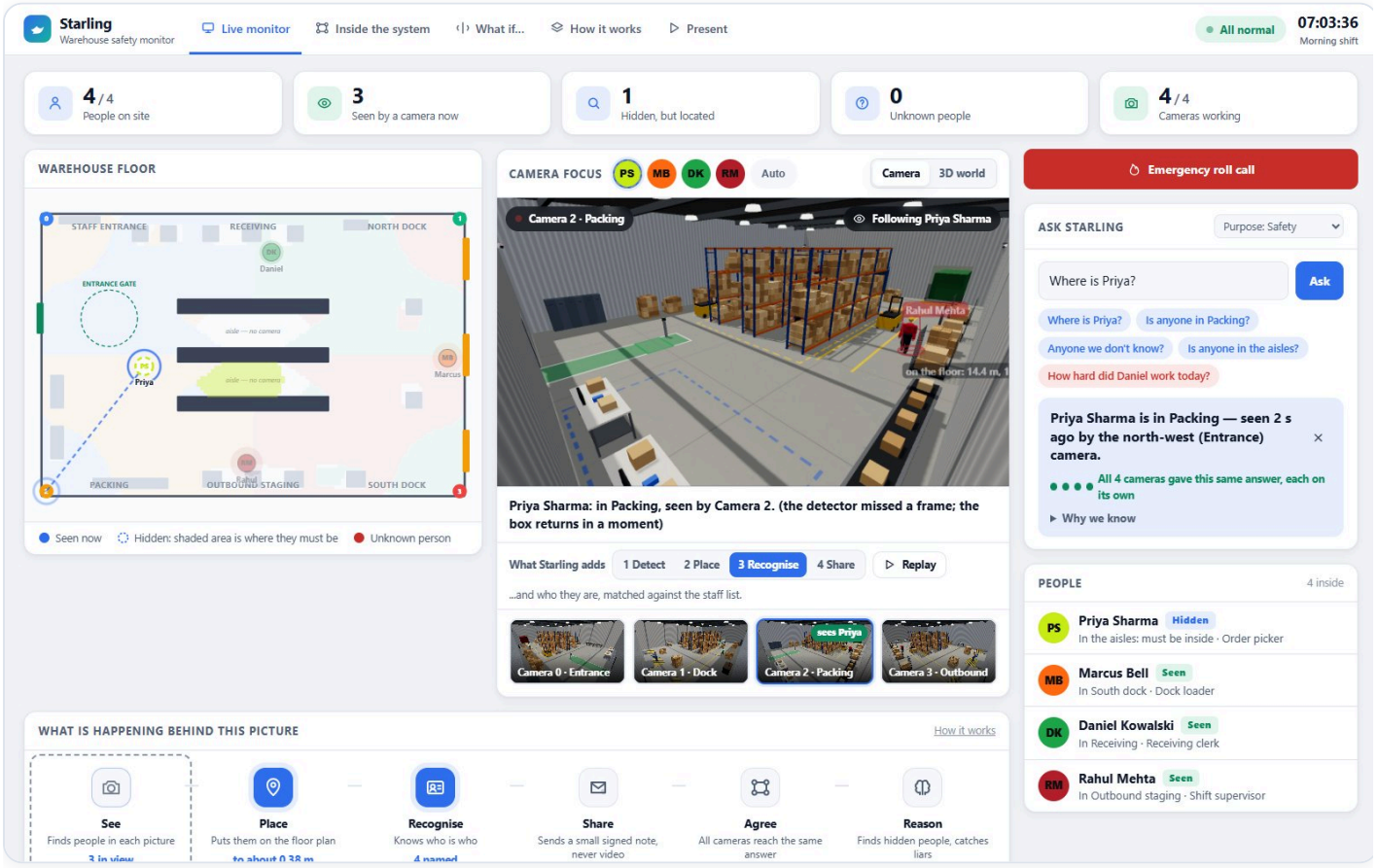


What you see. - **Top row, five tiles:** people on site, how many are seen by a camera right now, how many are hidden but located, unknown people, cameras working. - **Left, "Warehouse floor":** a map. Coloured dots are employees (solid = seen now, dashed ring = hidden). The shaded patch is the area a hidden person must be in. Numbered circles in the corners are the four cameras. - **Middle, "Camera focus":** a live picture from the camera that can see the person you are following (more below). - **Right:** the red **Emergency roll call** button, **Ask Starling**, the **People** list and **Needs attention**. - **Bottom strip, "What is happening behind this picture":** six stages (See, Place, Recognise, Share, Agree, Reason) that light up as real notes flow. A dashed box around **See** says "*Ordinary systems stop here*".

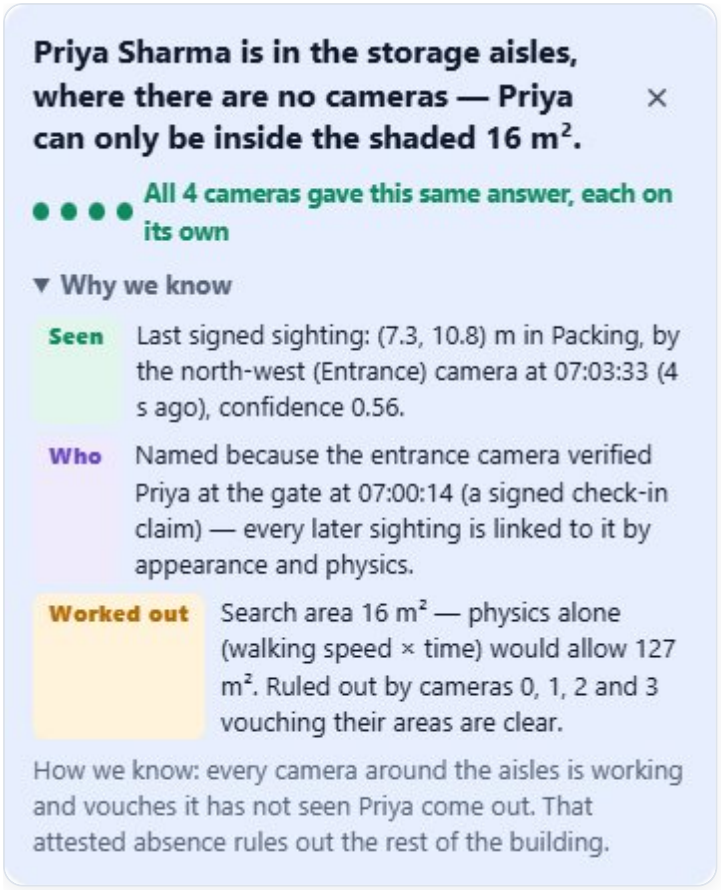
Why it matters. It answers the teacher's first reaction, "isn't this just object detection?" Object detection is the first of six boxes. The strip shows live numbers for each stage: people in view, floor accuracy in metres, people named, kilobits of notes with **0 video**, cameras in step, and hidden people located.

Technical detail. - The page polls `GET /api/state` once a second. That snapshot is built by `DashboardEngine` (`apps/demo_dashboard/engine.py`) from **overheard gossip only**: a passive ZeroMQ subscriber (`apps/dashboard/observer.py`) merges every claim into an in-memory store and runs the same `resolve()` function the nodes run. - "Hidden, but located" comes from a `CandidateBelief` (`starling_attest/negative_evidence.py`): a grid over the navmesh, grown by walking speed each tick and cut by every healthy camera's attested-clear zone. - Names come only from a signed check-in claim made at the **entrance gate**, or from a later recognition by any camera against the enrolled staff list (`starling_perception/registry.py`).

4.2 Ask Starling



What you see. The question "Where is Priya?" answered in one sentence. Beneath it, a row of four green dots: **all four cameras gave this same answer, each on its own.**



"Why we know" splits the answer into labelled evidence: **Seen** (a signed sighting), **Camera vouches** (a healthy camera attested its zone clear), **Worked out** (inferred), **Who** (how the name was established), and the network state. This is the *calibrated* answer of contribution C5: it never mixes what was observed with what was guessed.

Technical detail. - `POST /api/ask` (`apps/demo_dashboard/showcase.py: Showcase.answer`) understands: person, area, "anyone we don't know", arrival time and roll call. Each intent builds a structured answer with an evidence list. - The four green dots come from `GET /api/ask_cameras`: the dashboard asks **each node** `GET /replica` on its own control port. A node resolves *its own* SQLite replica over a pinned media-time window and returns a **fingerprint** (SHA-256 prefix) of the resulting assignment. Equal fingerprints mean equal answers. Claims are resolved in their canonical wire form, because a node holds its own claims at full float precision while peers hold the gossiped float32 form (a bug found and fixed while building this; regression test in `tests/test_showcase.py`). - The node itself verifies the caller's **capability token** (`_check_capability` in `apps/node.py`) before answering.



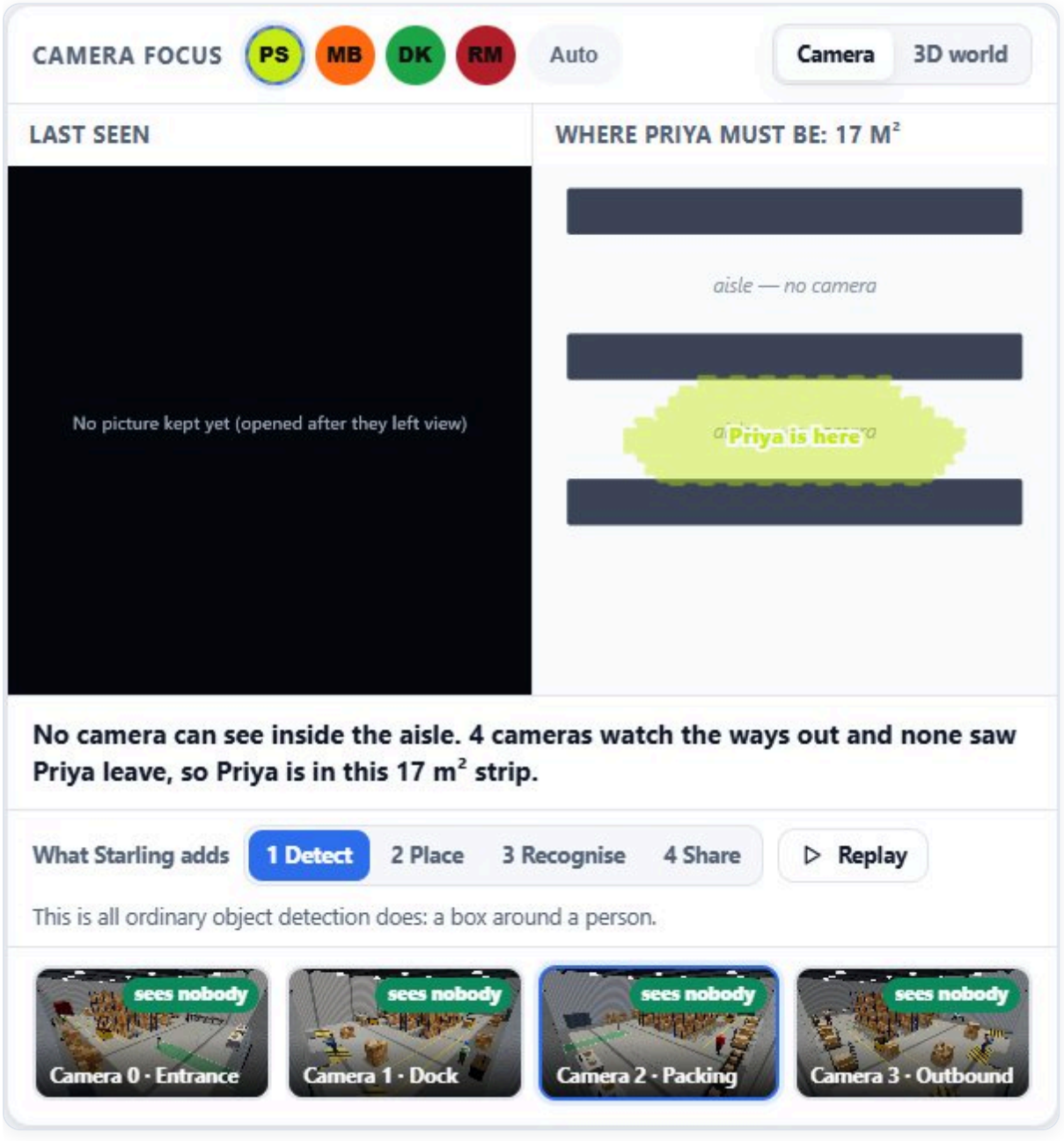
"Is anyone in Packing?" A healthy camera covers Packing and has signed that it saw nobody, so Starling can say **"No, it is empty"** and show the signed evidence. If the cameras could not vouch (covered lens, low confidence), it says *"I can't say it's empty"*. The camera-less aisles are **never** declared empty. This is design rule 7: **silence is never evidence of absence**.



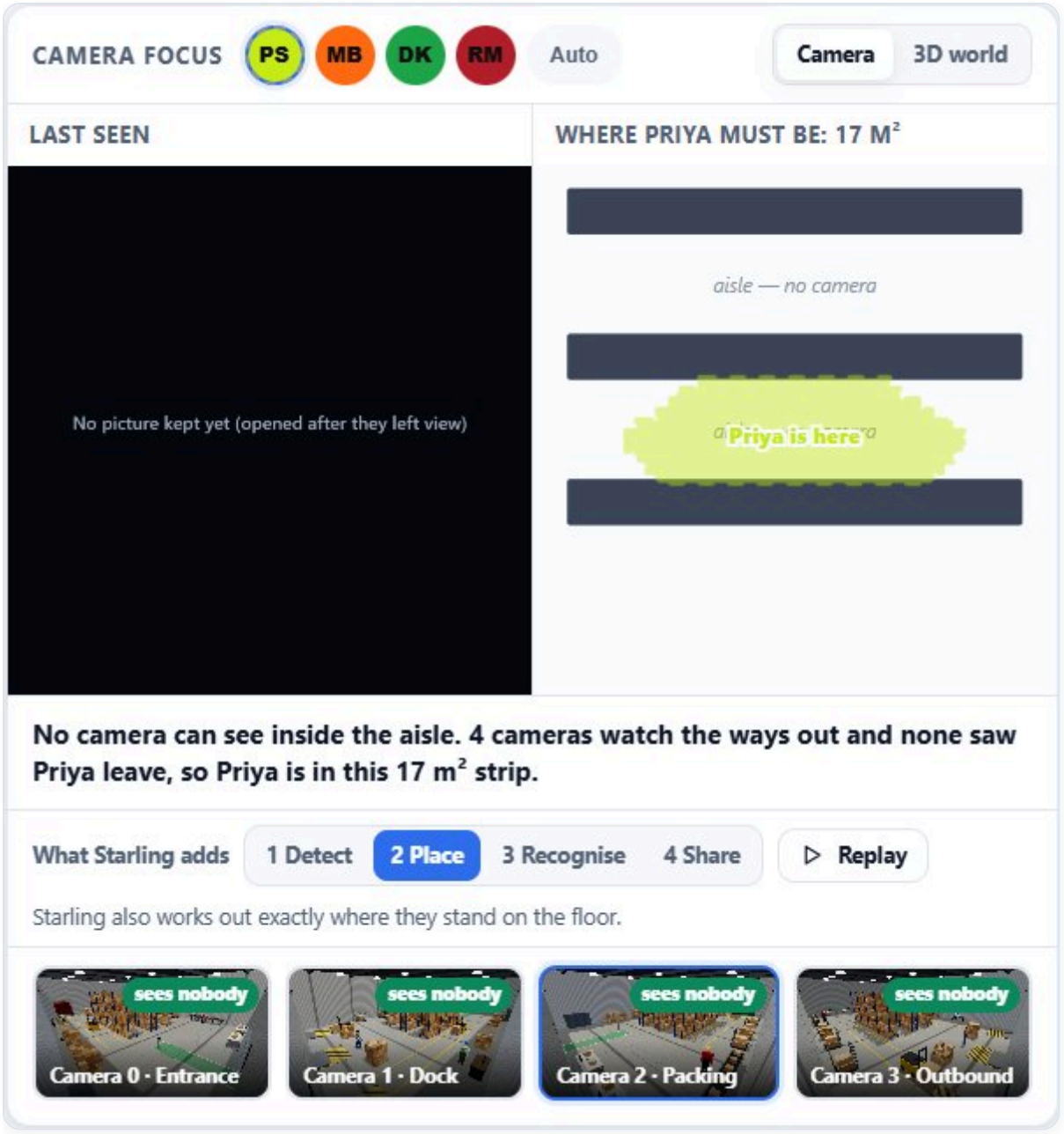
Privacy refusal. Asking how hard Daniel worked is sent with purpose *Productivity*. The system refuses, and the refusal is enforced by **each camera node** verifying a signed token (`starling_query/capability.py` : `ALLOWED_PURPOSES = safety, incident, audit`), not by a policy document. The answer says how many of the 4 cameras refused.

4.2b Camera focus: ask, and the right camera opens

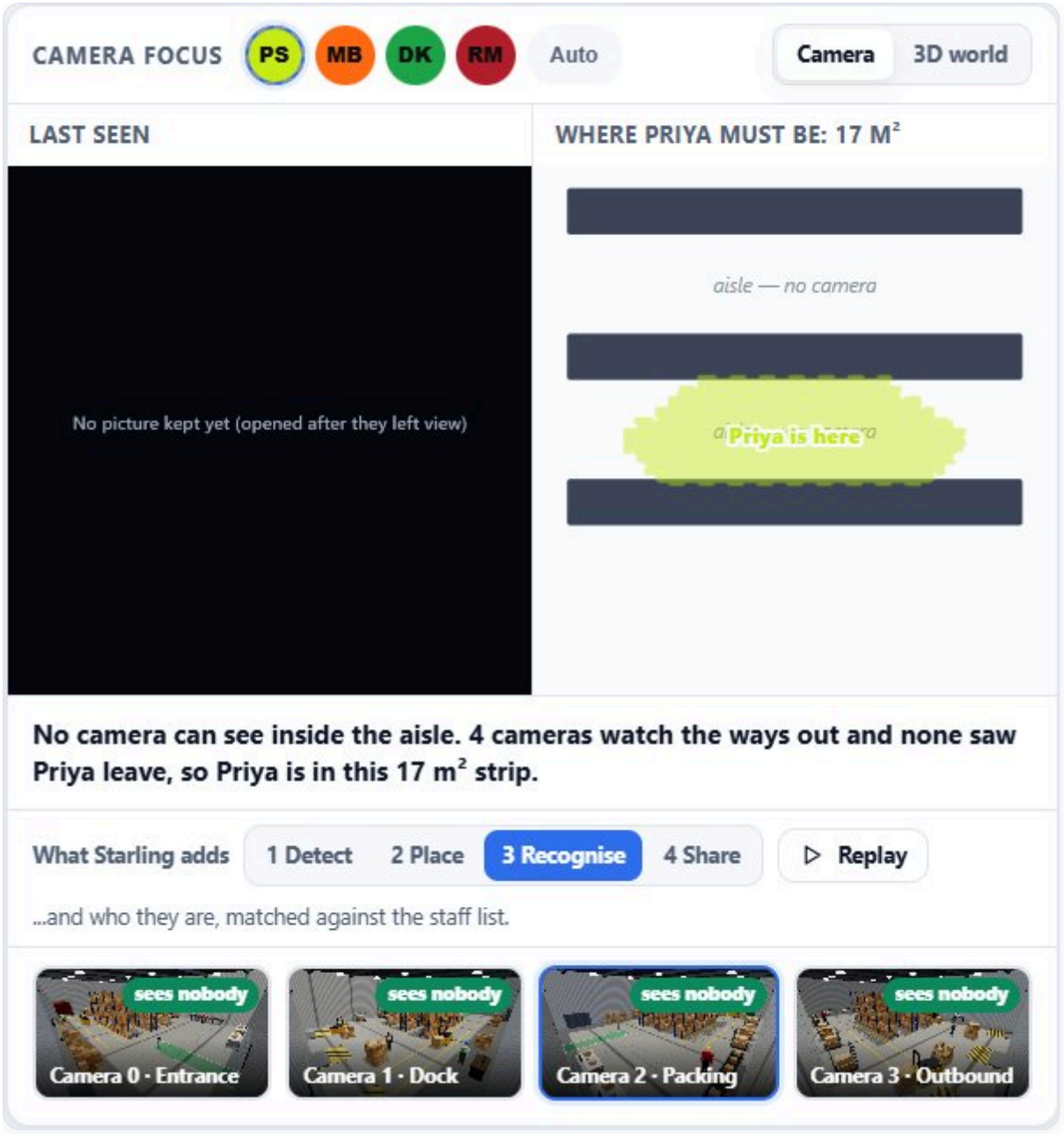
When you ask about a person (or click their avatar), the **Camera focus** panel follows them.



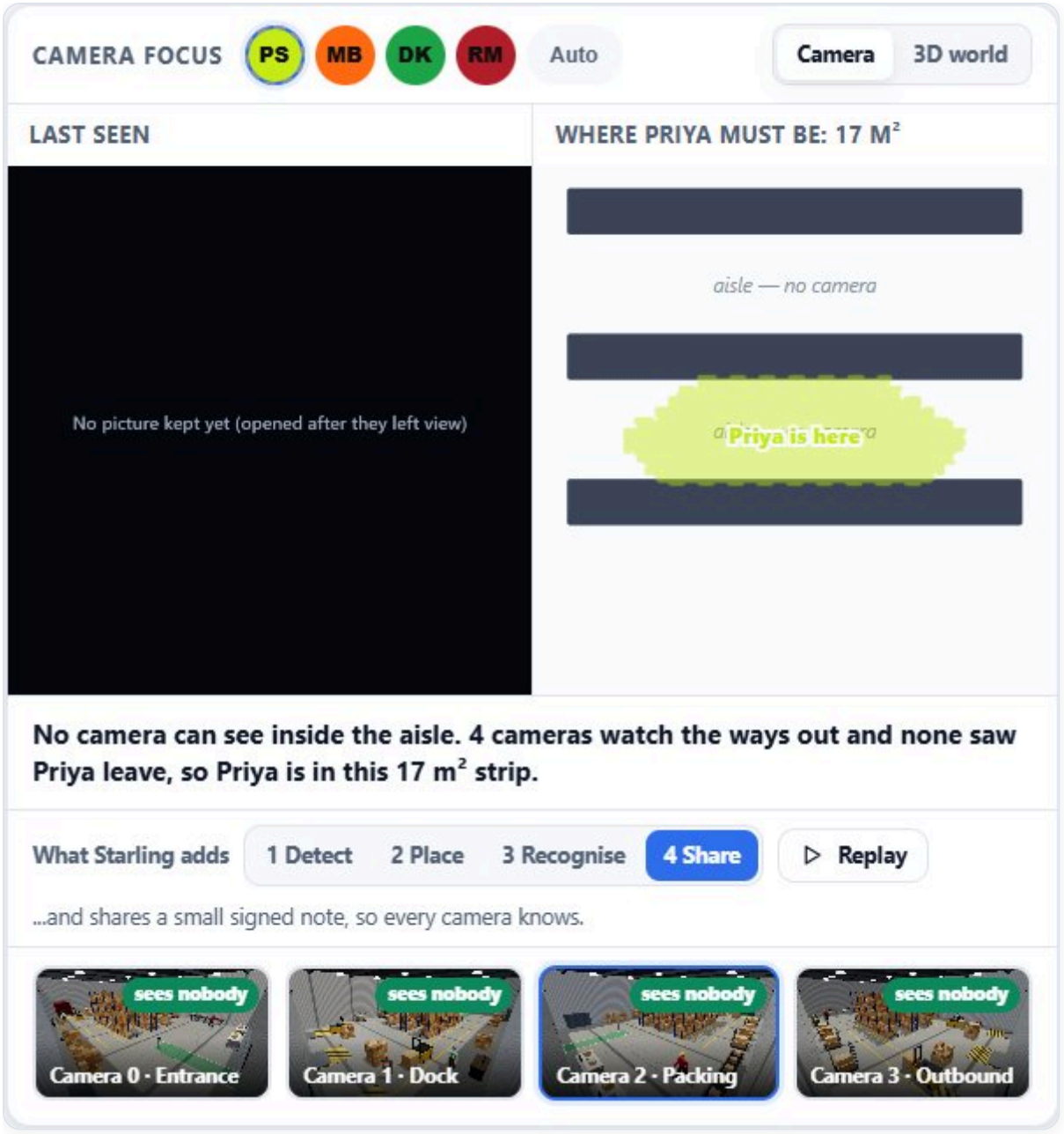
Layer 1, "Detect". A plain box around every person with a confidence, such as "person 87%". **This is all that ordinary object detection gives you.** The panel is zoomed onto the followed person.



Layer 2, "Place". Starling adds the person's **exact floor position** ("on the floor: 14.2 m, 8.1 m") and a footprint ellipse under the feet. A single camera's pixels have been converted to metres on the floor plan.

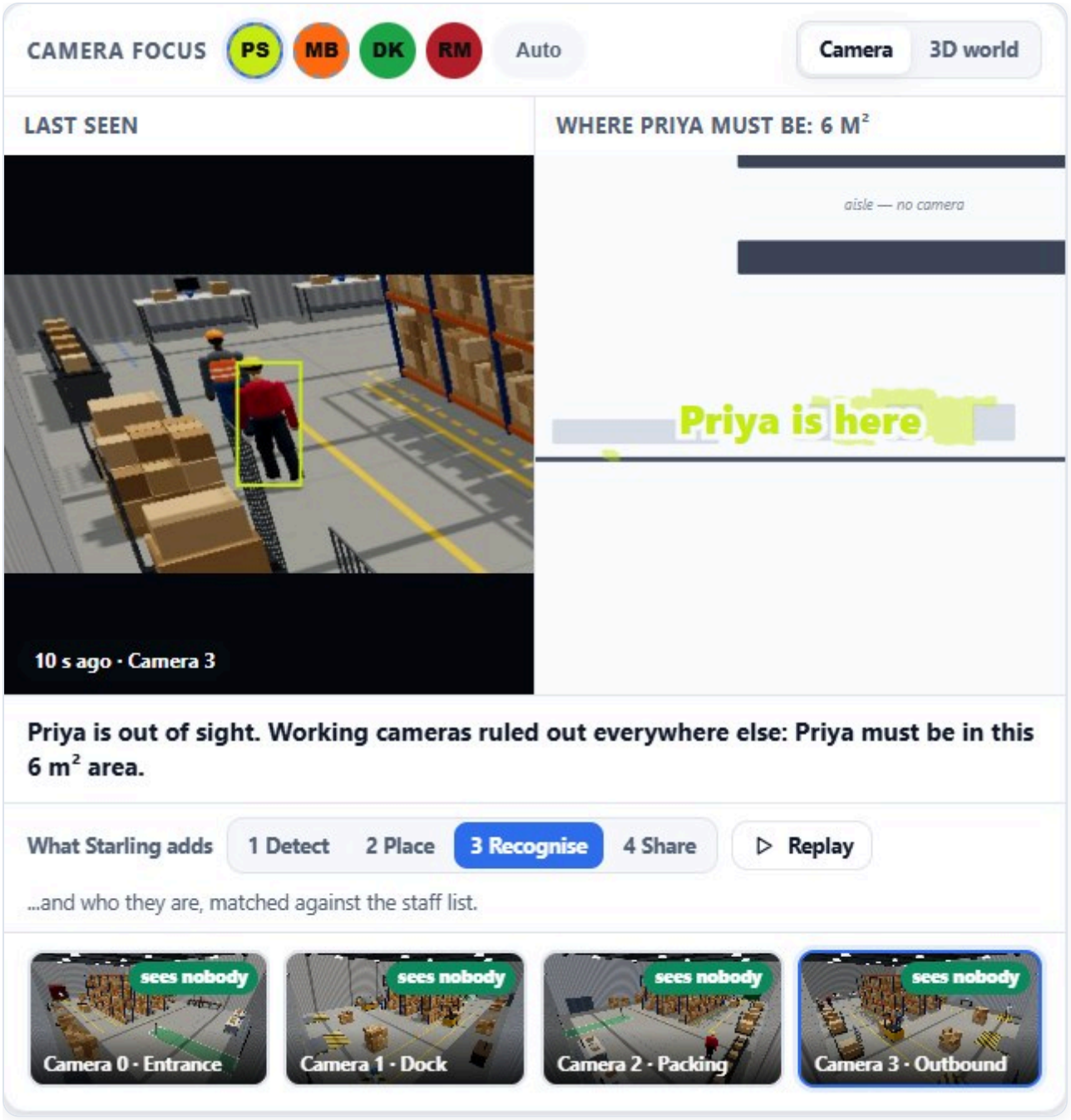


Layer 3, "Recognise". Boxes are coloured and labelled with **names**, matched against the enrolled staff list. The followed person gets a thick glowing box. Unknown people would be red and labelled *Unknown*.

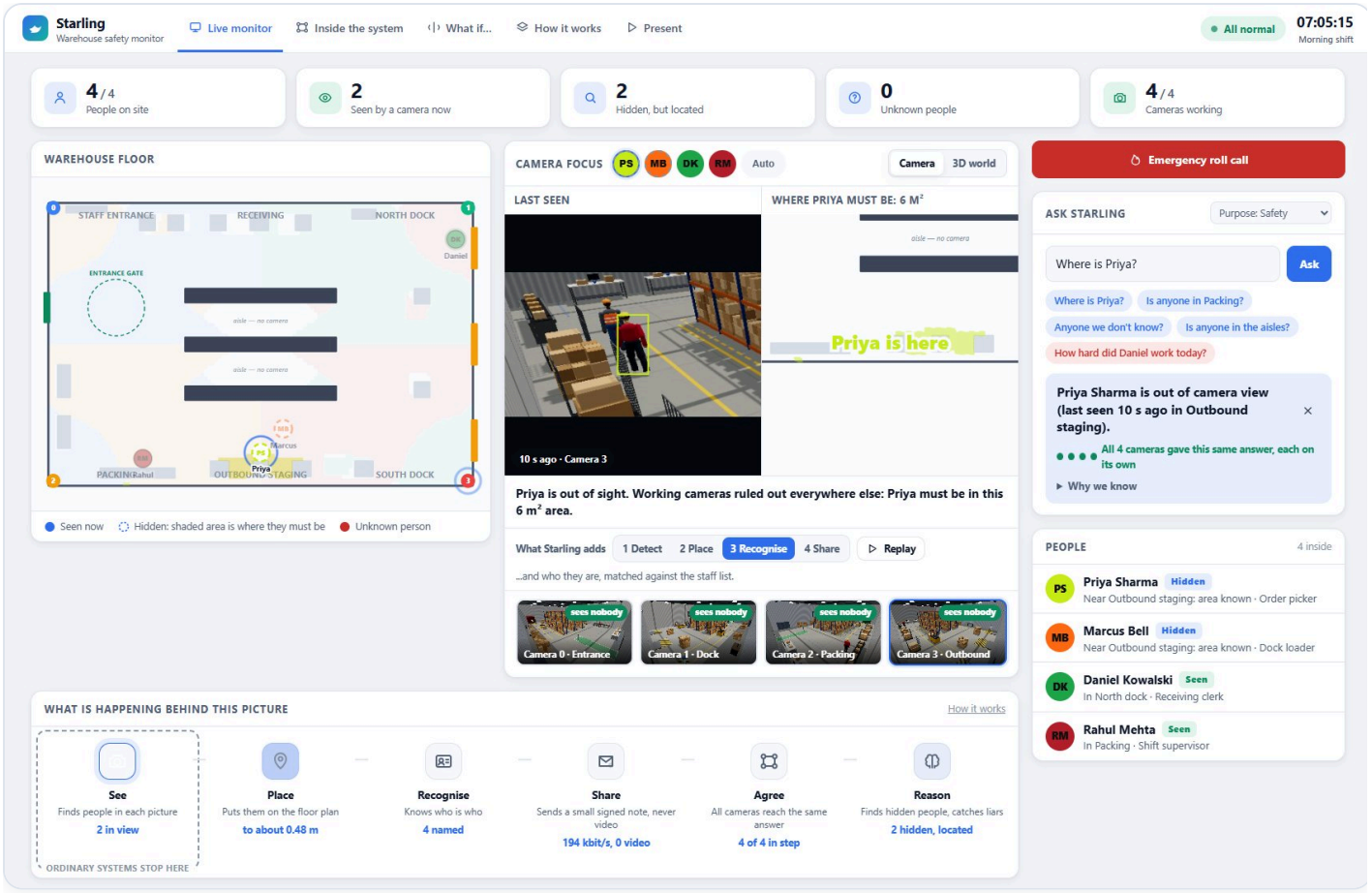


Layer 4, "Share". Each time the node publishes a claim, a ring pulses and a tag reads *"note signed and sent to 2 neighbours"*. The "Replay" button steps through 1 to 4 on a timer, which is the best 10-second way to show a non-technical person what is "more than object detection".

Technical detail. - The picture is `GET /api/camera/{n}.jpg`, a proxy of the world server's camera tap. The boxes come from the node's own `GET /api/node/{n}/perception` (YOLO boxes in normalised coordinates plus floor `x, y`, track id and recognised employee). The browser draws the boxes on a canvas laid over the picture. - **Which camera?** The camera whose floor zone contains the person (`world_view.where()` returns the zone's node). When the person crosses into another zone, a banner reads *"Priya moved: switching to Camera 3"* and the panel hands over. - **Zoom:** the browser scales the picture so the person's box is about a third of the frame height, capped at 3.1x. - **"Note signed and sent" pulse:** triggered when the node's `claims_made` counter increases, i.e. a real claim.

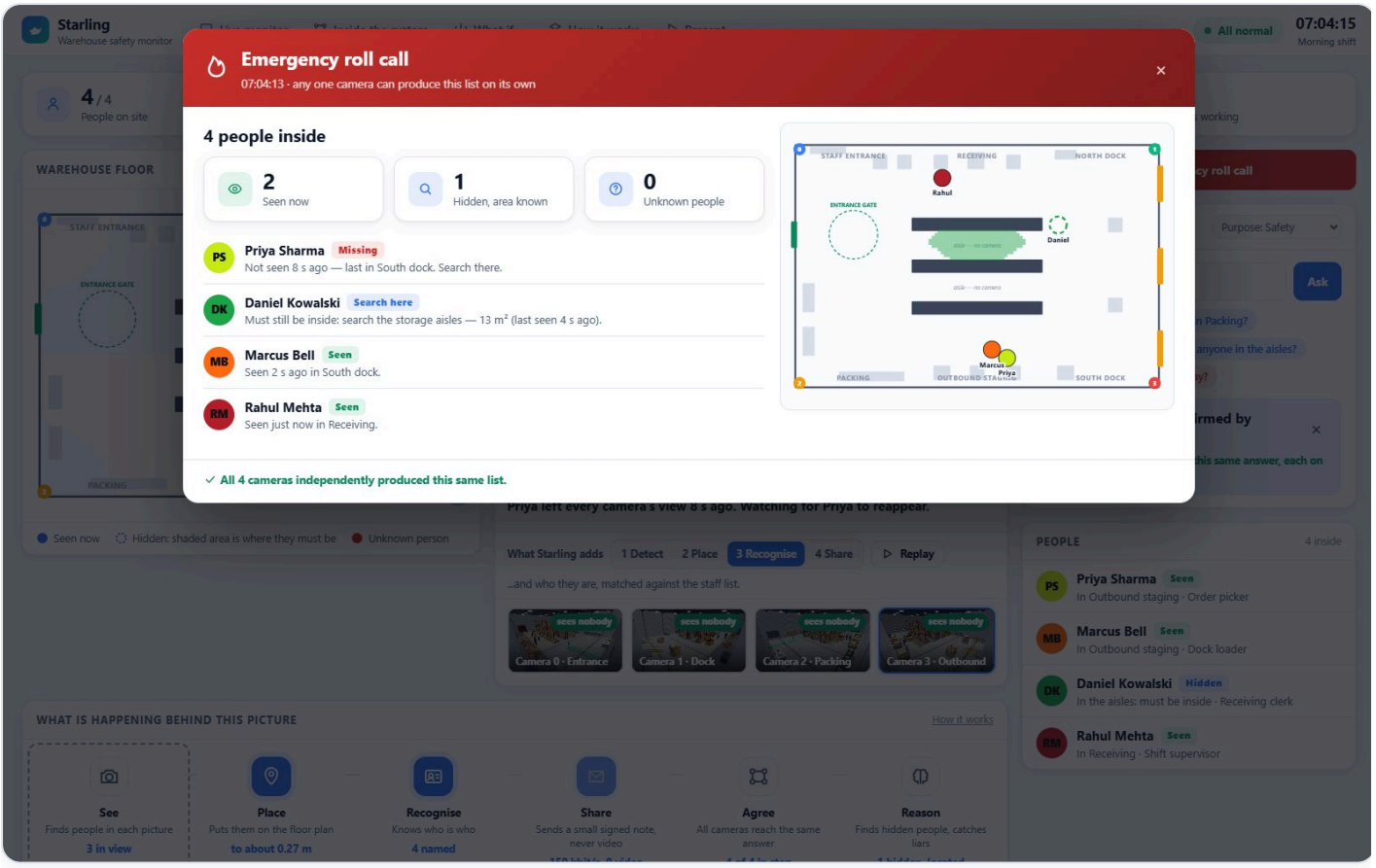


When nobody can see the person (a different view). If the person is in the racking aisles, which have no camera, the panel changes. **Left:** the last picture of them, kept in the browser at the moment they left view, outlined. **Right:** a zoomed map of the strip they must be in. **Bottom:** the four camera thumbnails each wearing a green **"sees nobody"** badge, because each healthy camera has vouched its zone is clear. The sentence under the picture explains: *"No camera can see inside the aisle. 4 cameras watch the ways out and none saw Priya leave, so Priya is in this 17 m² strip."*



Technical detail. The strip is `region.runs` from the engine's `CandidateBelief`. The badges use `region.healthy_nodes`, the cameras whose latest attestation is recent and above `tau_attest = 0.7`. A camera that is covered shows an amber **"cannot vouch"** badge instead, and its zone stays *possible*: silence is not evidence.

4.2c Roll call

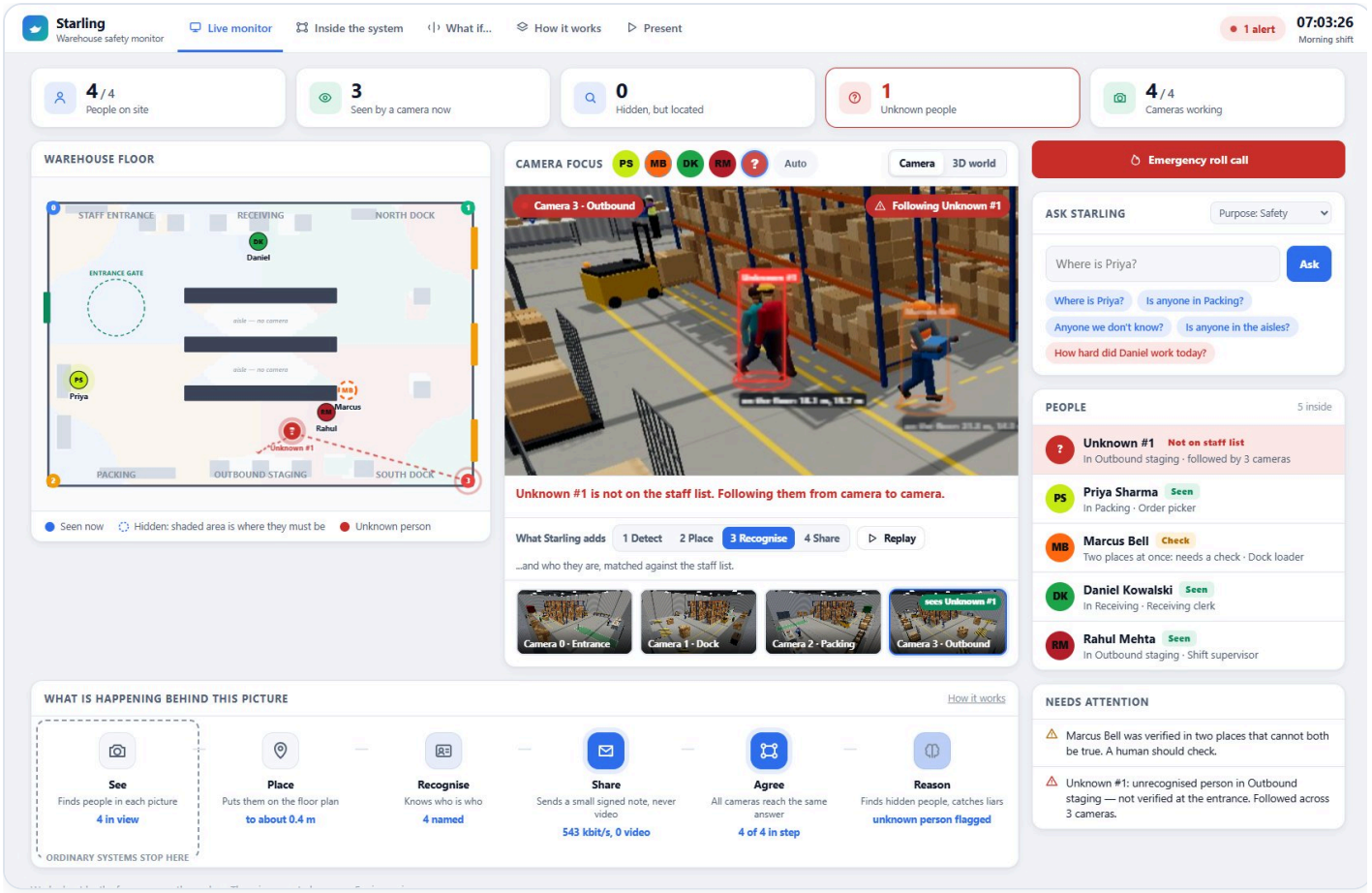


What you see. The red button opens a full-screen list: everyone inside, each with a status (**Seen**, **Search here**, **Missing**, **Unknown**, **Left**), what to do ("search the aisles, 17 m²"), and a map. The footer reads *"All 4 cameras independently produced this same list."*

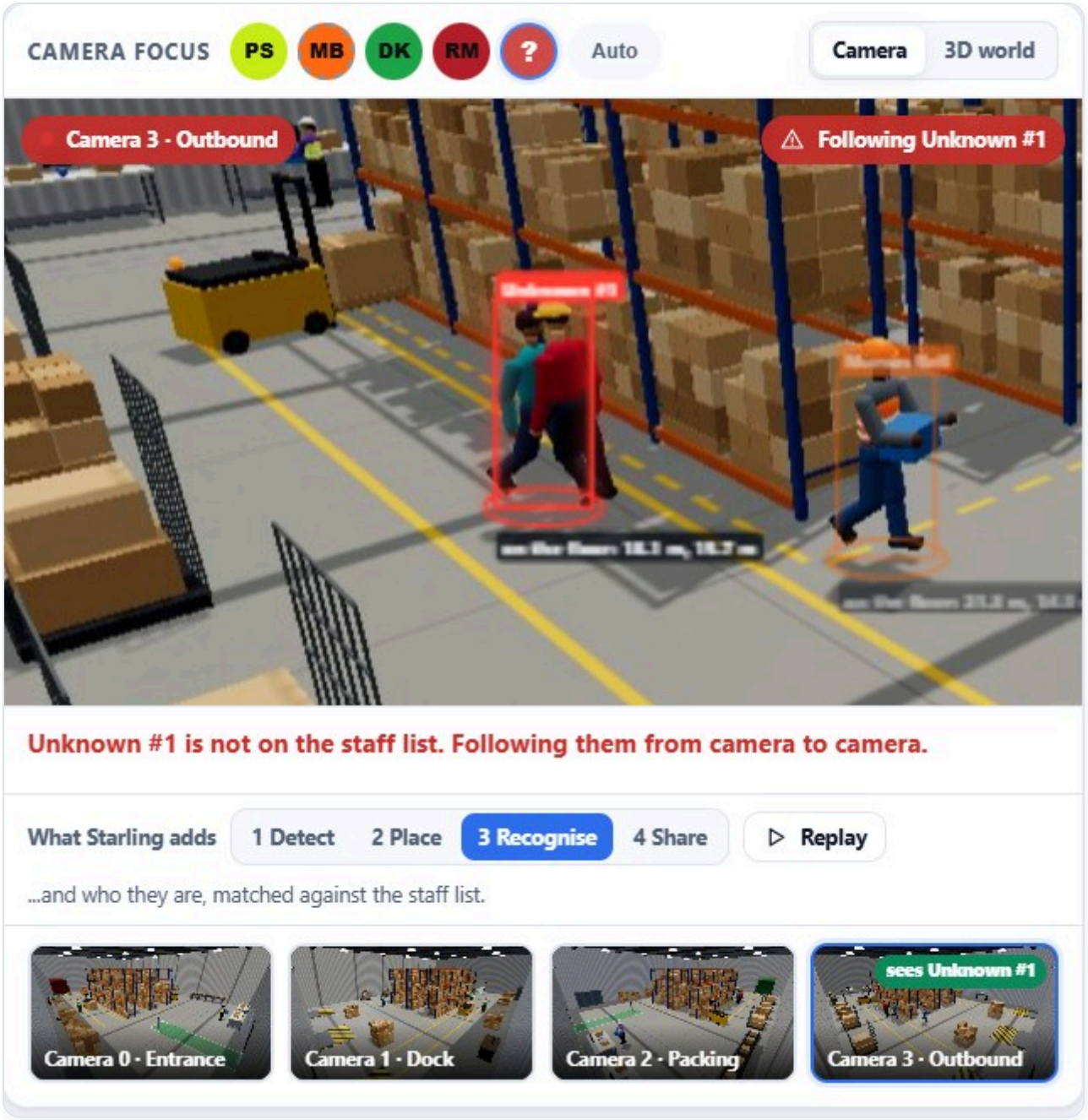
Why it matters. In an evacuation, *any surviving camera can produce this list from its own copy*, with no server.

Technical detail. `GET /api/rollcall` (`Showcase.roll_call`). People "inside" are those with a position; the unknown people are the strangers currently flagged. The agreement line reuses the per-node replica fingerprints.

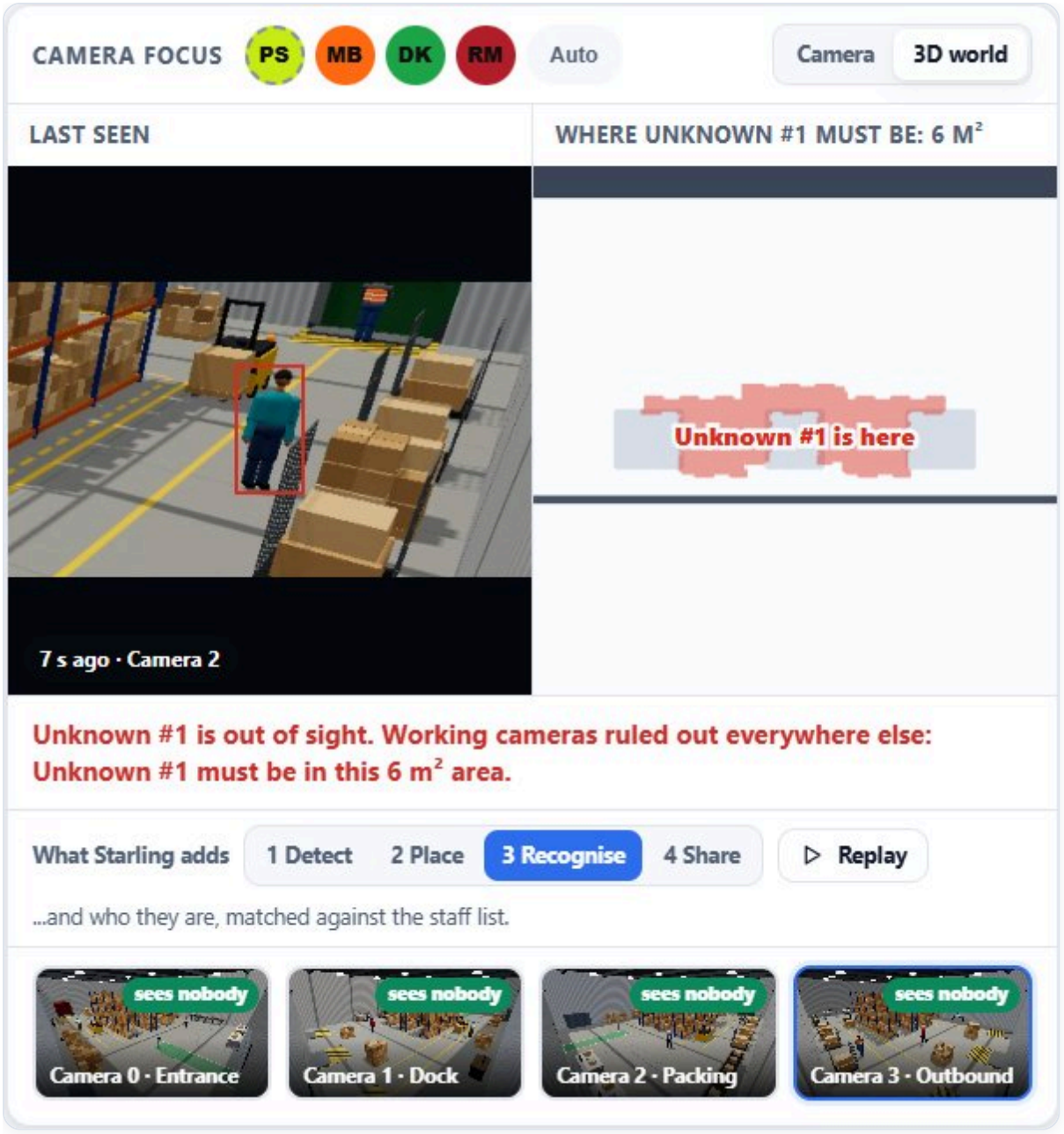
4.2d The stranger



What you see. Someone who is not an employee walked in through a loading dock. Nobody told the system. About 6 seconds later: a red **"Unknown #1"** appears in the people list, the **Unknown people** tile turns red, an alert lists them, a red dashed trail shows their path on the map, and the **Camera focus** panel **switches to them by itself**.



Camera focus on the stranger. Red boxes, **"Following Unknown #1"**, and the thumbnail for the camera that sees them is badged "*sees Unknown #1*". The system **does not need to know who someone is to follow them**: it tracks by appearance and physics.

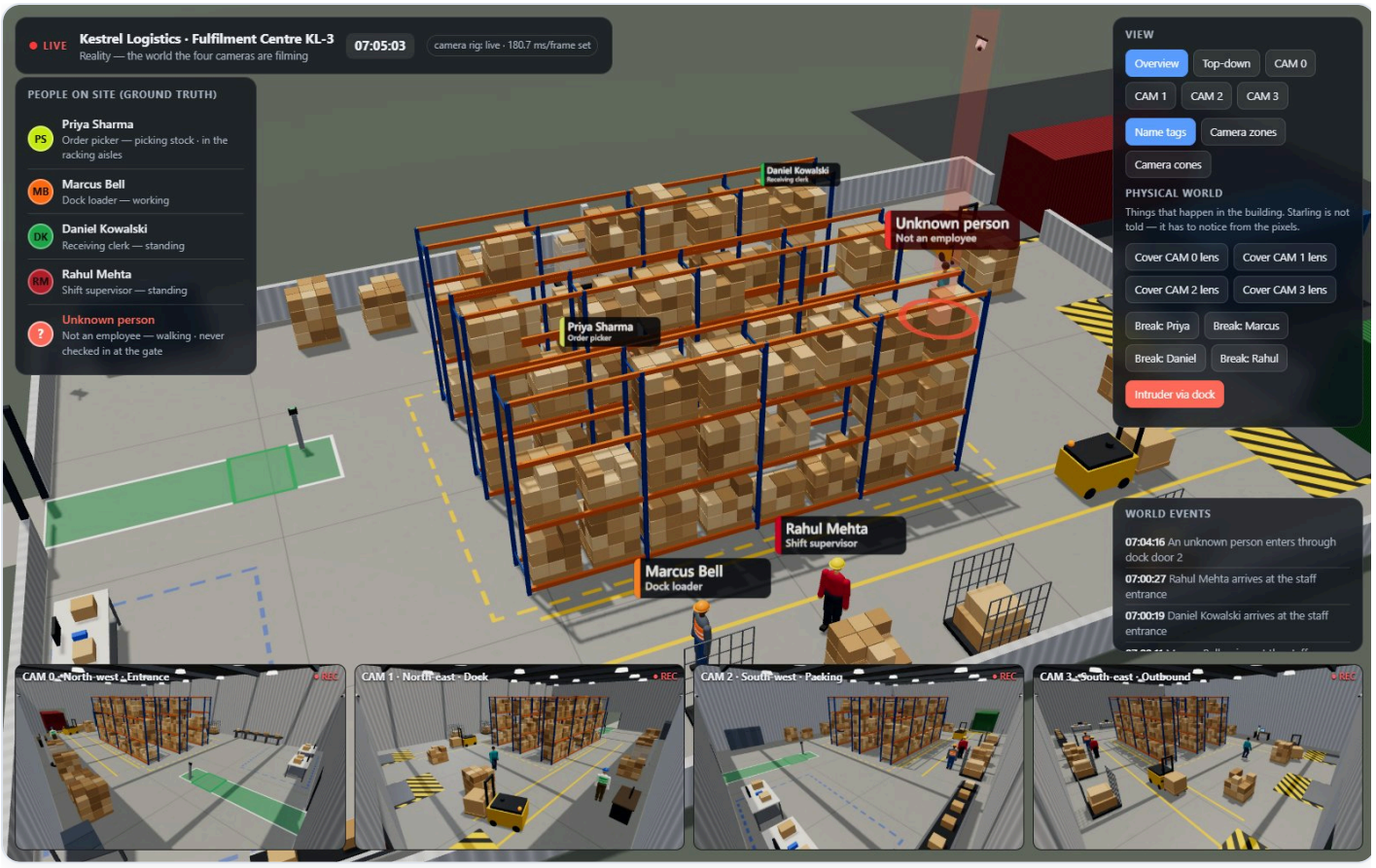


3D tab. The same panel can show the real 3D warehouse. The followed person gets a ring and the 3D camera flies to them. The stranger carries a pulsing red ring, a beam of light and a large tag so they cannot be missed behind racking.

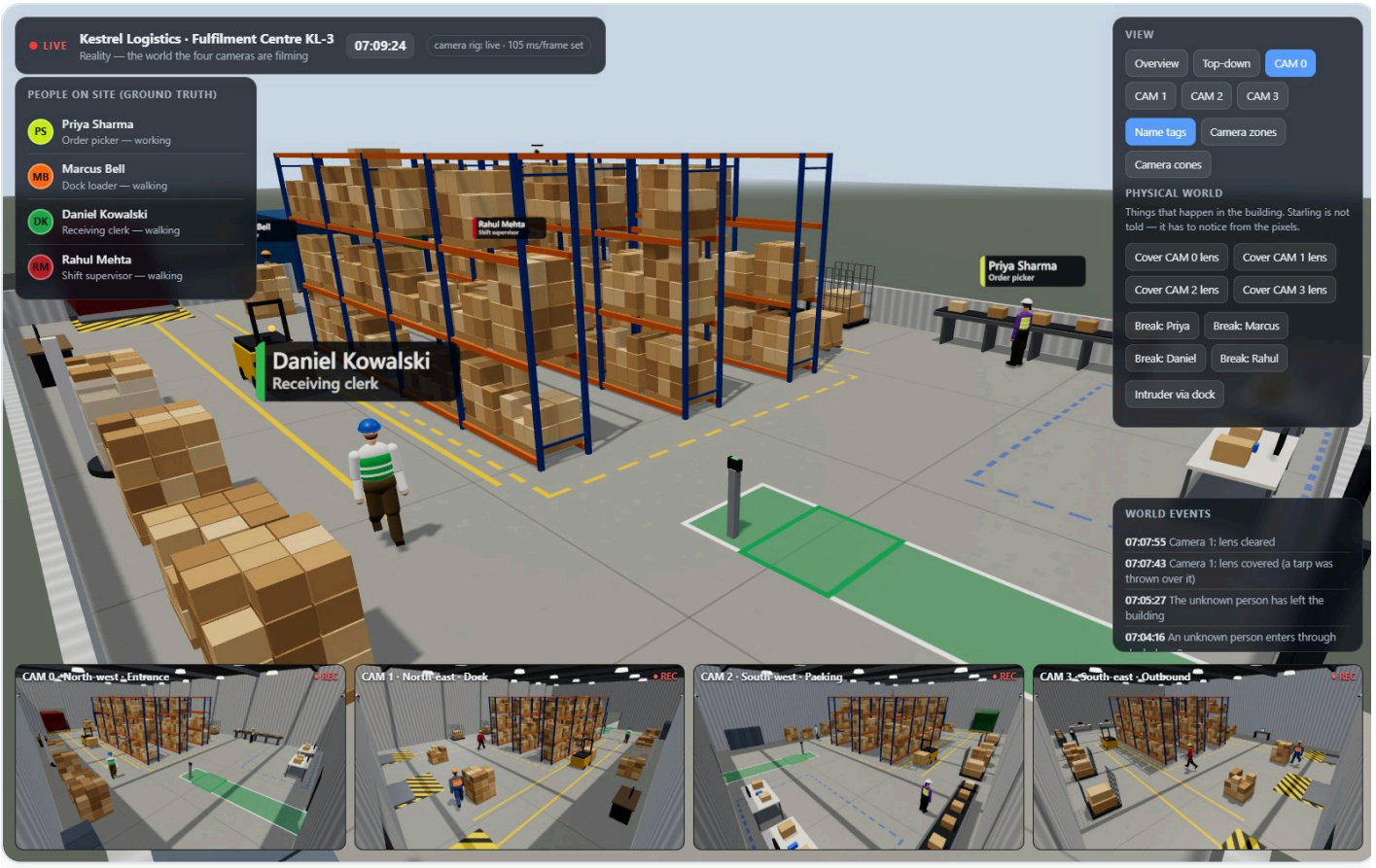
Technical detail. - A person is "unknown" when a resolved identity has **no FACE_ANCHOR claim** (nobody verified them against the staff list) and at least `unverified_min_claims = 14` sightings (about 8 seconds on camera). - Filters prevent false alarms (`apps/demo_dashboard/world_view.py`): not at the gate (about to be verified), not on top of a named employee at the same instant (a split track), not reported only by an *untrusted* camera (a liar's fabrication), and a hidden stranger stays listed for `unknown_hold_s = 20` s. - Numbers are stable ("Unknown #1"): a track that breaks and re-forms nearby inherits the old number. - A missing check-in is **not** the same as a stranger: asking "Where is Marcus?" when he has no check-in answers *"No camera has verified Marcus today, but there is an unverified person who may be him*". Starling never guesses a name.

4.2 The 3D warehouse (the simulated reality)

URL: `http://127.0.0.1:8770/` .



What you see. The virtual Kestrel Logistics warehouse: three rows of tall racking (the two aisles between them have no camera), a dock door, four employees (name tags are *ground truth*, shown for the audience only), a yellow forklift, and the four cameras' own pictures along the bottom. The right panel changes the view and triggers *physical* events: cover a camera lens, send someone on a break, **Intruder via dock**. These go to the **world**, not to Starling, so the nodes must notice from pixels. The red ring and beam mark the stranger.

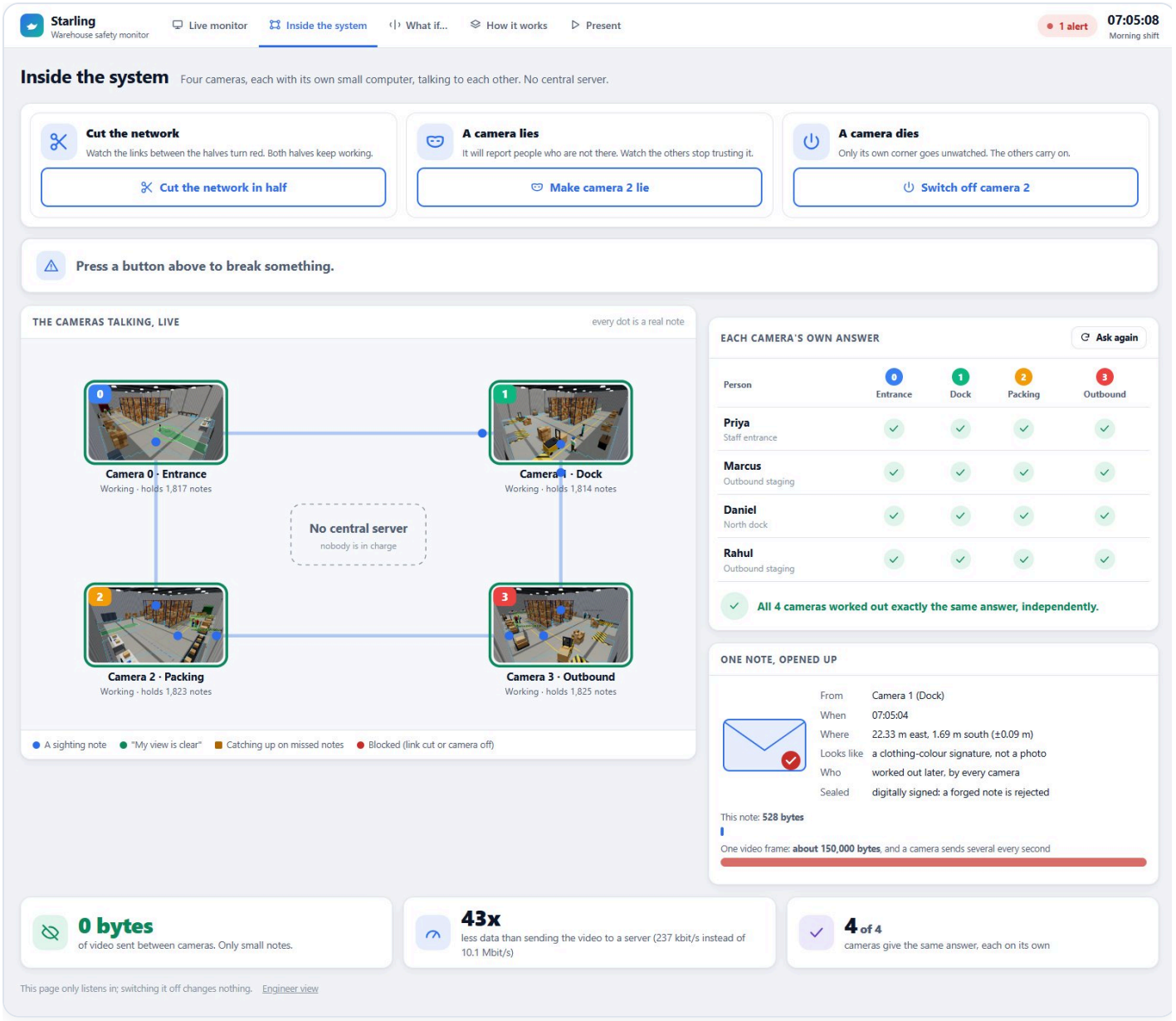


Camera 0's own view. This is what a node receives: a 1280x720 JPEG about 2 to 3 times per second. Everything else is derived from these pixels.

Technical detail. - The scene is `three.js` (vendored in `apps/world/static/vendor`). A headless Chromium **camera rig** (`apps/world/rig.py`) renders each camera's view and sends JPEGs to the world server, which publishes each on its own ZeroMQ topic (`cam:N|`). **A node subscribes to its own camera's topic only**, like a physical wire. - The world (`packages/starling_world/sim.py`) is a deterministic simulation (seed 7) with a navmesh, employee routines, two AGVs that stop for people, and an on-demand visitor. - The cameras' calibration (a floor homography) is generated from the camera pose, so the pixel-to-floor mapping is exact (reprojection error about 1e-13 px). In a real site you would measure it (section 8).

4.3 Inside the system

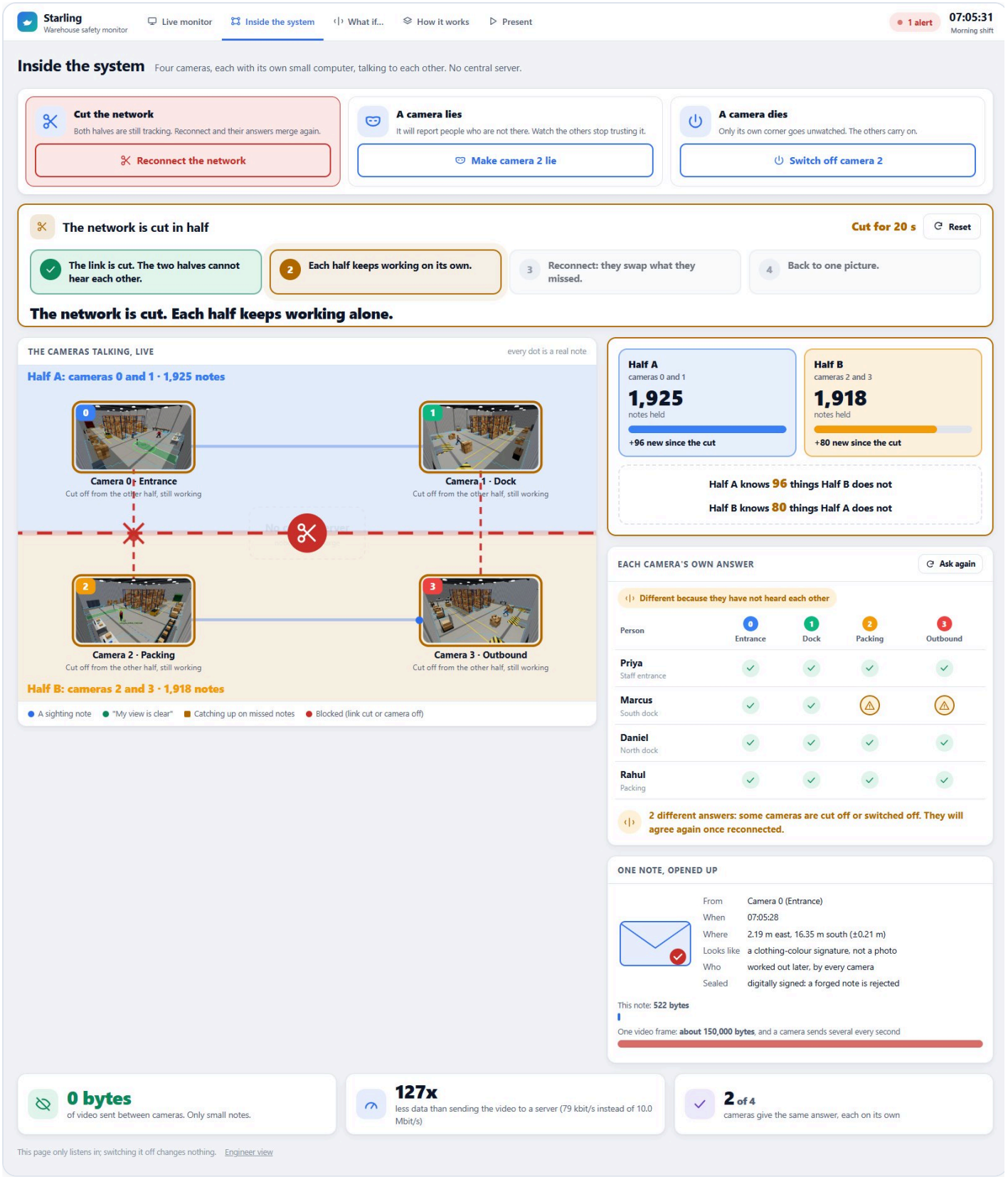
URL: `/inside`. This tab shows the machinery and lets you break things.



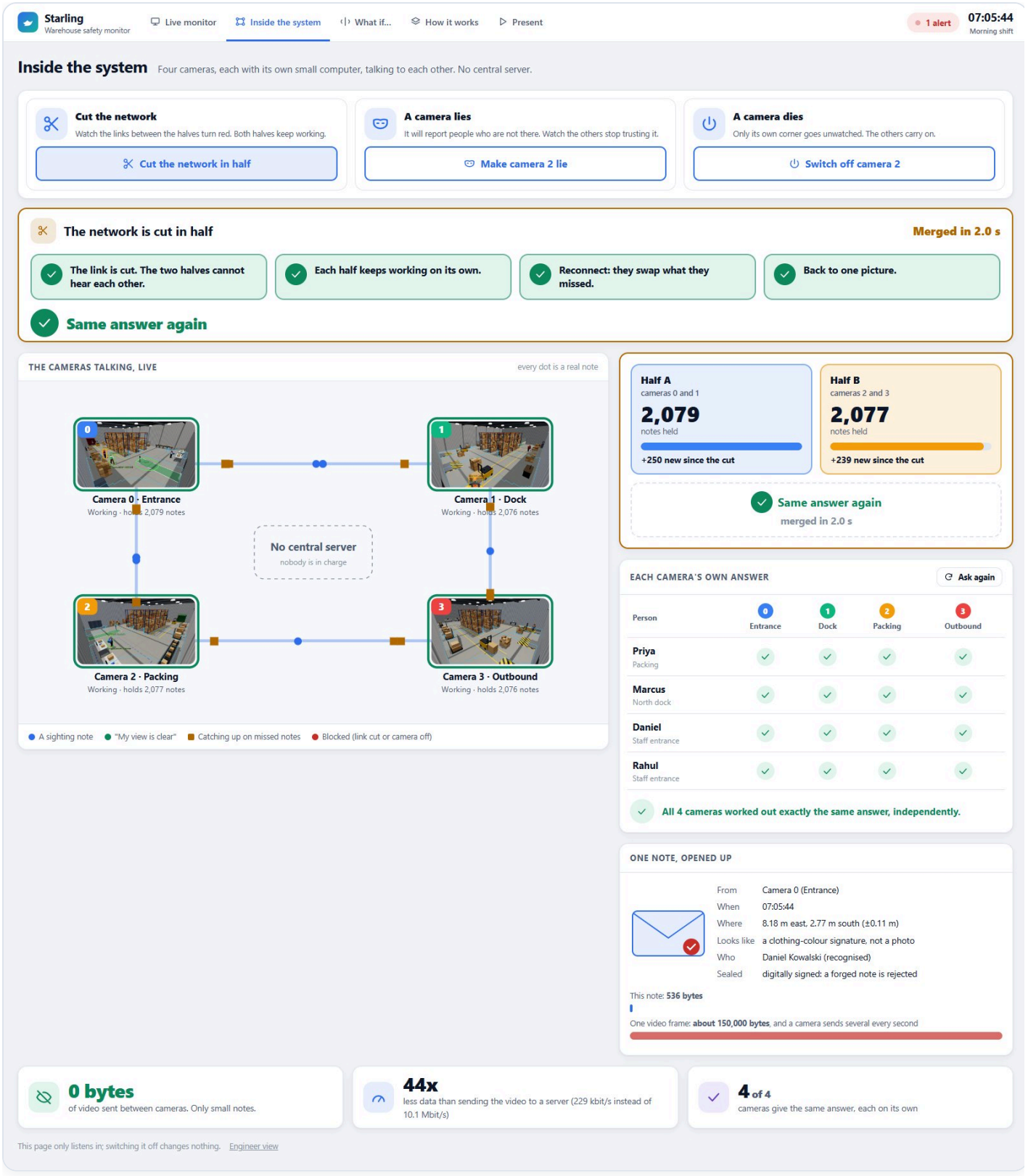
What you see. - **Three big buttons** at the top: *Cut the network*, *A camera lies*, *A camera dies*. - **The network drawing:** the four cameras with their live pictures. Dots travelling along the links are **real notes**: blue = a sighting, green = "my view is clear", orange square = a catch-up bundle. The centre says *"No central server, nobody is in charge."* - **"Each camera's own answer":** a grid of ticks, one column per camera. All ticks green means all four cameras computed the same positions independently. - **"One note, opened up":** an envelope with its fields (from, when, where, looks like, who, sealed) and a size comparison: **528 bytes** against about 150,000 bytes for a single video frame. - **Bottom, three big numbers: 0 bytes of video** between cameras; "**N times less data** than sending video to a server"; "**4 of 4** cameras give the same answer".

Technical detail. - The animation reads `GET /api/gossip_feed?since=N` : the observer keeps a ring buffer of every envelope it overhears (sender, kind, bytes). Nothing is staged. - "N times less data" is `video_bytes_per_s_total` (measured by the world server as it publishes JPEGs) divided by the **gossip bytes per second the dashboard overhears**, times each node's neighbour count (each message is sent to every neighbour). - The note fields come from `GET /api/sample_claim` : the newest claim overheard, decoded.

Cutting the network



What you see. Press *Cut the network* and the page tells the story in four steps. A red dashed **wall** splits the drawing into **Half A (cameras 0 and 1)** and **Half B (cameras 2 and 3)**. Notes that try to cross bounce back with a red X. A counter for each half shows how many notes it holds and how many it has gained **since the cut**: "*Half A knows 96 things Half B does not.*" In the answers grid, the cells where the halves disagree blink, with a chip: "*Different because they have not heard each other.*" A timer shows "Cut for 20 s".



Reconnect. The two halves swap what they missed. The page shows the difference falling to 0, then a green "*Same answer again*", and "merged in N s".

Why it matters. This is **partition tolerance**: both halves kept tracking while cut off, and after healing every copy was identical again, **automatically**, with no one deciding. A central system's far cameras would simply have gone dark.

Technical detail. - The button calls `POST /api/partition`, which tells each node (`POST /partition` on its control port, in parallel with one retry) to **drop everything received from the other half**. This is an application-level cut (`starling_consensus/attacks.py: PartitionControl`), not packet loss; a stronger test uses `deploy/netem`. - After

the heal, **anti-entropy** (`starling_net/anti_entropy.py`) fixes the gaps: each node periodically gossips a *version vector* (per origin node, the ranges of sequence numbers it holds). A neighbour replies with the claims missing from those ranges, chunked to stay under the wire size cap. - It works because the claim set is a **grow-only set**: union is commutative, associative and idempotent, so order and duplicates never matter. - Measured merge time: **about 2.5 s** in the scale experiment (16 nodes, split for 15 s) and 2 to 4 s in the demo. - If the two halves saw genuinely conflicting things (two sightings tying one identity to places that cannot both be true) the **fork stays open**: it is shown to a human, never settled by picking a winner.

A camera lies

Inside the system

Four cameras, each with its own small computer, talking to each other. No central server.

Cut the network
Watch the links between the halves turn red. Both halves keep working.
Cut the network in half

A camera lies
Its false notes contradict what other cameras saw, so its trust is falling.
Stop the lie

A camera dies
Only its own corner goes unwatched. The others carry on.
Switch off camera 2

Camera 2 is lying

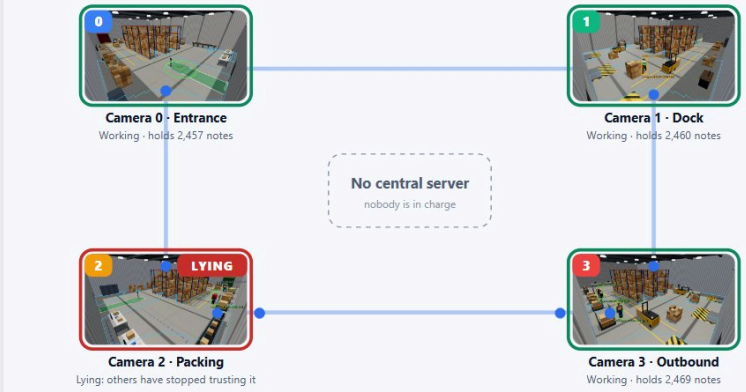
Lying for 53 s
Reset

1. Camera 2 starts inventing people.
2. The other cameras check every note: could someone walk there? Did anyone see them?
3. Fake notes are refused.
4. Camera 2 loses trust and is ignored.

Camera 2 has lost trust. The others ignore it.

THE CAMERAS TALKING, LIVE

every dot is a real note

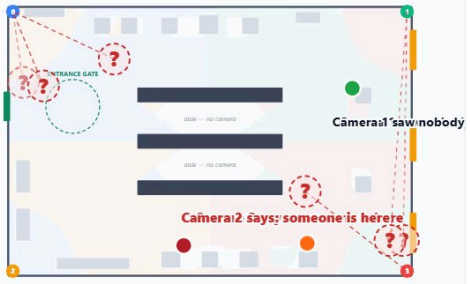


A sighting note · "My view is clear" · Catching up on missed notes · Blocked (link cut or camera off)

0 Camera 0	100% trusted	
1 Camera 1	95% trusted	
2 Camera 2	82% ignored	
3 Camera 3	99% trusted	

29 Notes refused
since the lying started: the other cameras did not believe them

What camera 2 claims



- Camera 1 vouched that this spot was empty
- Camera 0 vouched that this spot was empty
- Camera 3 vouched that this spot was empty

● a real person ○ ? a person camera 2 invented

EACH CAMERA'S OWN ANSWER

Ask again

Person	0 Entrance	1 Dock	2 Packing	3 Outbound
Priya — South dock	✓	✓	✓	✓
Marcus — South dock	✓	✓	✓	✓
Daniel — North dock	✓	✓	✓	✓
Rahul — Outbound staging	✓	✓	✓	✓

All 4 cameras worked out exactly the same answer, independently.

ONE NOTE, OPENED UP

From	Camera 1 (Dock)
When	07:06:32
Where	28.34 m east, 3.34 m south (±0.34 m)
Looks like	a clothing-colour signature, not a photo
Who	worked out later, by every camera
Sealed	digitally signed: a forged note is rejected

This note: **528 bytes**

One video frame: **about 150,000 bytes**, and a camera sends several every second

0 bytes
of video sent between cameras. Only small notes.

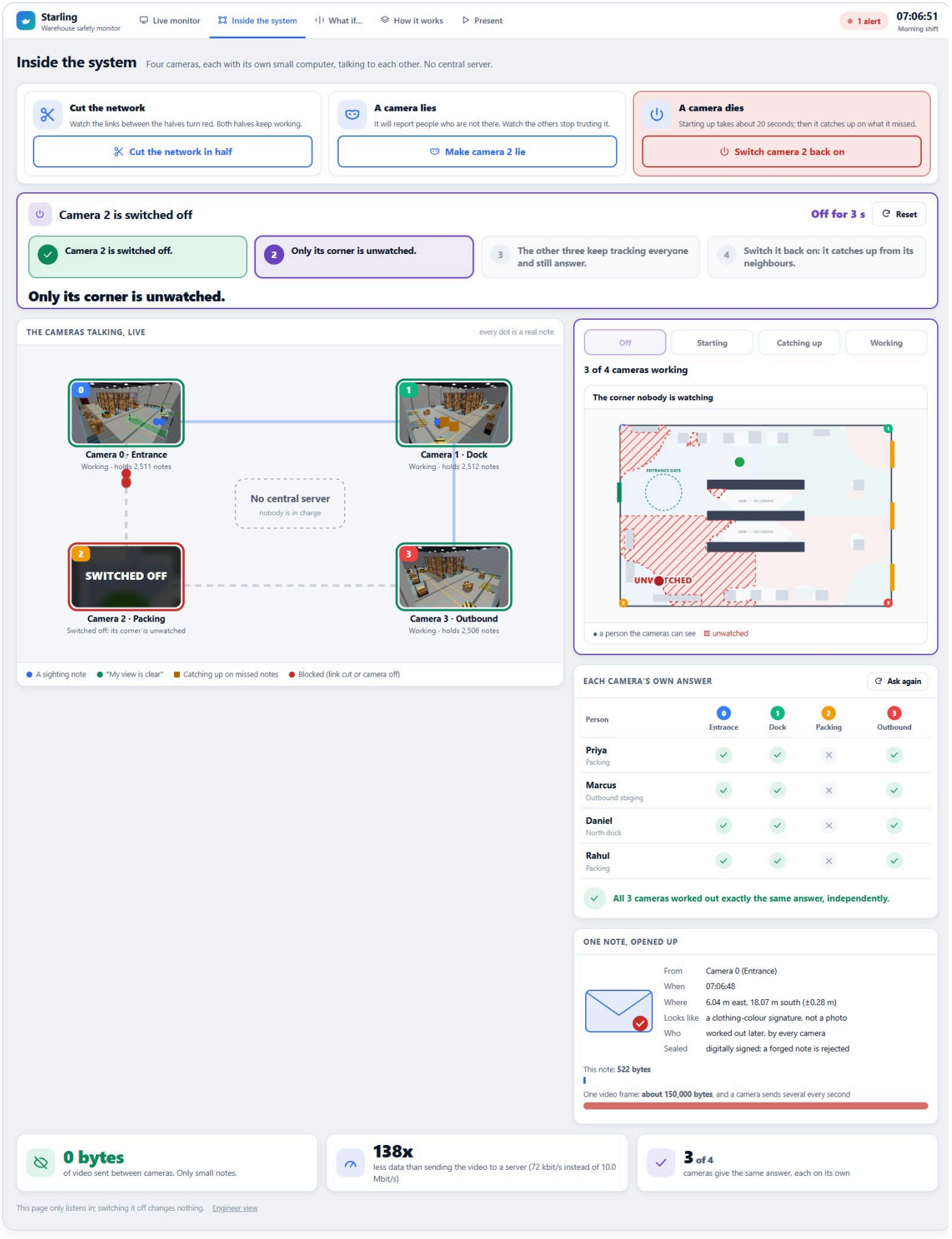
37x
less data than sending the video to a server (272 kbit/s instead of 10.1 Mbit/s)

4 of 4
cameras give the same answer, each on its own

What you see. Camera 2 is made to invent people. The step strip walks through it: (1) it starts inventing people, (2) the other cameras check every note, (3) fake notes are refused, (4) camera 2 loses trust and is ignored. On the right: four **trust meters** (camera 2's drains and turns red, "82% ignored"), a counter **"Notes refused: 29"**, and a floor plan titled **"What camera 2 claims"**. The red dashed **"?" figures are people camera 2 invented**; thin lines run to the cameras that contradicted them, with reasons listed underneath, for example "*Camera 1 vouched that this spot was empty*". The real people are the normal dots.

Technical detail. - A claim is checked (`starling_consensus/plausibility.py`) for **reachability** (could a person walk there in that time?), **kinematics**, **corroboration** by nearby claims from other nodes, and **freshness** (replays). - The contradiction check (`starling_consensus/contradiction.py` , "C2 x C4") adds: if a claim places someone inside *another* camera's zone, and that camera **attested the zone healthy and saw nobody** there, the claim is contradicted. - Each failed check lowers the sender's **reputation** (an exponential moving average, `alpha = 0.05` , floor `r_min = 0.05` so a repaired node can recover). Resolver scores multiply by this reputation. A manager alert fires below 0.85. - Two safeguards prevent honest cameras being punished: the "someone was there" tolerance grows with both claims' position uncertainty, and repeated contradictions of the *same track* within 15 s count once. - The ghosts on the floor plan come from `rejections` in `/api/state` , the last rejected claims with `(x, y, why, contradicted_by)` , recorded by the node's plausibility loop (`recent_rejections` in `apps/node.py`). - **Honest limit:** a lie placed *inside the liar's own zone* contradicts nobody (no other camera saw that spot), so the liar's trust settles around 0.8 instead of near 0.

A camera dies

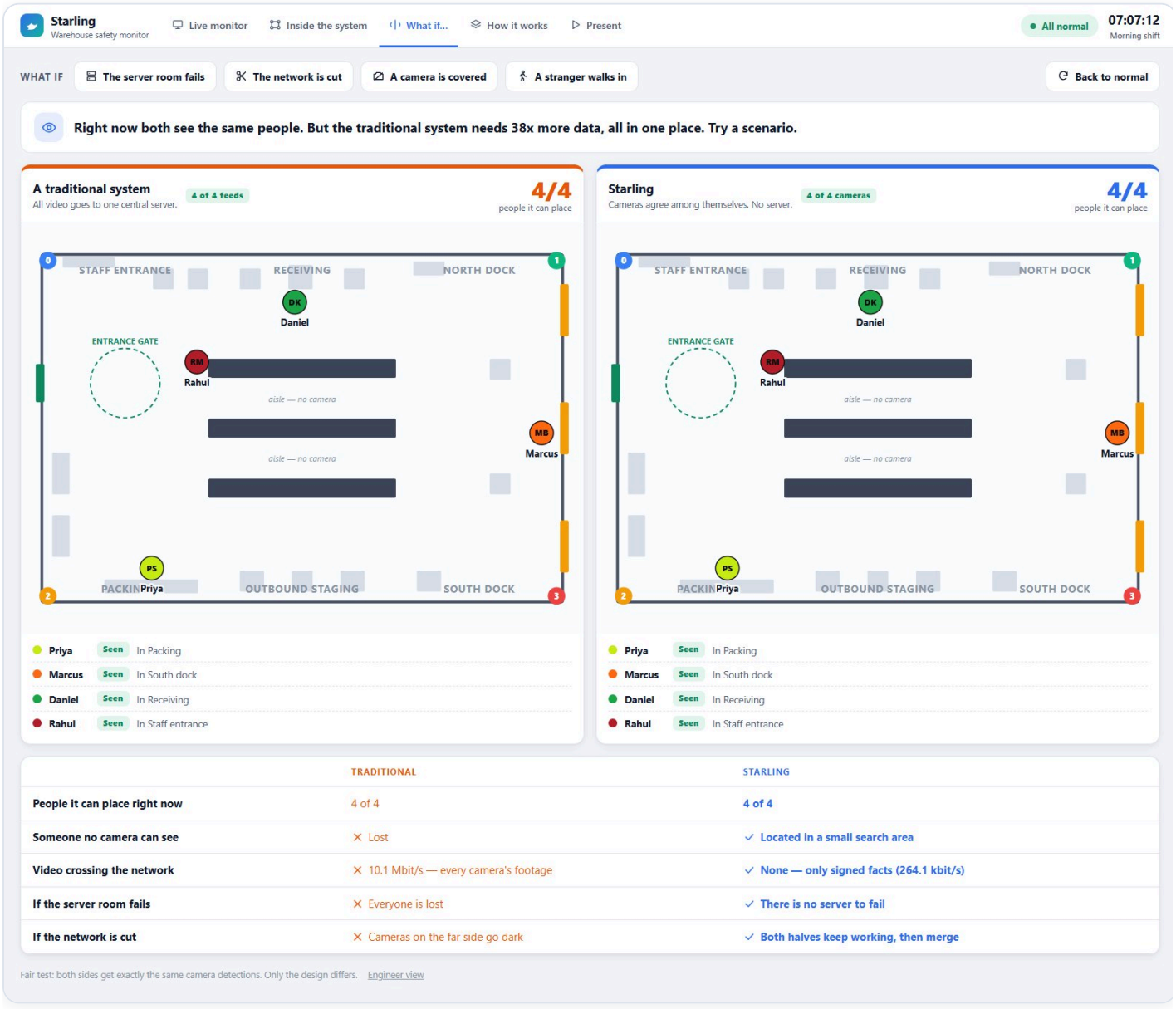


What you see. Camera 2 is powered off. Its corner is hatched as **unwatched**. The other three keep tracking and still answer. Switching it back on, it catches up from its neighbours.

Technical detail. `POST /api/node/2/kill` calls `POST /shutdown` on that node's control port (the process exits cleanly; its replica stays on disk). `POST /api/node/2/restart` starts it again from the launch manifest (`data/demo/world/launch.json`), waiting 8 s if it was only just stopped. On restart the node reloads its SQLite replica and anti-entropy fetches whatever it missed. Only the dead camera's own zone loses coverage; the dashboard shows it as a hatched area because a switched-off camera signs no attestations, and **silence is not evidence**.

4.4 What if: Starling vs a traditional system

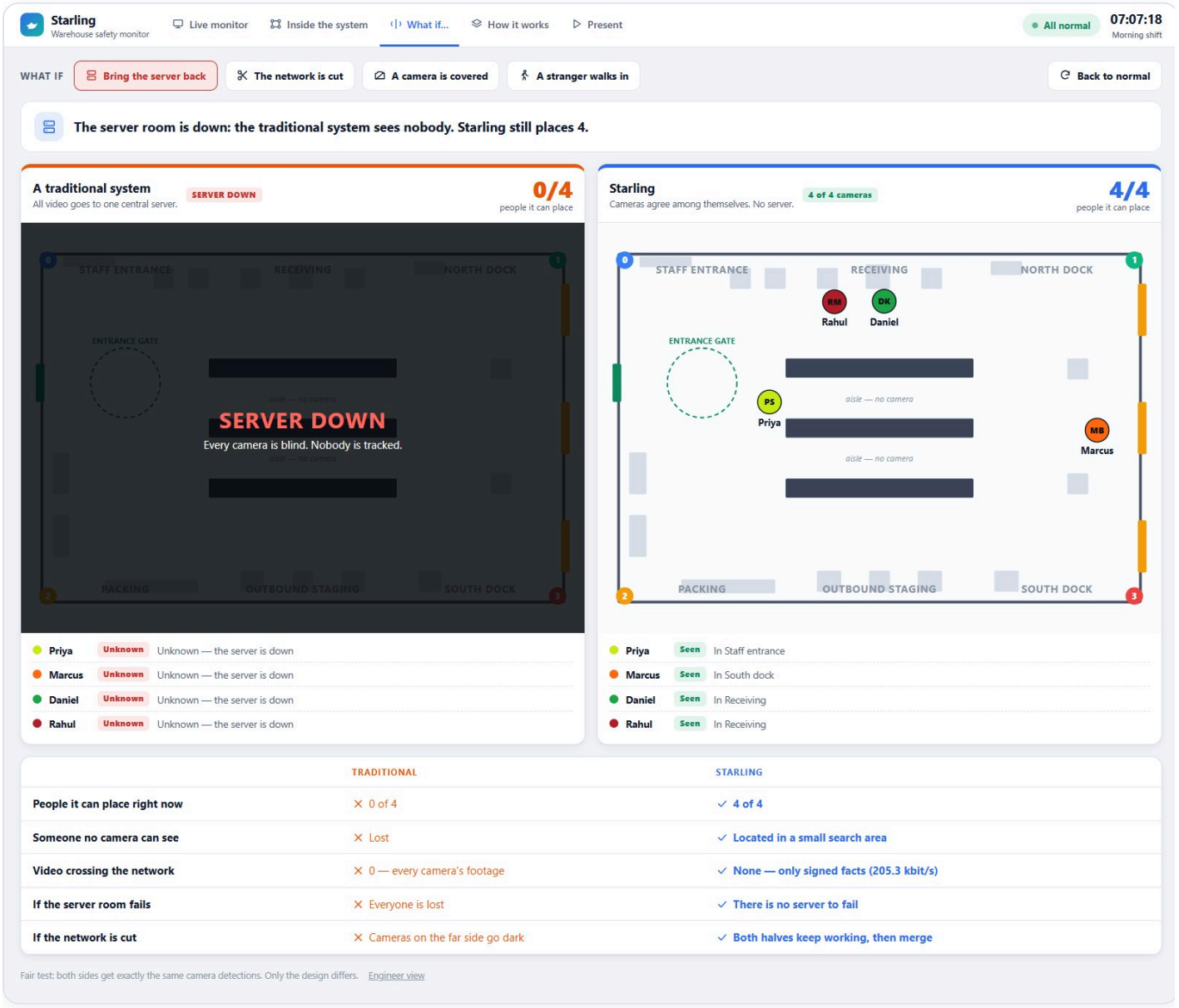
URL: `/compare`. Two live maps of the **same moment, from the same camera detections**.



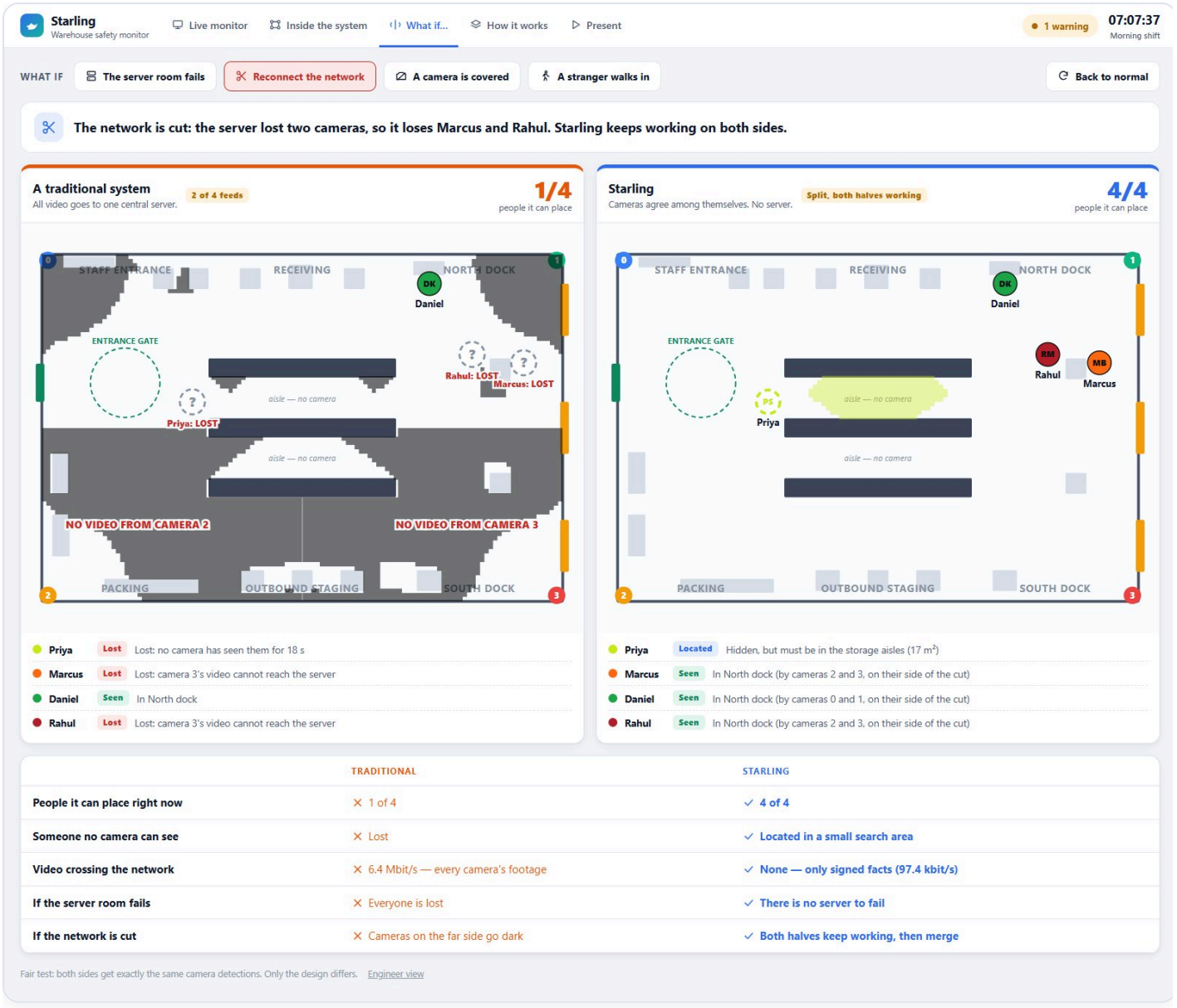
What you see. Left: a **traditional system**, one central server that receives every camera's video. Right: **Starling**. Each side has a score ("people it can place"), a map, a list, and a one-sentence **story** at the top. The scoreboard at the bottom compares people placed, video on the network (**10.2 Mbit/s** against **tens of kbit/s**), what happens if the server room fails, and what happens if the network is cut.

A fair test. Both sides get exactly the same person detections, so the traditional side is just as good at seeing people. What it lacks is architectural: it has no idea where someone is once no camera sees them, and it is one box on one side

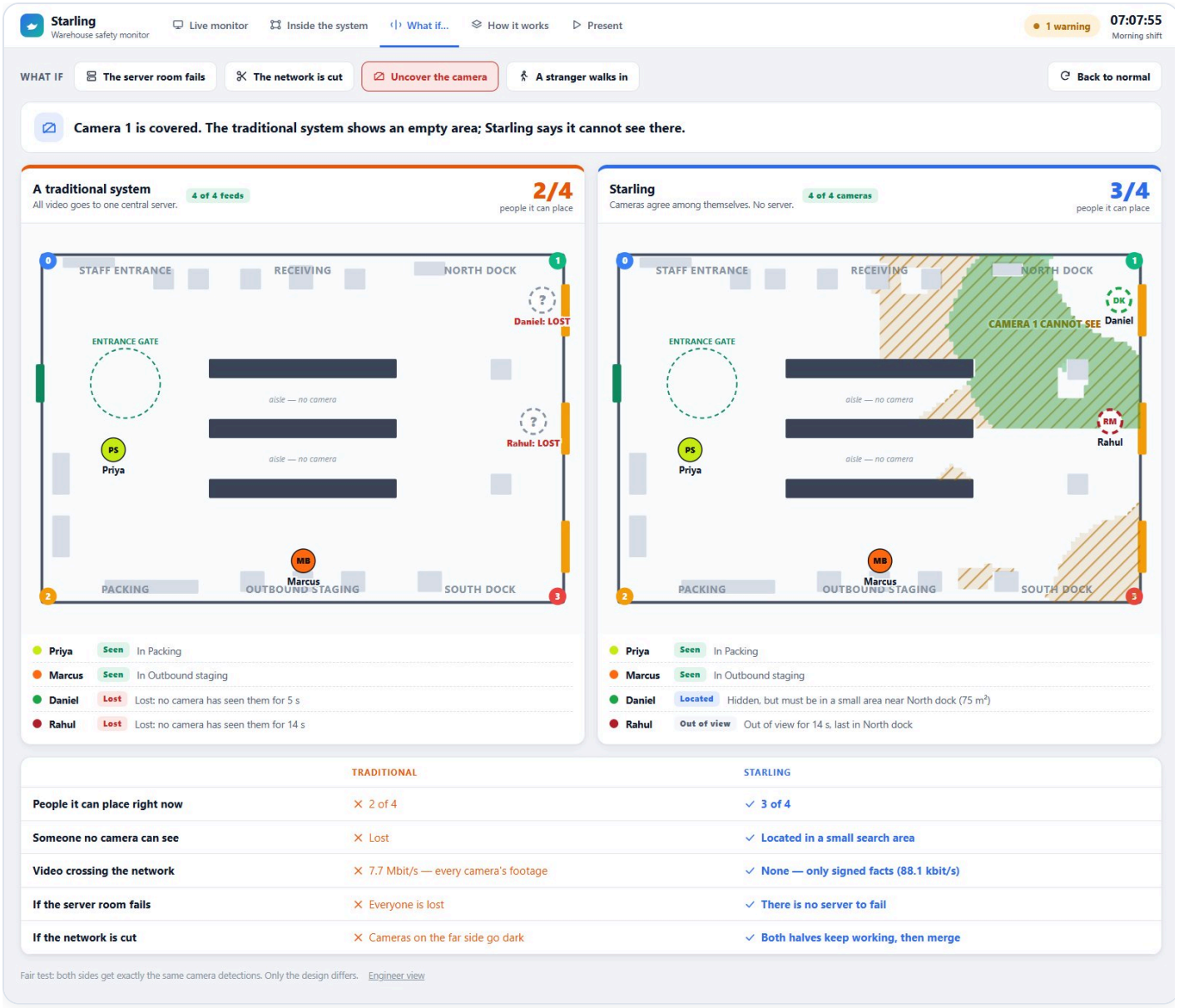
of the network.



"The server room fails". The left side goes black: **0 of 4 placed**. Starling keeps placing everyone: no server to fail.



"The network is cut". The central server sits with cameras 0 and 1, so cameras 2 and 3's video can no longer reach it. The left map darkens those zones (**"NO VIDEO FROM CAMERA 2/3"**) and people there are **LOST**. Starling's two halves keep tracking on their own.

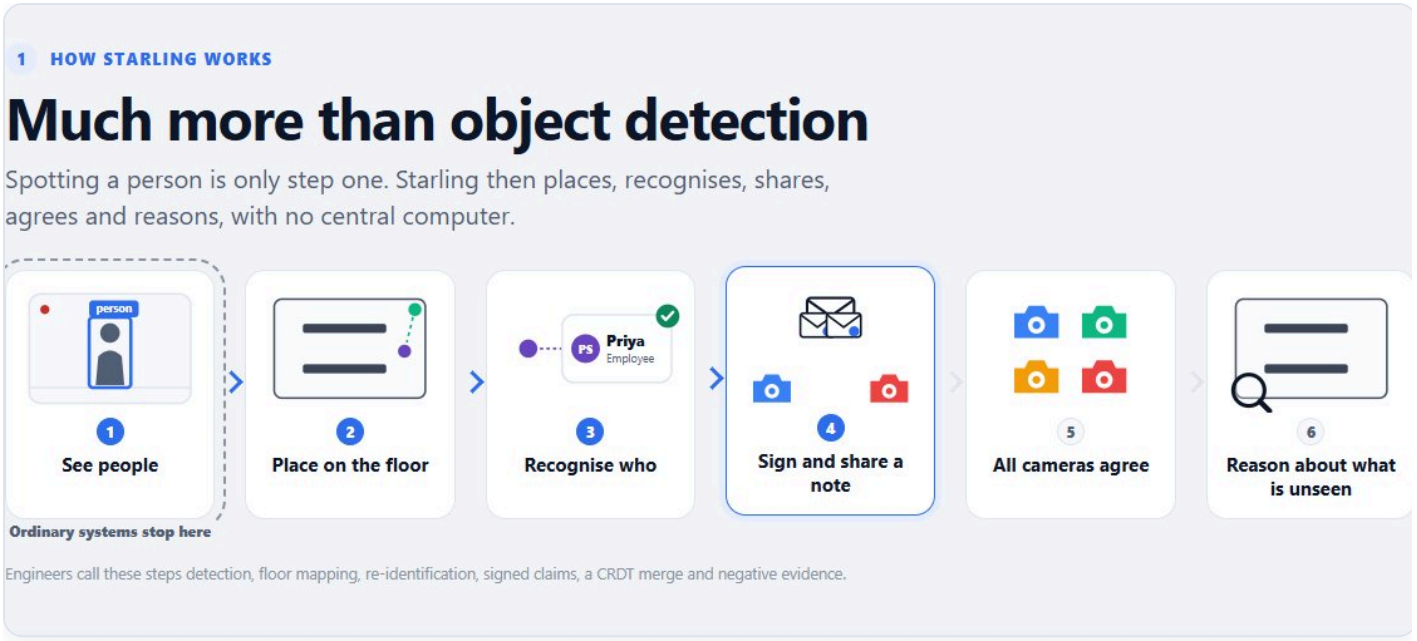


"A camera is covered". A tarp is thrown over camera 1's lens (a *physical* event Starling is not told about). The camera notices from its own pixels, stops vouching for its zone, and the hatched area on the right is labelled **"CAMERA 1 CANNOT SEE"**. The traditional system simply shows an empty area, falsely reassuring.

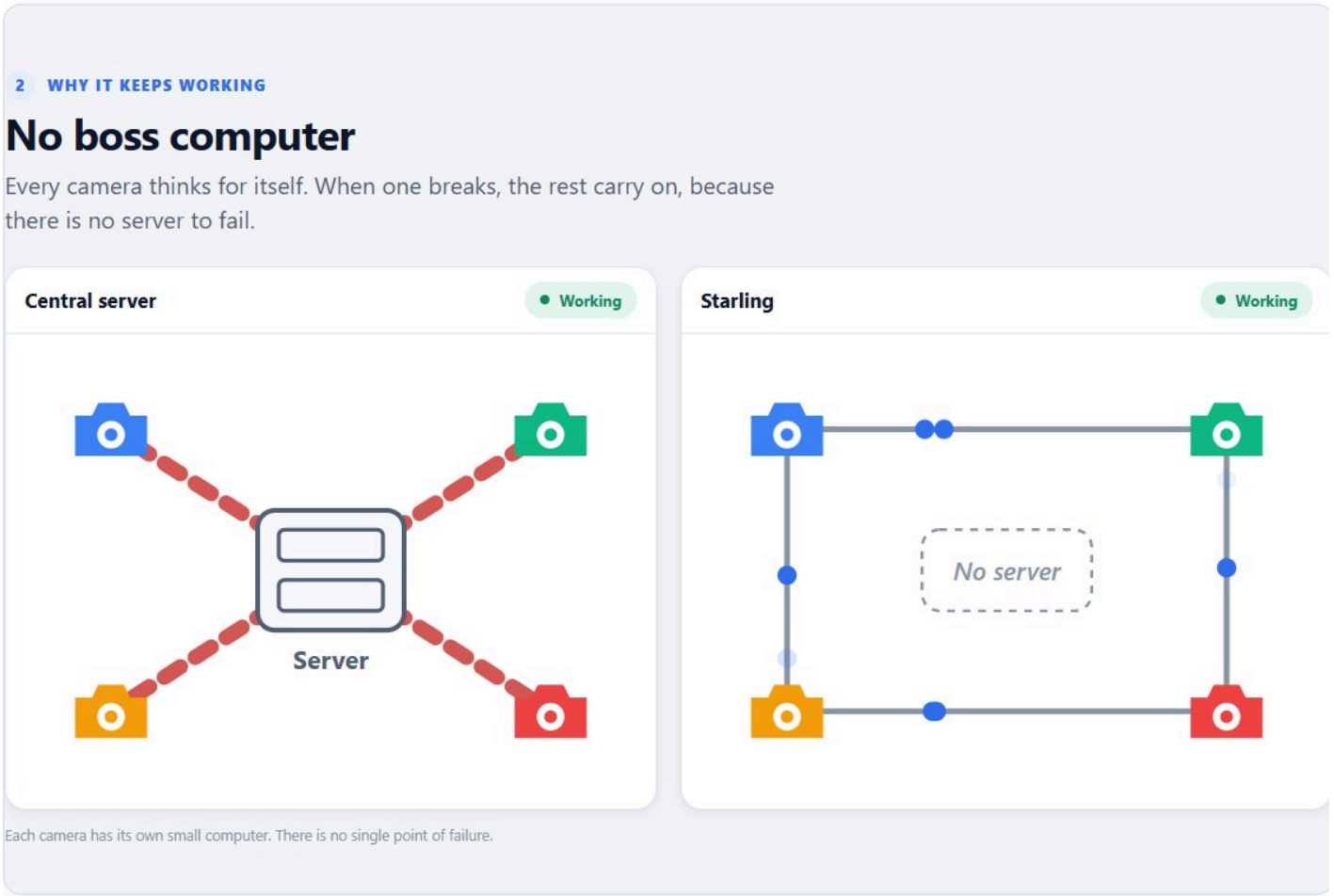
Technical detail. - The "central server" is a **real process**, `apps/world/central_ingest.py`. It subscribes to all four camera topics (the same frames the nodes receive) and measures frames and bytes per second per camera. `POST /partition` makes it drop the cameras on the far side of the cut; `POST /shutdown` kills it. It does no tracking itself; the comparison gives it Starling's detections. - `GET /api/compare` (`Showcase.compare`) builds both views. A person unseen for under 10 s still counts as placed on both sides (any tracker coasts that long), so the comparison is about long gaps and failures. - Live bytes: video from the world server's counters, notes from the gossip the dashboard overhears. - **Not yet measured:** the full three-way experiment (original V1 centralised system vs Starling healthy vs Starling under partition and attack) is planned but not populated (`docs/results_headline.md`).

4.5 How it works (the animated explainer)

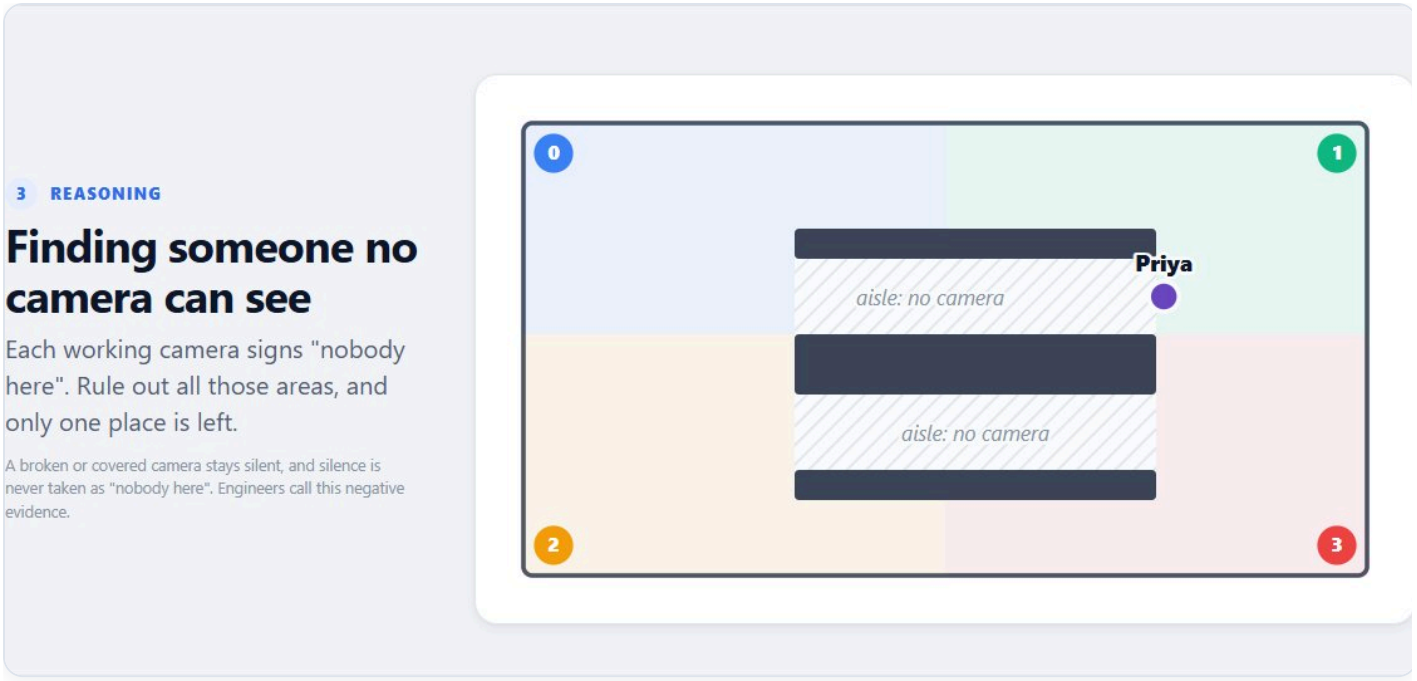
URL: `/how` . Eight sections, each with one sentence and one looping animation, for a non-technical audience.



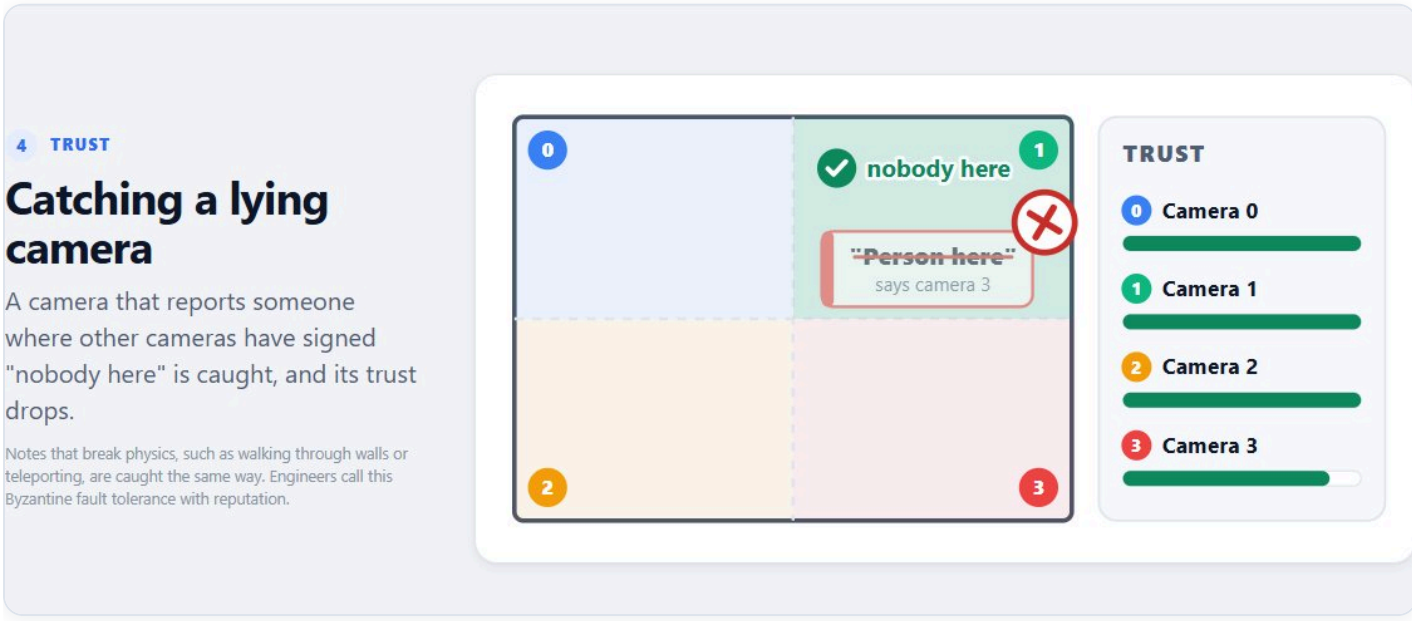
1. "Much more than object detection." Six tiles. A sighting travels through them: a box appears around a person, a dot lands on a floor plan, a name badge appears, a sealed envelope copies to two neighbour cameras, four cameras get a green tick, a magnifier reveals a "must be here" area. The dashed box around tile 1 reads *"Ordinary systems stop here."*



2. "No boss computer." Left, a central server fails and every line goes grey: *Everyone lost*. Right, a camera fails and the others keep passing notes: *Only one corner affected*.



3. "Finding someone no camera can see." A person walks into an aisle and fades; a "could be here" circle grows; each working camera's area flashes green **"nobody here"** and is cut out, until only the aisle strip remains: **"must be here."** A broken camera's silence is never taken as "nobody here".



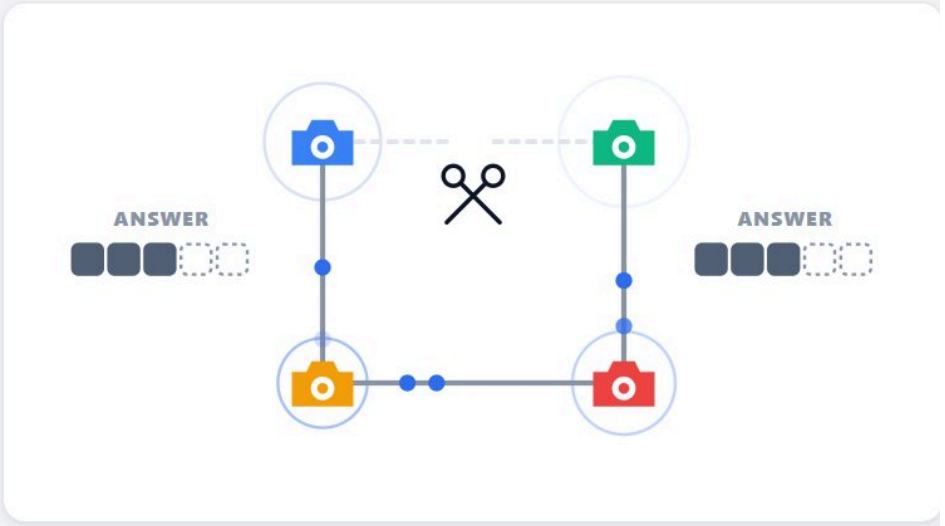
4. "Catching a lying camera." Camera 1 has signed "nobody here". Camera 3's note "person here" in that area is stamped with a red cross and camera 3's trust bar drains.

5 RESILIENCE

Cut the network, nothing breaks

Split the network and both halves keep working. Reconnect it and they agree again, by themselves.

Measured: after a split, every copy matched again about 2.5 s after the link came back. Engineers call this partition tolerance.

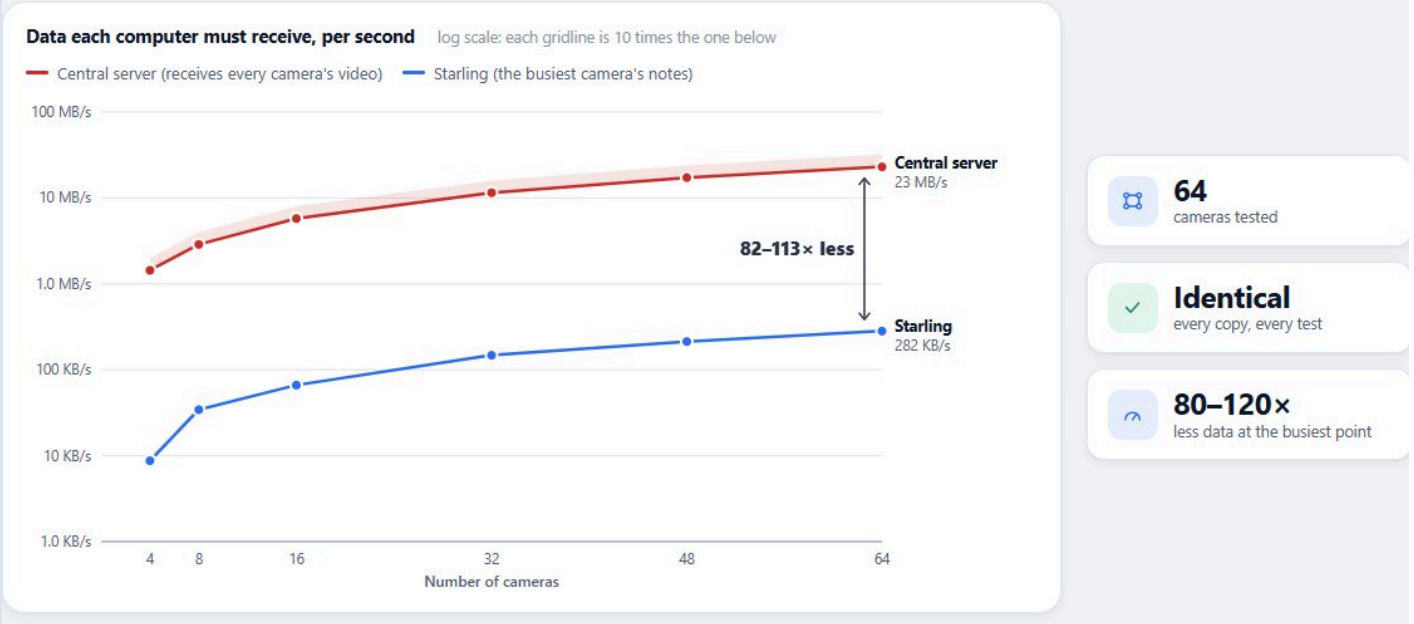


5. "Cut the network, nothing breaks." Scissors cut a ring of four into two pairs; each pair keeps working and shows its own "answer" (a row of coloured squares that differ); on reconnection missing squares cross over and both rows match: "*merged in about 2.5 s*" (the figure is filled in from the scaling experiment).

6 SCALE

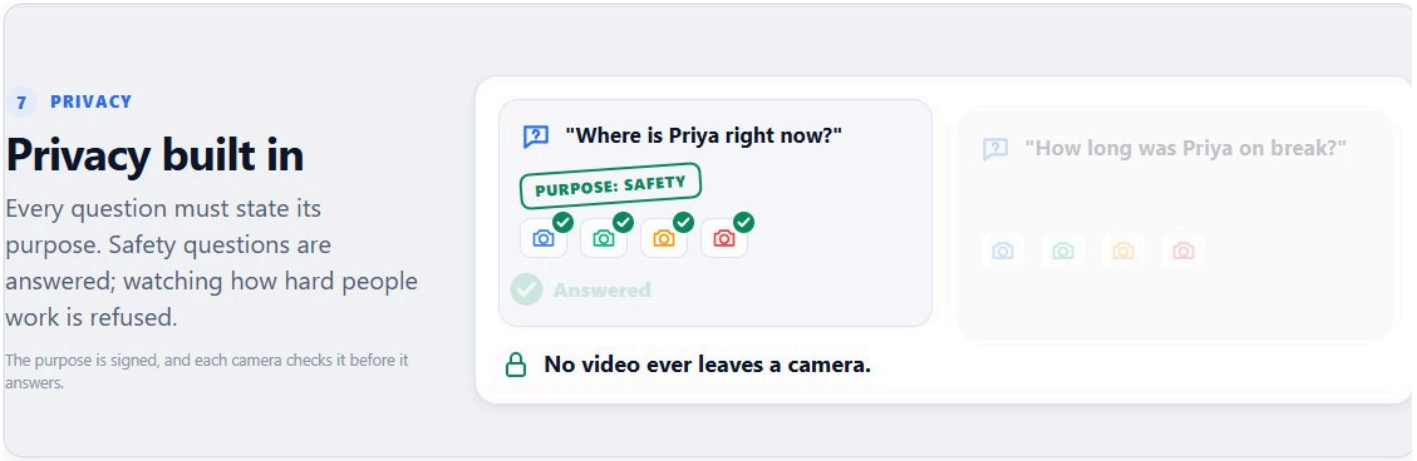
It grows with the site

Cameras share small notes, not video, so the network stays light as you add more cameras.

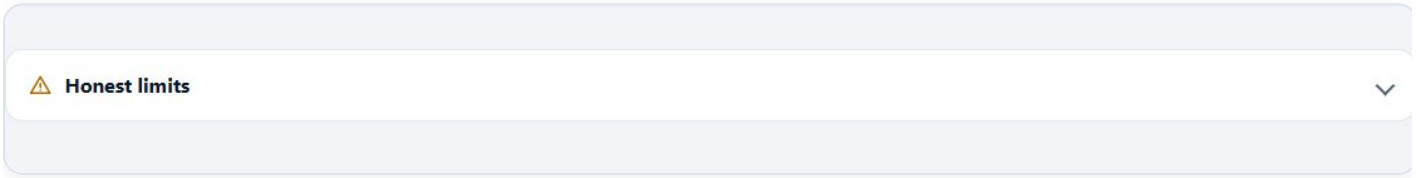


Starling figures are measured on one laptop running up to 64 camera computers. Central-server figures are computed: number of cameras × one camera's video (360 KB/s in this demo, up to 500 KB/s for typical 1080p CCTV, shown as the shaded band). In the live demo, four cameras film about 1.3–2 MB/s of video while their notes need only tens of KB/s.

6. "It grows with the site." The measured scaling chart (section 9): traffic into the **busiest** Starling camera against what a **central server** must ingest, on a log scale, for 4 to 64 nodes, with a hover tooltip.



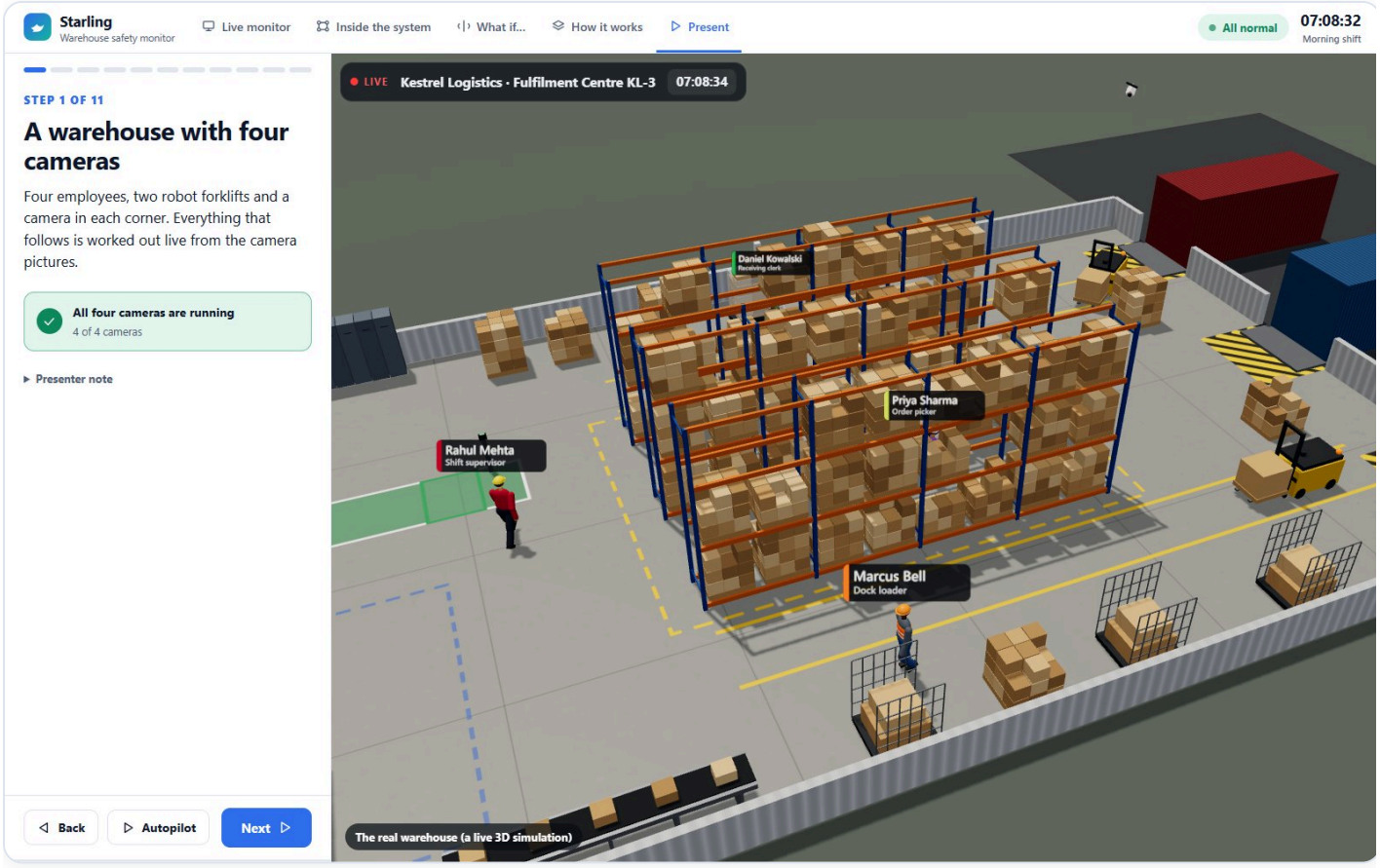
7. "Privacy built in." A question stamped *Purpose: Safety* is answered by all four cameras; one stamped *Purpose: Productivity* is **refused by every camera**. *No video ever leaves a camera.*



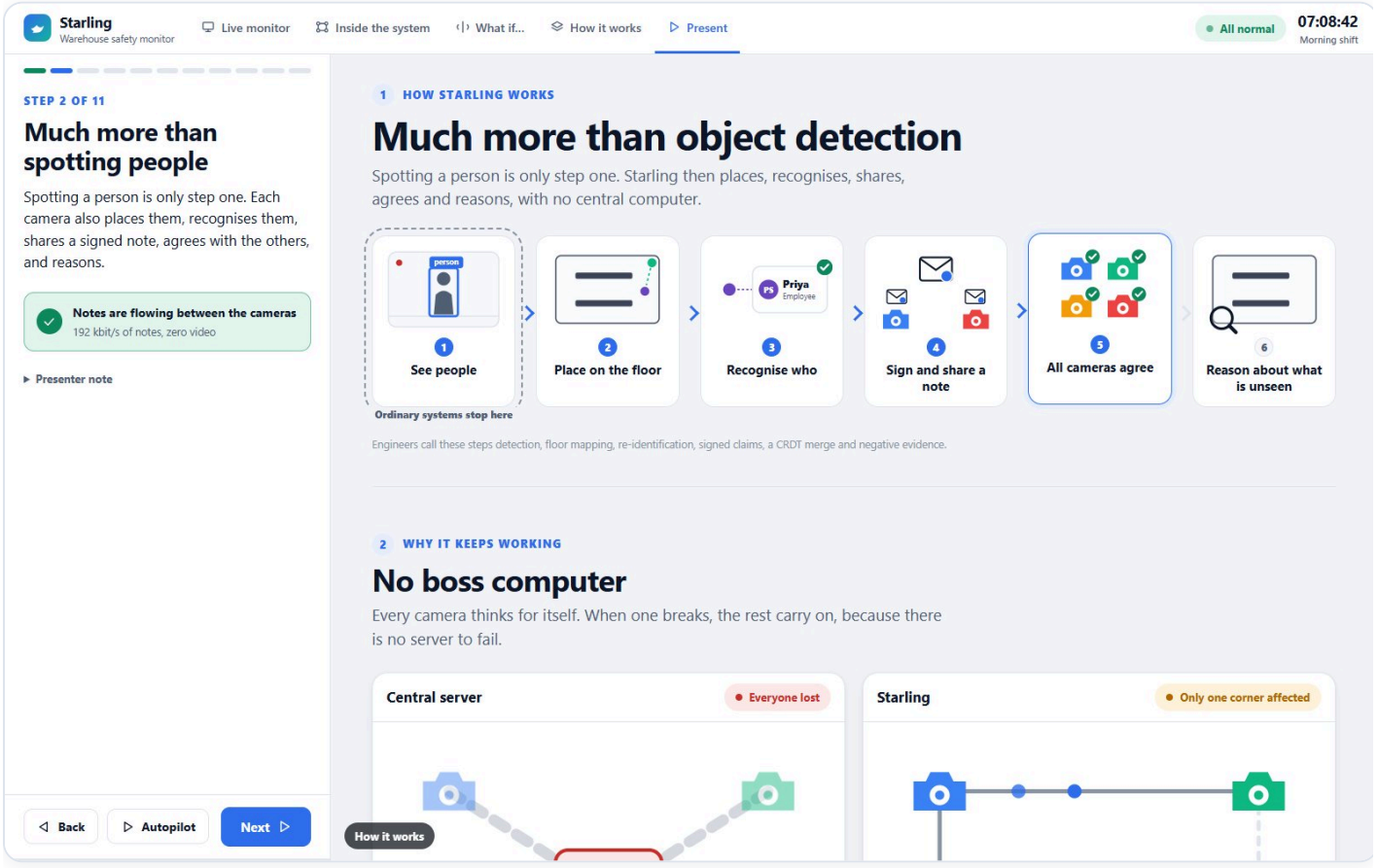
8. "Honest limits." Three bullets; see section 7 of this guide.

4.6 Present and Autopilot

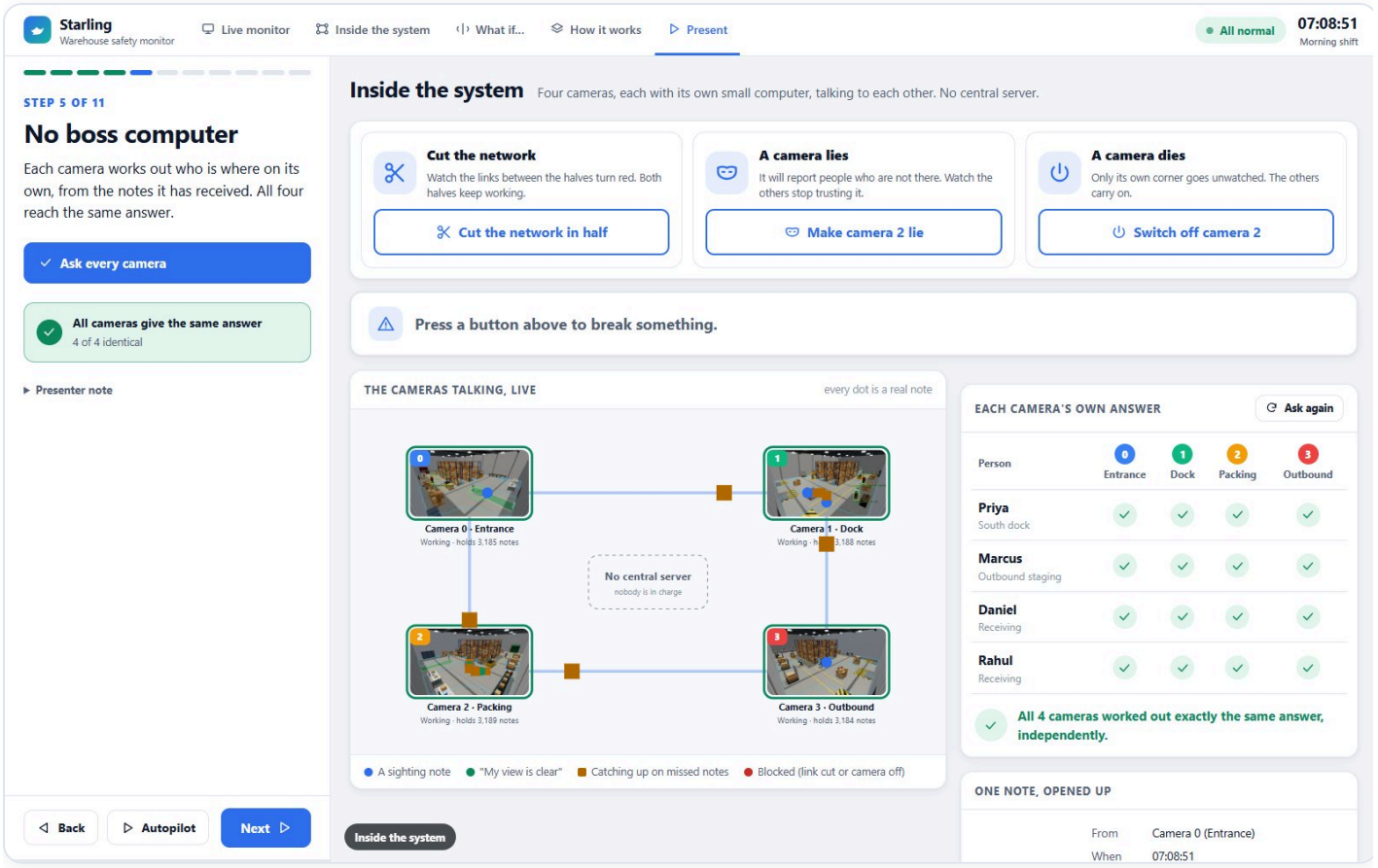
URL: `/tour`. The guided presentation: 11 steps, each with a title, one sentence, one or two buttons and a **live check** that turns green when the system really shows the thing.



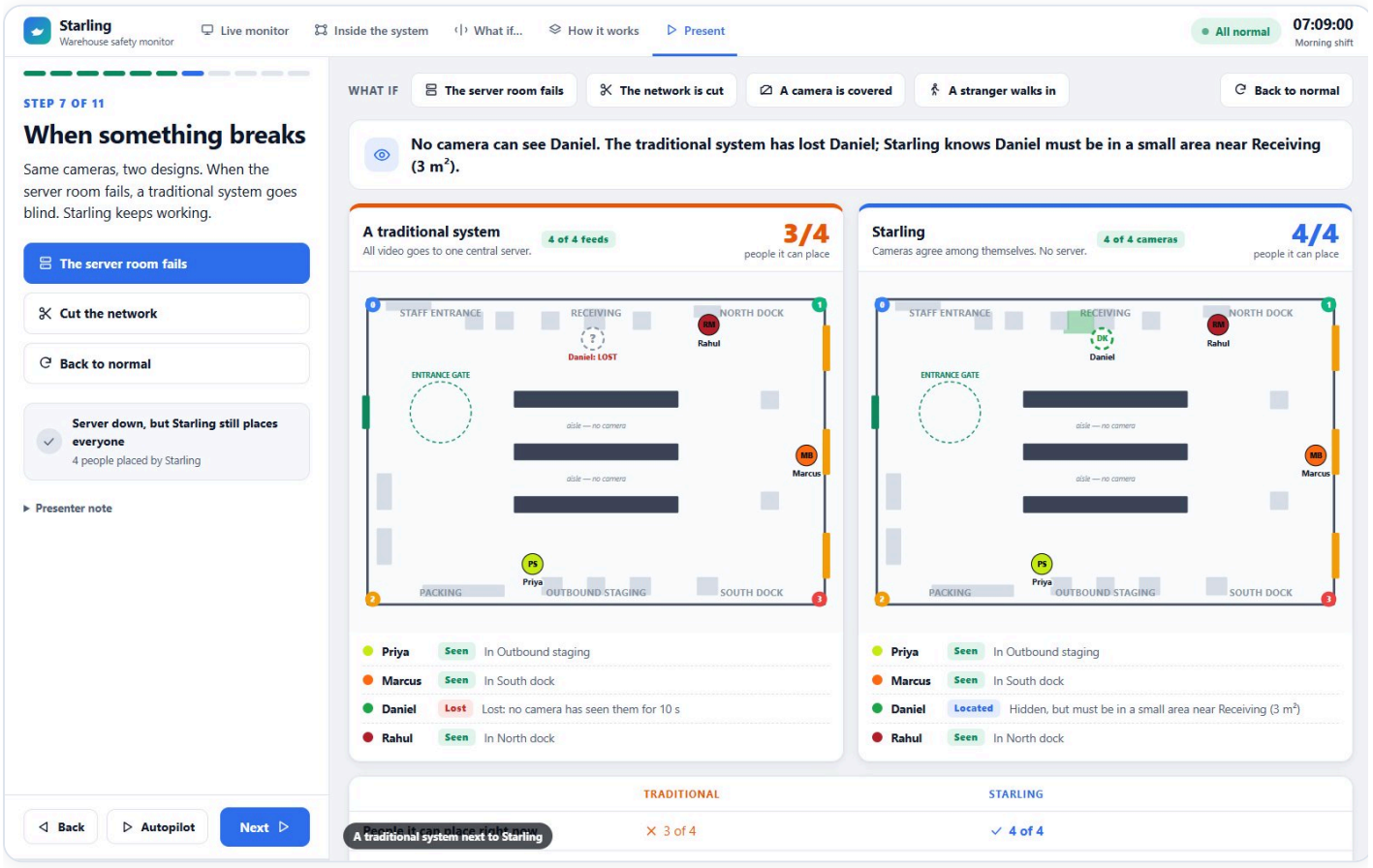
Step 1 embeds the 3D warehouse on the right. The left panel holds the step title and sentence, a progress bar of 11 chips, and a live check ("All four cameras are running" turns green).



Step 2 embeds the "How it works" page ("Much more than spotting people").



Step 5, "No boss computer". The button **Ask every camera** asks all four nodes; the check turns green when all four fingerprints match.



Step 7, "When something breaks". The embedded "What if" page, with buttons to fail the server room or cut the network. The check turns green when the central server is down *and* Starling still places everyone.

Autopilot (bottom left) runs all 11 steps in about 5 minutes: it opens each page, presses each scenario button at the right moment, advances on a timer, and **restores everything at the end** (network healed, no liar, server up). Press any step to take back control.

Technical detail. `apps/demo_dashboard/static/world_tour.html`. Pages are embedded as iframes (`?embed=1` hides their header). The 3D iframe is unloaded when not shown, because it shares the graphics card with the camera rig (running both starves the nodes). Checks read the same `/api/state`.

4.7 Engineer view

URL: `/engineer`. Every internal number, for technical questions.

Starling
Warehouse safety monitor

Live monitor | Inside the system | What if... | How it works | Present

All normal
07:09:07
Morning shift

Engineer view

Every internal number, for technical questions. The other tabs explain the same things in plain words.

Converged · spread 8
3D world

OPERATOR CONTROLS

NETWORK RING INTACT
Split network

BYZANTINE NODE ALL HONEST
node 0 · Entrance
Make it lie

CAMERA LENS ALL CLEAR
cam 0 · Entrance
Cover lens

EMPLOYEE
Priya Sharma
Send on a break

DASHBOARD
Reset view

PIPELINE

live, all nodes

CAMERA	DETECT	TRACK	APPEAR	LOCATE	VERIFY	CLAIM	GOSSIP	MERGE	RESOLVE
3,529	4 in view	2.0 fps	2,593 made	σ 0.23 m	3 on site	3,313	4/4 live	Δ8 · gaps 0	5 ids · 4 reg.

Node 0 · Entrance — ATTESTING

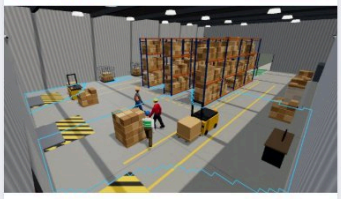


frame 99.2 ms · 2/2 fps
claims 733 made · 3310 in replica
rep. 1 · 4/733 rejected
links all up

occlusion	0.00
illumination	0.82
detector	0.92
attest (min)	0.82

nobody in view

Node 1 · Dock — ATTESTING



frame 104.8 ms · 2/2 fps
claims 972 made · 3308 in replica
rep. 0.995 · 53/964 rejected
links all up

occlusion	0.18
illumination	0.82
detector	0.98
attest (min)	0.79

#635 (27.9,3.8) ±0.07 conf 0.73

Node 2 · Packing — ATTESTING

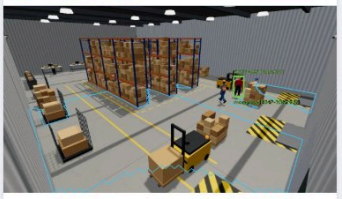


frame 102.7 ms · 2/2 fps
claims 207 made · 3305 in replica
rep. 0.992 · 64/912 rejected
links all up

occlusion	0.00
illumination	0.83
detector	0.97
attest (min)	0.83

#58 (3.4,16.4) ±0.07 conf 0.73 · EMP-1041

Node 3 · Outbound — ATTESTING



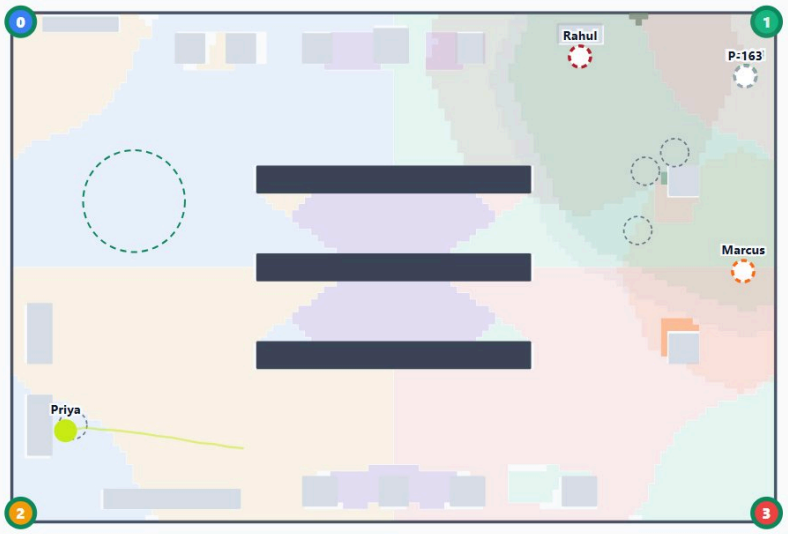
frame 103 ms · 2/2 fps
claims 681 made · 3302 in replica
rep. 0.999 · 17/678 rejected
links all up

occlusion	0.08
illumination	0.83
detector	0.97
attest (min)	0.83

#839 (3.3,16.5) ±0.45 conf 0.15 · EMP-1041 [unclaimed]
#857 (24.0,4.3) ±0.23 conf 0.39 [unclaimed]

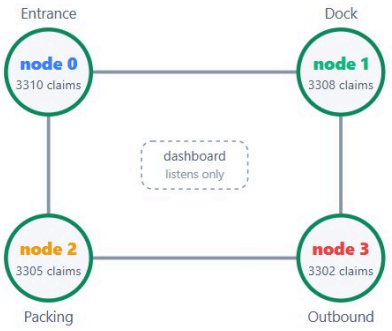
BELIEF VS REALITY

resolved | truth | C4 region | reachable



GOSSIP RING

ring intact



NODE	CLAIMS	GAPS	REP.	REJECTED	ZONE
0	3310	0	1	4/733	attested
1	3308	0	0.995	53/964	attested
2	3305	0	0.992	64/912	attested
3	3302	0	0.999	17/678	attested

RESOLVER

3,313 claims · 47 ms

IDENTITY	EMPLOYEE	STATUS	CLAIMS	LAST BY	POSITION	CONF.	VS TRUTH
EMP-1041	Priya Sharma	SEEN	55	node 0	(2.1, 16.4)	0.84	0.36 m
EMP-1027	Marcus Bell	UNSEEN 3s	95	node 1	(28.7, 10.2)	0.95	–
EMP-1002	Rahul Mehta	UNSEEN 4s	109	node 1	(22.3, 1.7)	0.98	–
EMP-1033	Daniel Kowalski	UNSEEN 6s	88	node 3	(28.8, 2.5)	1.00	–
P-163	unnamed	UNSEEN 7s	13	node 3	(28.8, 2.5)	0.99	–

C4 NEGATIVE EVIDENCE

4 regions

Marcus Bell — unseen 3 s

C4 region	reachable	blind space
1 m²	42 m²	1 m²

Rahul Mehta STALE DUPLICATE — unseen 4 s

C4 region	reachable	blind space
0 m²	72 m²	0 m²

Daniel Kowalski STALE DUPLICATE — unseen 6 s

C4 region	reachable	blind space
0 m²	95 m²	0 m²

P-163 STALE DUPLICATE — unseen 7 s

C4 region	reachable	blind space
0 m²	110 m²	0 m²

IDENTITY VERIFICATION

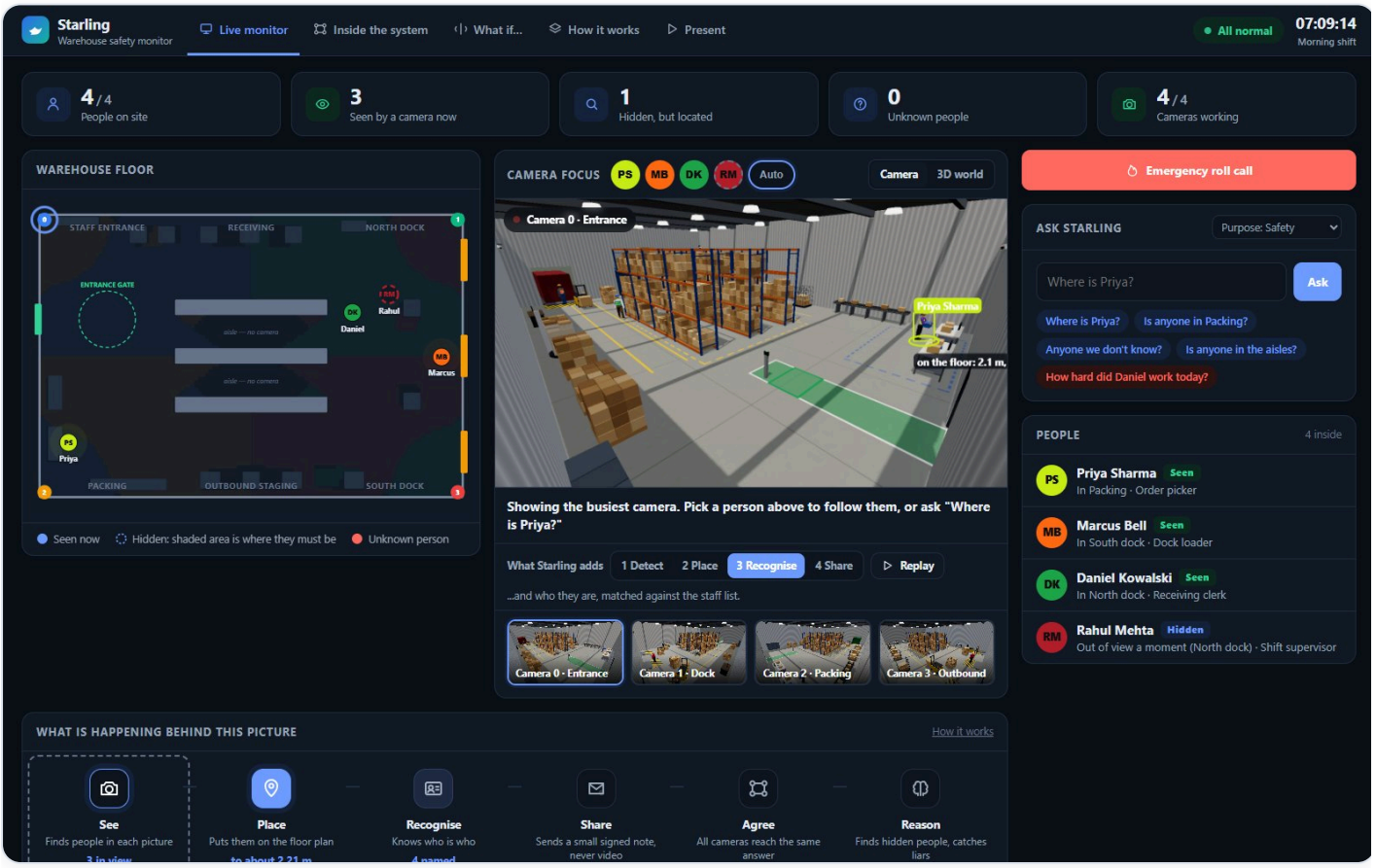
18

C5 QUERY

FORKS & EVENTS

2 forks

What you see, top to bottom. - **Operator controls:** split/heal the network, make a node lie, cover a lens, send an employee on a break, reset the dashboard's own view. Each shows its state as a tag ("RING INTACT", "ALL HONEST", "ALL CLEAR"). - **Pipeline:** ten stages with live counters: camera frames, detections in view, tracker fps, appearance signatures made, floor accuracy (sigma in metres), people verified, claims signed, gossip links, merge spread and gaps, resolver identities and regions. - **Four node tiles:** each node's *own annotated view of its last frame*, with frame time, claims made vs in replica, reputation as peers see it, rejected counts, and its own **coverage self-assessment**: occlusion, illumination, detector health and the resulting *attest confidence*. Below the threshold (0.7) the node **stops attesting**. - **Belief vs reality:** the resolved positions (solid), the C4 hidden-person regions (shaded) and the ground truth (dashed rings, **for grading only**). - **Gossip ring:** each node's claim count, gaps (0 when fully caught up), reputation, rejected/evaluated and zone status. - **Resolver output:** every identity, its employee, status, claim count, last node, position and error vs truth. - **C4 negative evidence:** per hidden person, the search area with negative evidence vs reachability alone, and how much of it lies in camera-less space. - **Collapsed:** identity verification log, the C5 query console, forks and events.

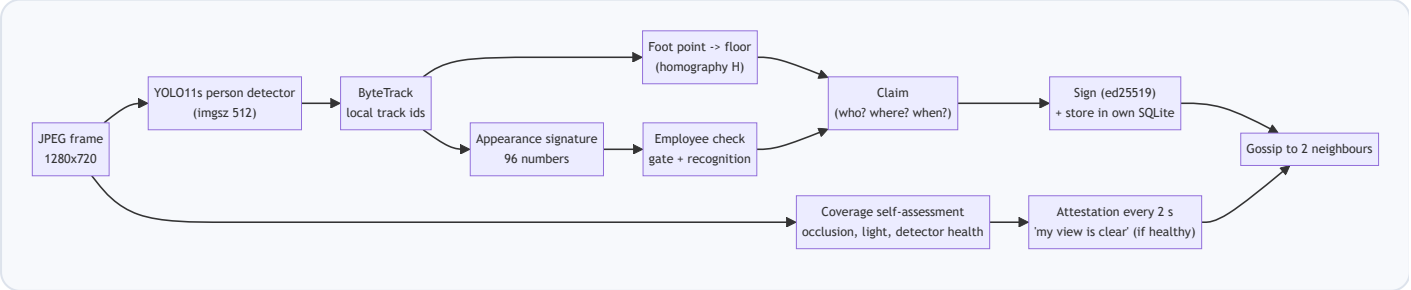


Dark mode. Every page follows the operating system's light or dark setting.

5. The ideas in depth

Starling's research contributions are named **C1 to C6** (C7 was cut). Each section below explains the idea in plain words, then how it is built, where the code is, and what was measured.

5.0 The perception pipeline inside one node (the part before the ideas)



- **Detection and tracking:** YOLO11s (COCO "person"), then ByteTrack gives a local track id per person. The detector confidence is deliberately low (0.15) so ByteTrack can keep a track alive on weak detections; a *new* track still needs 0.25. (`packages/starling_perception/world_pipeline.py`, `detector.py`, `tracker.py`.)
- **Floor position:** the **foot point** of the box goes through the camera's calibrated **homography** (a 3x3 matrix mapping image pixels on the floor plane to metres). When the feet are hidden (behind a rack) a **head-ray** fallback uses the head and the person's known height. Each claim carries a position uncertainty `pos_sigma` (`starling_geometry/calibration.py: position_sigma`). Measured median error: **0.18 m** (calibration test) and **about 0.3 m** in live runs.
- **Appearance signature:** a 96-number clothing-colour descriptor (6 horizontal stripes of the body, each a 12-bin hue histogram plus 4 value bins; `appearance.py`). It is **not a photo and cannot be turned back into one**.
- **Employee checks:** at the entrance gate, the average signature of 3 frames is compared with the **enrolled registry**; accepted if similarity is at least **0.80** with a margin of at least 0.06 over the runner-up. Away from the gate every camera also recognises employees (4 frames, threshold 0.80, margin 0.12) and re-anchors at most once per 8 s. A verified or recognised claim carries `anchor_type = FACE_ANCHOR` and the employee id.
- **Coverage self-assessment** (`coverage.py`): every frame the node estimates what fraction of its zone is obscured (a background model finds big blobs), how well lit the picture is, and whether its detector is healthy (frame rate achieved vs target, dropped frames, a drift test on detection confidences). `attest_confidence = min(1 - occlusion, illumination, detector health)`. Only if it is **at least 0.70** does the node sign an attestation. It attests with the *worst* value seen in the 2-second window, so a camera blind for part of it cannot vouch for all of it.

5.1 C1: identity as a partition-tolerant CRDT

Idea. Do not send "this is person 7". Send **observations**; let everyone *derive* identities from the same set.

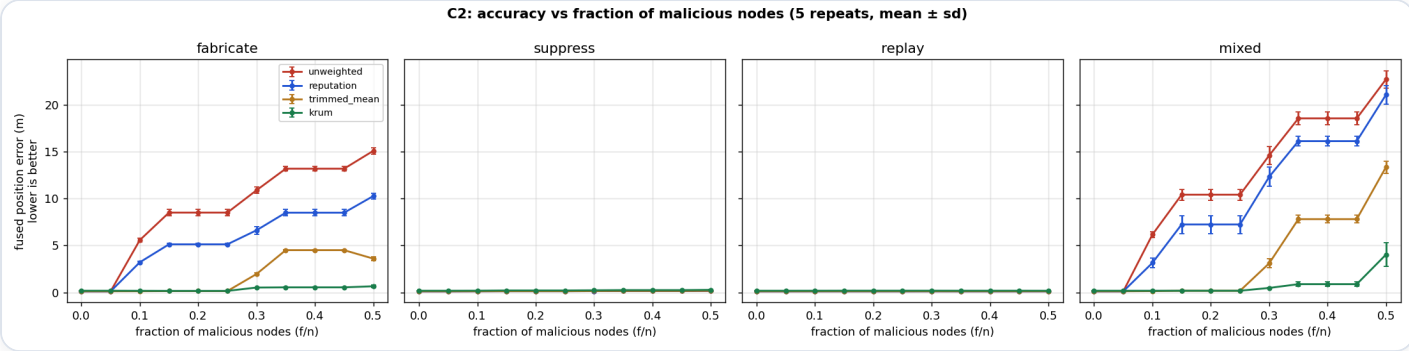
How it is built. - A claim is **immutable** and identified by `(node_id, seq)`. The set of claims is a **grow-only set**: merging two replicas is set union (`LocalStore.append_remote_claims` , idempotent on `(node_id, seq)`). - **Identity assignments are never replicated.** They are the output of `resolve(claims)` , a **pure function**: it sorts claims into one canonical order, `(hlc_physical_ms, hlc_logical, node_id, seq)` , then sweeps once: - A **FACE_ANCHOR** claim (a verified employee) binds directly to that employee's identity. - An unanchored claim is matched to a candidate branch by `score = appearance similarity × topology prior × reputation × claim quality` , but only if it is **physically reachable** from the branch's last position in the elapsed time, the similarity is at least `sim_threshold` (**0.70**, calibrated by an ROC curve on held-out renders), and it beats the runner-up by `margin_threshold` (0.10). Otherwise the claim is left **unassigned** rather than guessed (about 10% of claims in a live run). - **Anchor adoption:** a person tracked for a few frames before being verified would exist as two near-identical hypotheses; a verified anchor absorbs the one unanchored hypothesis that is plainly the same person. - **Determinism is the correctness condition.** No random numbers, no dict or set iteration order, no wall-clock reads (time comes from claim timestamps and hybrid logical clocks), floating-point operations in a fixed order. `assert_deterministic()` re-runs the resolver on 100 independently shuffled insertions and requires byte-identical output. - **Forks stay open** (rule 6). A new face anchor that cannot be explained as a continuation of the identity's live chain, but is reachable from its last *confirmed* position, creates a fork that is **not** resolved by score. It is closed only by reachability (exactly one branch possible, with the implied speed stated) or by a later anchor. - **Anchor slack:** two cameras naming the same person at the same instant disagree by their combined measurement error, which is not a conflict. `anchor_continue_slack_m = 3.5` m absorbs that for the live world (a far camera's floor estimate is less precise).

Where. `packages/starling_crdt/{claims,resolver,forks}.py` , `packages/starling_net/{gossip,anti_entropy,hlc}.py` . **Tests.** Property tests with Hypothesis (`tests/test_claimset.py` , `tests/test_hlc.py`) and determinism/fork tests in `tests/test_resolver.py` . **Measured.** Replicas reconverge after a partition in **2 anti-entropy rounds** (synthetic harness, `docs/results_c1.md`), about **2.5 s** at 16 real processes, **2 to 4 s** in the live demo; every replica identical at every size from 4 to 64 nodes.

5.2 C2: Byzantine-robust attestation and reputation

Idea. Signatures stop *forgery*, but a hacked camera can still lie in its own name. So every node **checks every other node's claims** and keeps a **trust score** for each peer.

How it is built. - **Plausibility check** per remote claim: reachability gate, kinematic limit (speed), corroboration by nearby claims from other cameras, freshness (replay), plus the **contradiction check** that uses other cameras' signed attestations (see 5.4). - **Reputation** is an exponential moving average of pass/fail: `r ← (1−α)·r + α·s` with `α = 0.05` , `s = 1` if the claim passed else 0, floored at `r_min = 0.05` so a repaired node can recover. Opinions are themselves gossiped (`ReputationUpdate` , under 100 bytes), and the dashboard shows the median of what peers report. - Resolver scores are multiplied by reputation, so a distrusted node's claims lose influence. - **Attack injector** (`starling_consensus/attacks.py`) for evaluation: `fabricate` (invent people), plus replay, drift and others, switchable live through a node's control endpoint.



Reading the chart: the horizontal axis is the fraction of nodes that are malicious; the vertical axis is the error of the fused position. Plain averaging (`unweighted`) collapses as soon as one node lies; reputation weighting, a trimmed mean and Krum hold up better.

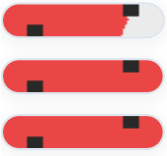
Measured (`docs/results_c2.md` , synthetic sweep: 10 nodes, one person walking, 880 runs). With a `fabricate` attack at f/n = 0.10 (1 of 10 nodes malicious) the fused position error is **5.58 m unweighted**, **3.21 m with reputation**, **0.13 m with a trimmed mean** and 0.17 m with Krum. In the live demo a liar's reputation falls below the 0.85 alert level in 15 to 25 s while honest cameras stay at about 1.0.

Honest limit. Lies placed inside the liar's *own* zone contradict nobody, so reputation settles near 0.8, not 0.

5.3 C3: geometry-constrained gap bridging

Idea. Use the **floor plan** to decide what is physically possible.

How it is built. `NavMesh` (`starling_geometry/navmesh.py`) is a 0.25 m grid built from the floor-plan GeoJSON (walkable floor minus racking and props). `ReachabilityModel` computes the **geodesic** (around obstacles, not straight-line) distance from any point, with maximum walking speed `v_max = 1.6 m/s` . `is_reachable(a, b, dt, sigma)` answers "could a person get from `a` to `b` in `dt` seconds?", permissively: slack = 2σ + one grid cell. It is used by the resolver (matching), plausibility (lies), and candidate regions (where a hidden person can be). The pictures below show the reachable set (shaded) growing from one point over 5, 15 and 60 seconds, bending around obstacles:

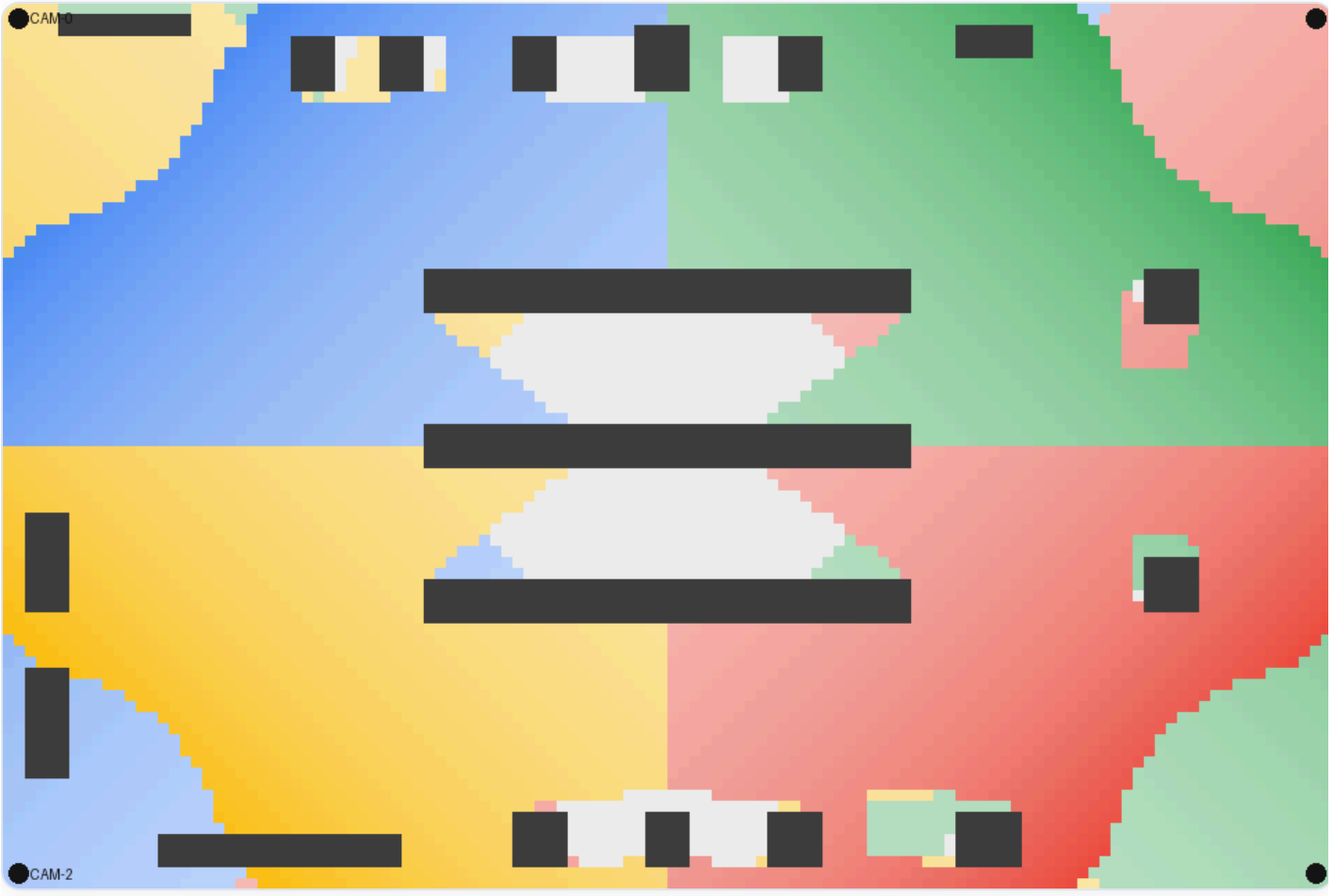


5.4 C4: negative evidence (the flagship)

Idea. *Absence of evidence is evidence*, but **only** when a camera is demonstrably able to see.

How it is built. 1. Every healthy node signs a `CoverageAttestation` every 2 s: "zone region 1000+id was covered, crossing not observed", with its `attest_confidence` . A node below the threshold signs **nothing**. 2. For a person no one currently sees, each dashboard (or any node) keeps a `CandidateBelief` : a probability grid starting at the last known position. Each tick it **spreads by walking speed** (a geodesic dilation) and then **removes every cell inside the zone of a camera whose attestation is recent (valid 4.5 s) and above** `tau_attest = 0.70` . 3. What remains is the

candidate region. A silent, low-confidence or stale camera removes **nothing** (rule 7). 4. A **safe reasoning loop with C2:** the contradiction check turns the same attestations into evidence against liars.



The warehouse's coverage map: each colour is the zone one camera can actually see a standing person in (computed from the camera geometry, not drawn by hand); the dark racks between them leave two aisles with no camera. This is where negative evidence earns its keep.

Seen in the demo: Priya hidden in the aisles: **16.7 m²** of search area instead of **about 250 m²** from physics alone (`docs/img/08_focus_hidden_aisle.jpg` ; review result `3_deadzone`). Cover camera 1's lens and the region *leaks* into camera 1's zone, with the reason stated.

Measured, and its honest limit (`docs/results_c4.md` , a synthetic dead-zone harness with a noisy attestation model): region reduced from 74 m^2 to 32 m^2 (**56.8%**), but the **false-exclusion rate was 41.2%** at `tau_attest = 0.7` in that harness, which assumed attestation noise (`P_BLIND = 0.3`) independent of whether the camera actually missed a crossing. The live system avoids this by deriving attestation confidence from the camera's own pixels and by attesting with the worst value in the window; but the abstract experiment is the reason the document calls false exclusion "safety-critical" and why `tau_attest` must be calibrated per site.

5.5 C5: calibrated, purpose-limited queries

Idea. Answers must separate **what was seen** from **what was inferred**, say who did not answer, and refuse surveillance.

How it is built. A **capability token** (`starling_query/capability.py`) names a *purpose* (`safety` , `incident` , `audit`), an optional area and a validity window, signed with an enrolled key. `verify()` is the one function every node calls: it rejects an unauthorised purpose, a bad signature, an expired window, or an out-of-scope area. `answer_query` returns: last confirmed position and time, the anchor, an *inferred* note, the candidate region area, and **completeness** ("4 of 4 nodes responded"), with blind spots named. Unanswerable queries get a clear refusal with the reason. **This is a structured stub, not a free-text language model** (a design decision).

5.6 C6: self-healing topology discovery

Idea. Learn which cameras are neighbours (who sees a person next after leaving whom, and how long it takes) instead of hard-coding it, and re-learn if a camera is moved.

How it is built. `TopologyLearner` (`starling_topology/learner.py`) observes **handoffs** (a person leaves camera A's zone and appears at B) and fits transit-time distributions per edge; the learned graph becomes the `topology` prior in the resolver. **Measured** (`docs/results_c6.md` , synthetic): after a simulated camera swap the new edges qualified after 111 s. **Honest limit:** it only *acquires* edges and has no decay yet, so the two old edges remain. In the live demo the gossip ring is configured by hand (`net.neighbours`); the learner is not driving it.

5.7 Hybrid logical clocks, and why no `time.time()`

Every claim has an HLC timestamp `(physical_ms, logical, node_id)` seeded from **media time**, not wall-clock. This (a) gives every node the identical total order of claims, which the deterministic resolver needs, and (b) keeps the identity path independent of clock skew. Wall-clock reads exist only for token expiry and for measurement harnesses.

---

6. Technical reference

6.1 The wire protocol (`packages/starling_proto/starling.proto`)

Every message is an `Envelope` carrying one of:

Message	Purpose	Size
`IdentityClaim`	claim id, node id, seq, HLC start/end, local track id (node-scoped only), appearance embedding + scale, floor position + sigma, anchor type + identity ref, confidence, quality, **ed25519 signature**	**about 528 bytes** measured (96-number signature = 384 bytes of it)
`CoverageAttestation`	node id, time window, region ids covered, occlusion, illumination, detector health, `attest_confidence` , signature	under 200 bytes
`ReputationUpdate`	opinion of one node about another	under 100 bytes
`VVDigest` / `VVDelta`	anti-entropy: "ranges of seq I hold" / "claims you were missing" (chunked under a size cap)	variable
topology observation	handoffs for C6	small

A receiving node verifies the signature against the **enrolled public key** of the sender and drops anything unsigned, badly signed, or too large (`assert_wire_safe`).

6.2 Storage (`packages/starling_store/identity_store.py`)

One SQLite file per node. The `claims` table holds the fields above plus the signature; `(node_id, seq)` is the primary key. Reads/writes take a lock (`check_same_thread=False`). Old claims are pruned past `retention_window_s` (default 3600 s). The resolver normally sweeps only the last `resolve_window_s` (the dashboard uses 60 s) so a long run stays fast.

6.3 A node's control endpoint (operator HTTP, port = gossip + 1000)

Route	What it does
`GET /status`	attack and partition state
`GET /perception`	the node's last detections, events, coverage numbers and frame rate
`GET /preview.jpg`	the node's own annotated last frame
`GET /replica?t_end=&window_s=&token=`	**ask this camera directly** (verifies the capability token itself)
`POST /attack` , `POST /partition`	demo faults: make this node lie / ignore chosen peers
`POST /shutdown`	exit cleanly

6.3b The dashboard API (port 8765)

Route	Use
`GET /api/state`	the one JSON snapshot every page reads (people, unknown people, regions, nodes, forks, rejections, health, central server, network bytes)
`POST /api/ask` , `GET /api/rollcall` , `GET /api/ask_cameras`	questions, roll call, per-camera answers
`GET /api/compare` , `GET /api/scale` , `GET /api/gossip_feed` , `GET /api/sample_claim`	comparison page, scaling data, live notes, one decoded note
`GET /api/camera/{n}.jpg` , `GET /api/node/{n}/perception`	live camera picture and detections
`POST /api/partition` , `/api/heal` , `/api/lie` , `/api/stop_lie` , `/api/node/{n}/kill` , `/api/node/{n}/restart` , `/api/central/kill` , `/api/central/restart` , `/api/world/block` , `/unblock` , `/break` , `/visitor`	the scenario buttons

6.4 Configuration (no magic numbers)

Every threshold is a **Pydantic** model loaded from YAML (`packages/starling_node/config.py` , `configs/`): match thresholds, `tau_attest` , `v_max` , reputation `alpha` , gate and recognition thresholds, neighbour list, frame rate. A

world node's file is `configs/nodes/world/node-0N.yaml`; the world itself is `configs/world/warehouse.yaml`; the dashboard is `configs/world/dashboard.yaml`.

6.5 Logging and quality

Logging is **structlog JSON**, every record tagged with `node_id`. Tests: pytest, with **Hypothesis property tests** for the CRDT and resolver. At the time of writing: **497 unit tests pass**, plus 10 integration tests that start real node processes. The original centralised system (`apps/baseline.py`, from Kunal Gaikwad's `multicam-reid` thesis, MIT licence) is kept **frozen** as the control condition for every comparison.

6.6 How the demo launcher works (`scripts/run_world_demo.py`)

Starts, each as its own OS process and in this order: the **world server**, the **camera rig**, the **central comparison server**, the **four nodes** (one after another, each once the previous one's detector is loaded, since four simultaneous torch imports spike memory), then the **dashboard**; only then does the shift start. It warns when less than 4 GB is free and then switches to **light mode** (cameras at 2 frames per second). Each node runs with `OPENBLAS_NUM_THREADS=1`, which saves about 740 MB of committed memory per node.

---

7. Numbers you can quote, and the honest limits

7.1 Measured

Claim	Number	Source
Notes vs video	one claim ≈ **528 B** vs a video frame ≈ **150,000 B**	`/api/sample_claim`, `docs/img/20_inside_normal.jpg`
Network traffic vs video (4-camera demo)	about **50 to 120x less** (depends on moment: e.g. 195 kbit/s vs 10 Mbit/s)	live pages, `/api/state`
Floor localisation (calibration test)	median **0.18 m**, 90th percentile 0.37 m	`docs/results_world_perception.md`
Floor localisation (live runs)	median **0.28 to 0.33 m**, 90th pct 0.6 to 0.75 m	`review/world/review_summary.md`
Person detection recall (single frame, inside zone)	**74%** (ByteTrack carries tracks through dips)	`docs/results_world_perception.md`
Cross-camera same-person vs different-person	**AUC 0.9965**; chosen threshold 0.70 gives 97% TPR / 0% FPR	same
Gate verification	**15/15** correct, **0 false accepts**	same
Recognition by every camera	**41/41** correct, **0 false**	same
Identity purity in a 4-minute live run	**99.4%** of 2,441 claims attributed to the right person; 23/23 named identity windows correct; 10% claims left unassigned	`scripts/evaluate_world_run.py`, `docs/LIVE_WORLD.md`
Hidden person search area	**16.7 m²** vs **~250 m²**	review moment `3_deadzone`
Time to notice a covered lens	**4 to 6 s**; resumes 3 to 5 s after clearing	review moment `4_lens`
Network split then heal	both halves keep tracking; identical again in **2 to 8 s** (demo), **2.5 s** (scale test)	review `5_partition`, `docs/results_scale.md`
Liar caught	reputation < 0.85 in **14 to 25 s**; honest cameras stay ≥ 0.95	review `6_lie`

Claim	Number	Source
Stranger flagged	about **6 s** after entering (≈ 14 sightings)	live test
Every camera answers identically	fingerprints match (4 of 4)	review `9_every_camera`
Scaling 4 to 64 nodes	every replica identical at every N; latency p50 0.68 to 2.17 s	`docs/results_scale.md`
Busiest node vs central ingest at 64 nodes	**282 KB/s** vs **23 MB/s** (82x; 113x with H.264 CCTV rates)	same

7.2 Honest limits (say these before you are asked)

1. **The warehouse is a 3D simulation.** The cameras are virtual. The detection, geometry, networking, CRDT and reasoning are the real code, but there has been **no run on real CCTV footage yet** (the video-file/RTSP input path exists in `apps/node.py` and `starling_perception/source.py` (`PacedSource` , covered by `tests/test_paced_source.py`), but it has not been run end-to-end on real multi-camera footage).
2. **Appearance is clothing colour.** The signature separates four distinct uniforms well (AUC 0.9965) but is *not* a general re-identification model. People dressed identically are separated by **movement physics and the gate check**, not by looks. A production system would use a trained re-ID model; an OSNet backend exists in `starling_perception/embedder.py` but was not validated here.
3. **Every camera keeps every note** (full replication). Per-node incoming traffic grows with the number of nodes, and *total* traffic grows with the square of the camera count. It stays 80 to 120x below a central video ingest at the busiest point, but **crosses "everyone streams video" at roughly 80 to 110 cameras**. Neighbourhood-only replication is the planned fix and **is not built**.
4. **Lies inside the liar's own unwatched area cannot be contradicted** (nobody else saw that spot).
5. **C4 false exclusion** in the synthetic harness was 41% at `tau_attest = 0.7` ; the live system derives attestation confidence from pixels, but the threshold needs per-site calibration.
6. **Topology learner** only acquires edges (no forgetting); the live ring is hand-configured.
7. **C5 is a structured stub**, not free-text language understanding.
8. **The centralised-vs-Starling headline experiment is not yet measured** (`docs/results_headline.md` is a scaffold).
9. **Under heavy machine load** a camera can mislabel someone; Starling then shows "needs a check" (an open fork) rather than guessing. That is intended behaviour, but it looks alarming; the dashboard shows an "overloaded" banner when the nodes fall behind their frame rate.

---

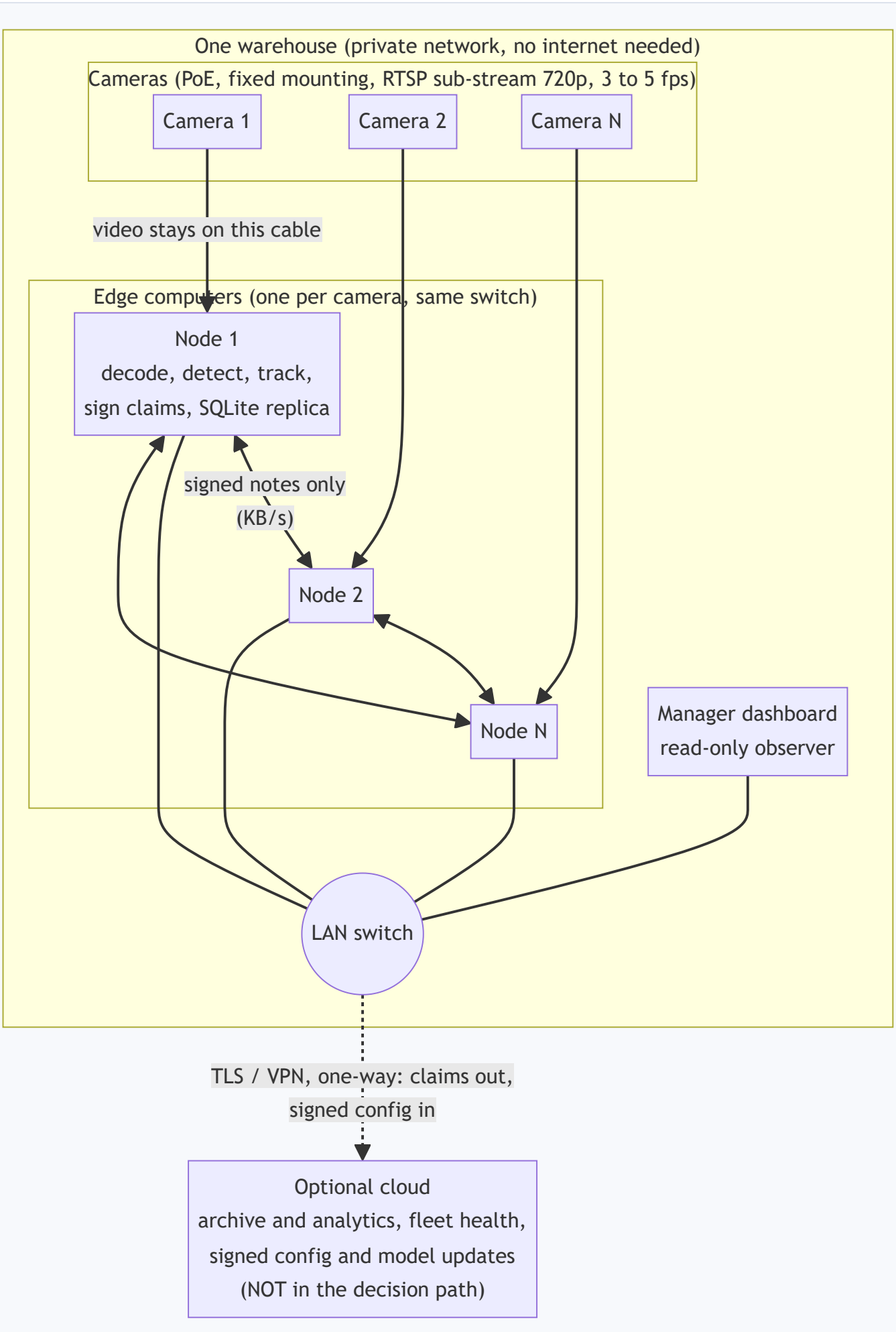
8. From demo to a real warehouse with CCTV

> **Read this first.** Sections 8.1 to 8.3 describe the *design intent* and map each simulated part to its real counterpart. Parts that exist in the repository are named with their file; parts that are **not built or not validated** are marked **[not built]** or **[not validated]**. Do not present the latter as done.

8.1 What changes when the cameras are real

In the demo	In a real warehouse	Status
A virtual camera renders a JPEG and publishes it on a ZeroMQ topic	An IP camera's **RTSP** stream (H.264/H.265) decoded on the node	input path exists (`source: "rtsp://..."` , `starling_perception/source.py`); **not run on real multi-camera footage**
Calibration generated from the camera pose (exact)	**Measured**: checkerboard intrinsics + floor-point homography	scripts exist (below); procedure **[not validated on a real site]**
Floor plan GeoJSON generated from the world config	**Surveyed** floor plan drawn in the same GeoJSON format	format exists; tooling to draw it is manual
Four employees recognised by clothing colour	A trained **re-identification** model plus the gate check; identically dressed staff separated by movement and gate	OSNet backend exists (`embedder.py`), **[not validated]**; the demo's colour signature is *not* suitable
"Cover the lens" = a 3D tarp	A dirty/blocked/dark lens, an overloaded detector	the pixel-based coverage assessor is real and acts on any frame
"Cut the network" = software drop	A switch fails, a cable is cut, Wi-Fi drops	same mechanism (anti-entropy) heals it; `deploy/netem` can inject real packet loss
One laptop runs everything	**One edge computer per camera**, on the site network	process model is identical (one OS process per node)
Dashboard on the same laptop	A manager PC or tablet on the site network	read-only observer, no special access

8.2 What a real deployment looks like


One warehouse (private network, no internet needed)
Cameras (PoE, fixed mounting, RTSP sub-stream 720p, 3 to 5 fps)
Camera 1
Camera 2
Camera N
video stays on this cable
Edge computers (one per camera, same switch)
Node 1
decode, detect, track,
sign claims, SQLite replica
signed notes only
(KB/s)
Node 2
Node N
Manager dashboard
read-only observer
LAN switch
TLS / VPN, one-way: claims out,
signed config in
Optional cloud
archive and analytics, fleet health,
signed config and model updates
(NOT in the decision path)

The key property: the decision path (who is where, who is lying, who is missing) lives entirely **inside the site, among the edge computers**. Cutting the site's internet changes nothing about safety.

8.3 Installation procedure for a real warehouse

This is the order to follow. Commands refer to scripts that exist in the repository.

Step 0. Survey and plan. Decide camera positions so that **entrances, exits and aisle ends are covered**: the dead-zone reasoning depends on cameras watching the *ways out* of any area they cannot see into. Avoid very steep top-down views: a COCO-trained detector degrades near top-down (the demo caps depression angle at about 47 degrees).

Step 1. Draw the floor plan as a GeoJSON file (the format of `data/floorplan/warehouse_world.geojson`). Features carry a `role`: `camera_zone` (with `node_id`), `obstacle` (racking, props; `kind: rack`), `blind` (areas no camera covers), plus boundaries. Metres, x east, y south. Check it loads: `NavMesh.from_geojson(path, cell_size_m=0.25)`.

Step 2. Mount cameras and get streams. Use the camera's **sub-stream** (about 1280x720, 3 to 5 frames per second is enough; the demo uses 2 to 3). Note each RTSP URL. Synchronise clocks with NTP (or PTP): claims use **media time**, so cameras should agree to within a fraction of a second.

Step 3. Calibrate each camera (`scripts/`): 1. `python scripts/calibrate_intrinsics.py --node-id 0 --images data/calib_images/cam0/` (checkerboard photos, or `--webcam 0`): solves the camera matrix `K` and lens distortion and writes `configs/calib/cam-00.yaml`. It **refuses to write** a calibration with reprojection error above **1.0 px**. 2. `python scripts/calibrate_homography.py --node-id 0 --image data/calib_images/cam0/floor_ref.jpg`: click 4 or more points on the floor in the image and type each point's real floor coordinates in metres (measure with a tape). It solves the floor homography `H` and reports each point's residual in metres. 3. `python scripts/validate_calibration.py --calib configs/calib/cam-00.yaml --video cam0.mp4 --known-distance-m 10`: the "10-metre walk" test. Click both ends of a known line; it must reconstruct the distance within **0.3 m**.
Everything Starling knows about position comes from this one matrix.

Step 4. Generate keys. `python -m starling_net.keys --generate N` creates one ed25519 key pair per node. Each node keeps its **own private key**; the **public keys of all nodes** are distributed to every node (and to the dashboard). This is the "enrolment" list: only enrolled keys are believed.

Step 5. Write one config per node (`configs/nodes/...`, start from `NodeConfig.write_template`): `node_id`, `source: "rtsp://user:pass@camera/stream"`, `calib_path`, `db_path`, `net.listen_port`, the `net.neighbours` list (a **ring or small-degree graph**, never everyone), `geometry.navmesh_path`, and the `coverage.roi_polygon` (the camera's own zone). Constants (thresholds) stay in the config, never in code.

Step 6. Enrol the staff. Capture each employee at onboarding (the demo's `scripts/enroll_employees.py` renders badge views; a real site would use onboarding photos through the same signature function) and write the registry (`data/world/employee_registry.json`). **Decide the identity method before rollout**: the clothing-colour signature is only suitable where uniforms differ; otherwise plug in a trained re-ID embedder and re-measure.

Step 7. Calibrate the thresholds on your own site. Run `scripts/calibrate_match_threshold.py` (an ROC curve of same-person vs different-person similarity on your footage picks `sim_threshold`), set `tau_attest` by walk tests (it decides when a camera is trusted to say "nobody here"), and check `v_max` (walking speed). The demo's numbers (0.70, 0.70, 1.6 m/s) are for the demo world.

Step 8. Start the nodes. Per node: `python apps/node.py --config configs/nodes/site/node-03.yaml`. For production run it as a service (a `systemd` unit, or `docker compose -f deploy/docker-compose.yml up -d`, which already gives each node its own volume so no container can read another's database). A crashed node restarts from its SQLite replica and anti-entropy fills what it missed.

Step 9. Start the dashboard with the list of node gossip addresses (`peers:`) and the public keys. Issue **capability tokens** for the purposes you allow (`apps/demo_dashboard/tokens.py` shows signing; the issuing authority's key should be held by the safety officer, not by the dashboard).

Step 10. Acceptance tests (all exist as buttons in the demo and as automated checks in `scripts/review_world_demo.py`): walk through every zone and every aisle; cover a lens (a node must stop attesting within seconds); switch a node off; cut the network and heal it; make a node lie in a test; send an un-enrolled person in. Record the results.

Step 11. Operate. Watch the dashboard's *overloaded* banner (a node below its target frame rate misses people), reputation alerts (a node is being contradicted), unwatched-area alerts, and replica gaps.

8.4 Hardware sizing (estimates; validate on your site)

Item	Estimate	Basis
Edge compute per camera	a small PC with a GPU/NPU (e.g. an embedded GPU module) or a modern mini-PC; **YOLO11n** if CPU-only	the demo runs YOLO11s at 512 px, **about 200 to 280 ms per frame** with four nodes sharing one laptop CPU (2 to 3 fps each). **[not measured on edge hardware]**
Memory per node	about **0.5 GB** working memory with the numpy thread pin; allow 2 GB	measured: 491 MB committed per node process
Camera to node link	2 to 4 Mbit/s H.264 sub-stream on the camera's own cable/VLAN	industry-typical
Node to node traffic	about **tens of kbit/s** per camera at demo rates (see section 9)	measured
Storage per node	claims are about 0.5 KB each; at 2.6 claims/s per camera that is about 5 MB/hour per camera's own claims, times N cameras if every node keeps every claim; pruned after the retention window	measured rates, computed

8.5 Security, privacy and compliance notes

- **Authenticity:** every claim, attestation and reputation update is signed (ed25519). A node cannot forge another's note. **Revocation of a compromised key is [not built]**; today you would remove its public key from every node's key directory.
- **Privacy by construction:** video never leaves the camera's own edge computer. What is shared is a floor position and a clothing-colour signature (or a re-ID vector in a real deployment), which is still personal data about movement.
- **Purpose limitation:** safety, incident and audit questions are answered; productivity monitoring is refused by each node's token check. This is a *technical* control that supports, but does not replace, a legal basis, employee notice and (where applicable) works-council agreement.
- **Retention:** claims older than `retention_window_s` are pruned from each replica. Set it to your policy.
- **Attack surface:** the control endpoint on each node (`/attack` , `/partition` , `/shutdown`) is **demo-only** and must be disabled (`attack.enable_control_endpoint: false`) or firewalled in production.

8.6 What would you build next to be production-ready? (a credible roadmap)

1. Run the RTSP path on real multi-camera footage; measure detection, localisation and identity on it.
2. Replace the colour signature with a validated re-ID model; re-calibrate thresholds on site data.
3. Neighbourhood-scoped replication and relay (section 9).
4. Key revocation and signed configuration distribution.
5. Run the three-way headline experiment (centralised V1 vs Starling healthy vs Starling under partition and attack).
6. Port the node to edge hardware and measure frame rate per watt.

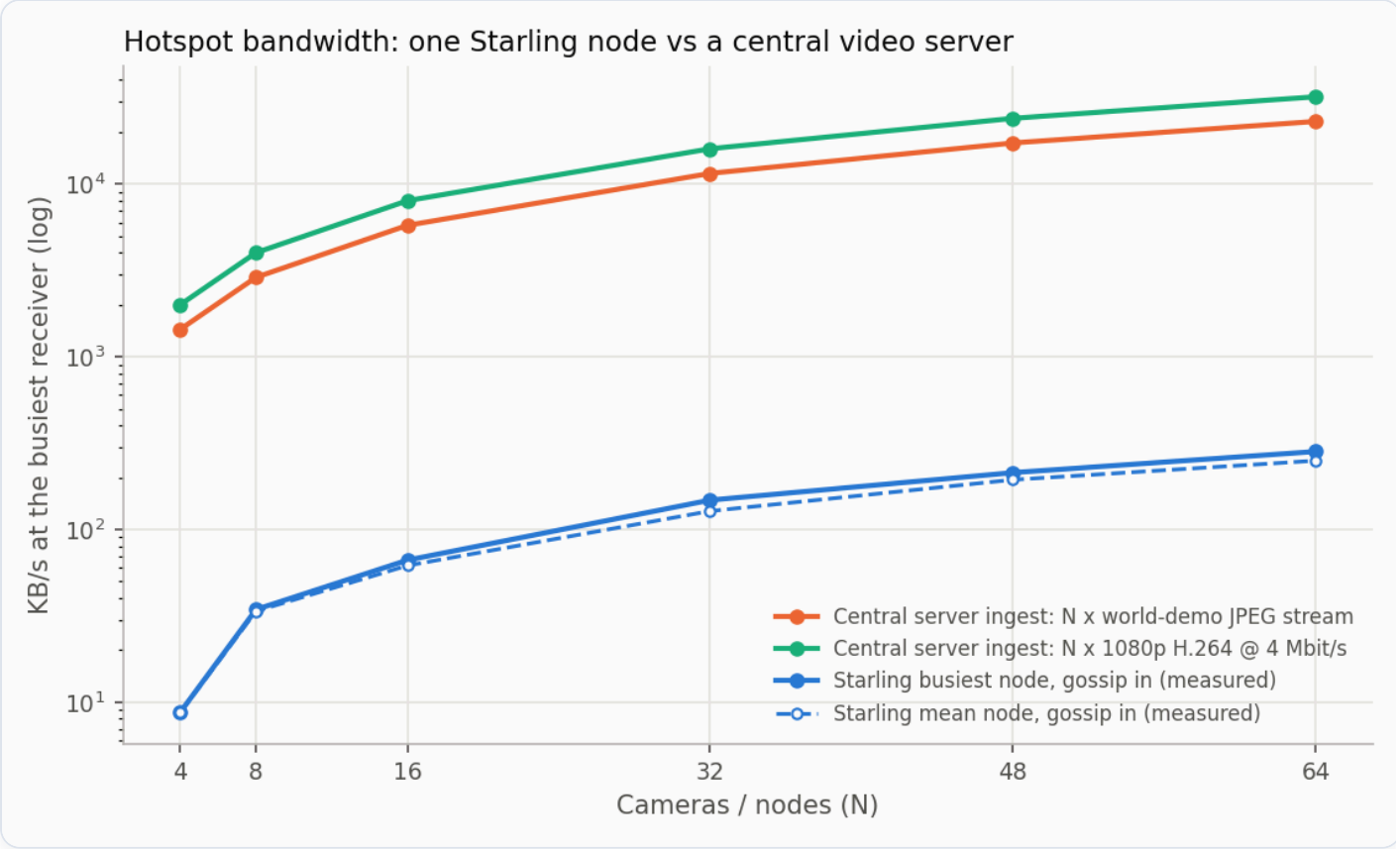
---

9. Scalability and the cloud

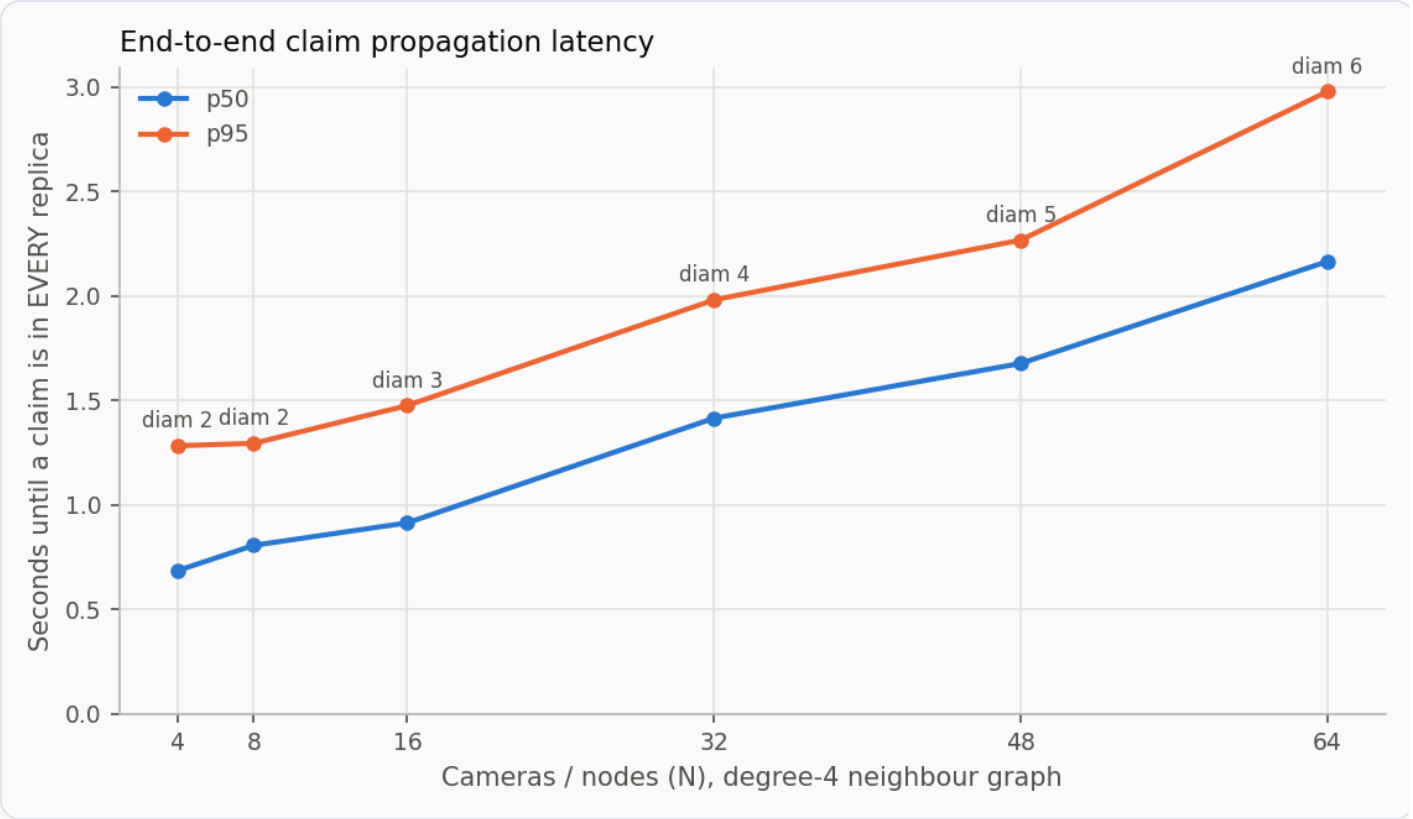
9.1 What was measured

`scripts/run_scale_experiment.py` starts **N real node processes** (4 to 64), each with its **own SQLite replica**, **own signed gossip socket**, and a **configured set of 4 neighbours** (ring plus a deterministic chord; the 4-node point is the demo's ring), producing claims at a live camera's rate (**2.6 per second per node**) for 40 s. It uses the real gossip, anti-entropy and storage code (`apps/bench_node.py`), and measures bytes, propagation latency and whether every replica ends identical.

Nodes	Hops across (diameter)	Traffic *into* a node, mean / max (KB/s)	Time for a claim to reach every replica, p50 / p95 (s)	All replicas identical?
4	2	8.7 / 8.7	0.68 / 1.28	yes
8	2	33.6 / 34.3	0.81 / 1.30	yes
16	3	61.7 / 66.3	0.91 / 1.48	yes
32	4	127.7 / 147.7	1.42 / 1.98	yes
48	5	193.8 / 213.1	1.68 / 2.27	yes
64	6	249.9 / 282.3	2.17 / 2.98	yes



Reading the chart. The lower line is the traffic into the **busiest Starling node**; the upper line is the video a **central server** must ingest (N cameras times the stream rate), on a log scale. At 64 cameras the busiest Starling node receives about **282 KB/s** while a central server ingests **23 MB/s** (world-demo JPEG rate) or **32 MB/s** (typical 1080p CCTV at 4 Mbit/s): **82x to 113x** more.



Latency. The first hop takes 2 to 3 ms; each further hop waits for an anti-entropy round (about 0.2 to 0.4 s at the median). That is why latency rises slowly with the number of hops.

Partition test (16 nodes). The graph was split into two halves of 8 for 15 s while every node kept producing (631 claims made during the split, none crossing). After the heal, every claim made before the heal was in every replica **2.52 s later** and all replicas ended identical.

CPU and memory. CPU rose from 0.3% to 6.9% per node and memory stayed flat at 62 to 67 MB across 4 to 64 nodes (bench nodes carry no vision model).

Full write-up: `docs/results_scale.md`.

9.2 Why it behaves like this, and where it stops

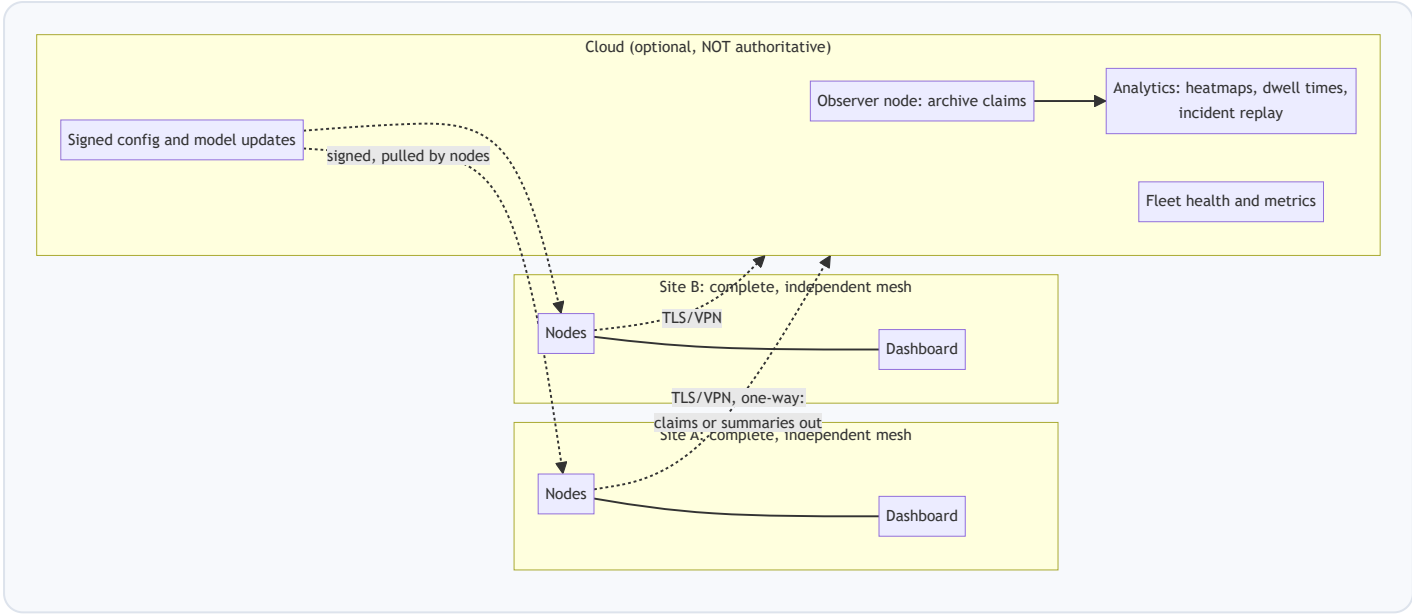
- **Every node keeps every claim** (full replication). So the traffic *into* each node grows about **linearly** with the number of nodes (each node must receive every other node's claims), and the **total** network traffic grows with its **square**.
- Measured detail: at 64 nodes **97% of incoming bytes are anti-entropy replies**, and each replica receives each claim about **2.6 times** (duplicates). The **minimum possible** at 64 nodes is about 64 × 2.6 × 528 B ≈ 88 KB/s, against the measured 250 KB/s: so there is roughly a **3x saving** available just from de-duplication.
- **Crossover (honest):** under full replication, *total* Starling traffic overtakes "every camera streams video to one server" at about **79 cameras** (world-demo stream rate) or **110 cameras** (typical H.264). But that crossover is about the sum over the whole network. **Per link and per machine, Starling stays far below a central video ingest** (the central server needs a machine that can swallow *all* streams; no Starling machine ever does).

What grows with the site, and what to do about it (**designs, not built**):

Cost	Why it grows	Remedy
Traffic into each node	full replication: O(N)	**Scope replication by physical neighbourhood**: a node needs the claims of cameras within *k* hops of its zone to track people near it; far claims are only needed as summaries. With scoping to about 13 neighbours, per-node load would stay near the **16-node figure (about 60 KB/s)** however big the site (estimate from the measured point).
Duplicates	anti-entropy replies re-send what already arrived	**Relay-and-dedupe** (each node forwards new claims once, tree-style) and slower digests; the 2.6x duplicate factor suggests about 3x saving.
Claim size	a 96-number float32 signature = 384 B of 528 B	Quantise the signature to 8 bits (about **100 B**, total claim about 250 B): roughly **2x smaller**.
Storage	every node stores every claim	Retention window plus scoped replication; archive to the cloud if history is needed.
Resolver cost	O(claims × identities)	Already windowed (last 60 to 300 s) and incremental; shard by zone for very large sites.
Latency	diameter × anti-entropy round	Add shortcut links (chords) to cut diameter; keep rounds at 0.2 to 0.4 s.

A rough extrapolation for a 100-camera warehouse (full replication, *extrapolated* from the 64-node trend of about 4 KB/s per added node): about **400 KB/s ≈ 3 Mbit/s into each node**, which is small next to the 2 to 4 Mbit/s of video each node already decodes locally. The **central alternative** is 100 cameras × 4 Mbit/s = **400 Mbit/s** into one machine, which must also decode and run detection on 100 streams.

9.3 Where the cloud fits (and where it does not)



The principle: the cloud must be an *observer*, not an *authority*. If the cloud decided who is where, it would rebuild the single point of failure Starling removes. So:

- **Site safety logic stays on site.** A flaky internet link changes nothing about alerts or roll call.
- **A cloud "observer node"** subscribes to claims exactly like the dashboard does (read-only), over an encrypted tunnel, and stores them for analytics and incident replay. Because claims are small signed facts, archiving is cheap (about 5 MB per camera-hour).
- **Multi-site:** each site is its own mesh. Sites exchange only **summaries or queries** (with capability tokens), never raw claims by default, which is both a privacy and a bandwidth choice.
- **Fleet operations:** the cloud pulls node health (frame rate, reputation, coverage) and publishes **signed** configuration and model updates that nodes pull and verify.
- **What not to put in the cloud:** the resolver's decisions, the attestations that gate safety alarms, or any raw video.

This is a **design**; none of the cloud components exist in the repository yet. The pieces that would make it easy do exist: claims are self-contained and signed, the dashboard already shows a read-only observer works with no privileges, and the capability-token system already expresses purpose-limited access.

---

10. Interview questions and answers

Q1. In one sentence, what is Starling? A camera network with no central server where each camera thinks for itself and shares small signed facts, so together they know where everyone is, even people no camera can see.

Q2. Isn't this just object detection? No. Detection is stage 1 of 6. Starling also places people on a floor plan in metres, recognises employees, signs and shares notes, makes four independent computers reach the same answer, and reasons about what is *not* visible (hidden people, lying cameras, cut networks). The Live monitor's layer selector (1 Detect to 4 Share) shows this on a live camera picture.

Q3. Why no central server? A server is simpler. A server is a single point of failure, needs all video in one place (bandwidth, storage and privacy risk), and its network link is a second single point of failure. The comparison page demonstrates it: kill the server and a traditional system places 0 of 4 people; Starling still places everyone. Starling trades simplicity for resilience and privacy.

Q4. How do the cameras agree without a leader? They never agree on an *answer*; they agree on **facts**. Facts are signed, immutable claims in a grow-only set. Merging two copies is set union, which gives the same result in any order. The answer is a **pure, deterministic function** of that set, so equal sets give equal answers. That is a CRDT.

Q5. What is a CRDT and why does it matter here? A conflict-free replicated data type: copies can be updated independently and merged automatically without conflicts. It matters because the network can be cut and cameras can die; after reconnection no negotiation or leader is needed.

Q6. What exactly is sent between cameras? Signed claims (about 528 bytes: who/where/when, floor position and uncertainty, a 96-number appearance signature, an ed25519 signature), coverage attestations (under 200 B), reputation updates (under 100 B) and anti-entropy digests/deltas. Never video or frames.

Q7. Why gossip to only two neighbours? Wouldn't broadcast be faster? A full mesh is a coordinator in disguise and does not scale (every node talking to every other). A small neighbour set keeps per-node connections constant; anti-entropy ensures everything still reaches everyone, at the cost of a few hundred milliseconds per hop.

Q8. How do you find someone no camera can see? Negative evidence. Healthy cameras sign "my view is clear and I saw nobody". The person's possible region grows with walking speed around the floor plan and is *cut* by each healthy camera's zone. What remains is where they must be (16.7 m^2 vs about 250 m^2 for physics alone in the aisle scenario).

Q9. What if a camera is broken or covered, and says nothing? Its silence counts for nothing (design rule 7). A camera with low attest confidence (occluded lens, dark, slow detector) stops signing, so its zone stays "possible". The system never treats a broken camera's silence as "empty".

Q10. What if a camera is hacked and lies? Signatures stop forgery but not a lie in its own name. Every node checks each claim for reachability, speed, corroboration, freshness, and contradiction with other cameras' signed "nobody here". Failed checks lower that camera's reputation (EWMA, alpha 0.05), and its claims count for less. In the sweep, with 1 of 10 nodes fabricating, reputation weighting cut the error from 5.58 m to 3.21 m and a trimmed mean to 0.13 m.

Q11. What are the limits of the lying defence? A lie inside the liar's *own* zone contradicts nobody, so trust only falls to about 0.8. A coordinated majority of liars defeats any reputation scheme (the sweep goes to f/n = 0.5).

Q12. What happens when the network is cut? Both halves keep tracking and answering on their own copies. When reconnected, anti-entropy exchanges the missing claims and every copy is identical again (2 to 3 s measured). If both sides saw genuinely incompatible things, the **fork stays open** for a human; it is never resolved by picking the higher score.

Q13. Why does a fork stay open instead of choosing the best score? Scores can be wrong, and a silent wrong choice in a safety system is worse than an honest "needs a human check". This is design rule 6.

Q14. How is identity decided if it is not replicated? `resolve(claims)` is deterministic and runs on every replica. Verified-employee claims bind directly; others are matched by appearance similarity × topology prior × reputation × quality, but only if physically reachable, above a calibrated 0.70 similarity and a 0.10 margin over the runner-up. Otherwise the claim is left unassigned rather than guessed.

Q15. How do you guarantee determinism? Canonical claim order by `(HLC physical ms, HLC logical, node id, seq)`, no randomness, no unordered iteration, fixed floating-point order, and no wall-clock reads in the identity path (time comes from claim timestamps and hybrid logical clocks). A test re-runs the resolver on 100 shuffled insertions and requires byte-identical output.

Q16. Why hybrid logical clocks? They give every node the same total order of claims despite clock skew, while staying close to real time (needed for reachability reasoning).

Q17. How does the system know someone is a stranger? An identity with no verified check-in at the gate and enough sightings (14, about 8 s) is "unknown". It is tracked by appearance and physics, flagged, and the camera panel switches to them. It never guesses a name.

Q18. How are employees recognised? At the entrance gate: average the clothing-colour signature over 3 frames and compare with the enrolled registry (threshold 0.80, margin 0.06). Any camera later re-recognises them (4 frames, 0.80, margin 0.12). In the held-out test: 15/15 correct at the gate and 41/41 elsewhere, 0 false accepts.

Q19. Is a clothing-colour signature a real re-ID method? No, and the guide says so. It works for distinct uniforms (AUC 0.9965). A real deployment needs a trained re-ID model (an OSNet backend exists but is unvalidated here), plus the movement-physics and gate checks that separate identically dressed people.

Q20. Is it tested on real cameras? Not yet. The perception, geometry, networking, CRDT and reasoning are the real code running on rendered pixels from a 3D simulation. The RTSP input path exists but has not been run on real multi-camera footage. That is the first item on the roadmap.

Q21. How does it scale? Measured up to 64 real node processes: every replica identical, p50 propagation 0.7 to 2.2 s, and the busiest node receives 80 to 120x less than a central video ingest. Honest limit: with full replication the *total* traffic grows with the square of the cameras and crosses "everyone streams video" at about 80 to 110 cameras. Neighbourhood-scoped replication (not built) removes that.

Q22. What breaks first at large scale? Per-node incoming bandwidth from full replication (linear in N), then storage. The resolver is windowed. Latency grows with network diameter.

Q23. What would you change for 500 cameras? Scope replication by physical neighbourhood (nodes keep their area's claims plus summaries of the rest), relay-and-dedupe gossip instead of anti-entropy-only, 8-bit signatures (about 2x smaller claims), shortcut links to bound diameter, and zone shards of the resolver. Sites would federate through summaries.

Q24. Where does the cloud fit? As an observer, never an authority: archive claims for analytics and replay, collect fleet health, distribute signed configuration. Safety logic stays on site so a dead internet link changes nothing.

Q25. How do you keep it private? Video never leaves the camera's own edge computer. Questions need a signed purpose; productivity monitoring is refused by every camera. Retention is bounded. Caveat: floor position and appearance are still personal data; a legal basis and employee notice are required.

Q26. How is the 'capability token' enforced? It is signed by an enrolled authority key and names a purpose, optional area and validity window. Each node's `/replica` endpoint calls `verify()` itself and refuses an unauthorised purpose, a bad signature or an expired token. So the refusal does not depend on the dashboard behaving.

Q27. What did you build versus reuse? Reused: YOLO (Ultralytics) and ByteTrack for detection/tracking; ZeroMQ, protobuf, ed25519 (PyNaCl), SQLite; three.js for the simulation; and the original centralised re-ID thesis system (`apps/baseline.py`, MIT licence) kept frozen as the control. Built: the claim CRDT and resolver, gossip and anti-entropy wiring, hybrid logical clocks, the attestation, negative-evidence and reputation logic, the navmesh/reachability model, the query layer with capability tokens, the topology learner, the 3D world and camera rig, the dashboard, and the evaluation harnesses.

Q28. How did you test it? 497 unit tests (including Hypothesis property tests for the CRDT, clocks and resolver determinism), integration tests that start real node processes, a 4 to 64 process scaling experiment, a Byzantine sweep of 880 runs, and a 14-moment automated live review that drives the real system through every demo scenario and grades it against replayed ground truth.

Q29. What bugs did building it uncover? A node held its own claims at full float precision while its peers held the gossiped float32 form, so the *same* claim set could resolve differently on the origin node (fixed by resolving the canonical wire form). Recognition ran only on the gate camera. Two cameras naming one person 2 to 3 m apart looked like a conflict (fixed with a measured tolerance). A numpy thread pool reserved about 740 MB per node and starved the demo laptop (fixed by pinning threads).

Q30. What is the single most important design rule? Identity assignments are **never replicated**. Only signed observations are; the assignment is a deterministic function of the merged set. Everything else (partition tolerance, no coordinator, convergence) follows from that.

---

11. Repository map and how to run it

11.1 Run it

```
pip install -r requirements-world.txt
python -m playwright install chromium
python scripts/run_world_demo.py          # opens http://127.0.0.1:8765/tour; press Autopilot
python scripts/run_world_demo.py --light  # cameras at 2 fps for a crowded laptop (automatic when mem
python -m pytest -q -m "not integration"  # 497 unit tests
python scripts/review_world_demo.py       # drives all demo moments and writes review/world/
python scripts/run_scale_experiment.py    # the 4 to 64 process scaling experiment
```

Keep **about 4 GB of memory free** (close other programs). The dashboard shows an *overloaded* banner if the cameras fall behind.

11.2 Where things live

Path	What it is
`apps/node.py`	**the node process**: perception loop, gossip, anti-entropy, reputation loop, control endpoint (`/replica` , `/shutdown`)
`apps/world/`	the 3D world server, camera rig, central comparison server (`central_ingest.py`), 3D pages
`apps/demo_dashboard/`	the dashboard: `engine.py` (state), `showcase.py` (answers, roll call, comparison), `world_view.py` (names, unknown people, alerts), `server.py` (routes), `static/` (all pages)
`apps/dashboard/observer.py`	the passive gossip listener (read-only observer, ring buffer of overheard envelopes)
`apps/baseline.py`	the **frozen** original centralised system (control condition; never modify)
`packages/starling_crdt/`	C1: claim set, resolver, forks
`packages/starling_net/`	signed gossip, anti-entropy, hybrid logical clocks, keys, partition tracking
`packages/starling_consensus/`	C2: plausibility, contradiction, reputation, attacks
`packages/starling_attest/`	C4: attestations, admission, negative evidence (`CandidateBelief`)
`packages/starling_geometry/`	C3: calibration (homography), navmesh, reachability
`packages/starling_topology/`	C6: topology learner
`packages/starling_query/`	C5: capability tokens, answers
`packages/starling_perception/`	detector, tracker, appearance, registry, coverage assessor, world pipeline
`packages/starling_world/`	world config, simulation, camera model, frame transport
`packages/starling_store/`	per-node SQLite store (`LocalStore`)
`packages/starling_proto/`	wire schema (`starling.proto`) and codecs
`packages/starling_eval/`	evaluation harnesses
`configs/`	all thresholds (Pydantic models loaded from YAML), per-node configs, calibrations

Path	What it is
`data/floorplan/`	floor-plan GeoJSON files
`scripts/`	launcher, calibration, evaluation, scaling experiment, review
`deploy/`	`docker-compose.yml` (one container per node, own volume) and `netem` fault injection
`docs/`	results documents (C1, C2, C4, C6, scale, perception), this guide, the presenter playbook
`review/world/`	automated review report and screenshots
`STATUS.md`, `STARLING_BUILD_STATE.md`	project memory and the audit/work-package history

11.3 The standing design rules (from `CLAUDE.md`)

1. A node is an OS process with its own SQLite replica and gossip socket. Never a thread.
2. No module reads another node's database, directory or memory.
3. Raw video, frames and crops never cross the wire.
4. Gossip goes to a configured neighbour set, never a full mesh.
5. Identity assignments are never replicated; signed claims are, and the assignment is a deterministic function of the merged set.
6. When two claim chains bind one identity to incompatible places and both remain reachable, the fork stays open.
7. Silence is never evidence of absence; only an attestation above the confidence threshold counts.
8. The dashboard is a read-only gossip observer with no privileged access.

11.4 Further reading in this repository

- `docs/JUDGE_DEMO.md`: a presenter playbook with the 11-step script and likely questions.
- `docs/LIVE_WORLD.md`: how the live 3D world is built and what is real versus simulated.
- `docs/results_scale.md`: the full scaling experiment.
- `docs/results_c1.md`, `results_c2.md`, `results_c4.md`, `results_c6.md`: results for each contribution, each with its own honest-scope note.
- `docs/results_world_perception.md`: measured perception numbers.
- `review/world/review_summary.md`: the automated live review.
- `STATUS.md`: the running log of decisions and what changed in each session.